

Dexterous Manipulation, Single-Hand and Bimanual: Machines, Simulators, and How Policies Are Trained

Luai Abuelsamen

Abstract—A hand is dexterous when it can change an object’s pose without putting the object down. This survey covers that problem for one hand and for two, across hands, simulators and the contact models underneath them, training methods and evaluation. The corpus holds 221 bibliography entries, 218 of which carry a structured row read from a note: 112 method papers, 33 hands, 15 simulators, 15 datasets, 14 benchmarks, 14 surveys, 8 tactile sensors and 7 evaluation protocols.

Three findings are measured rather than asserted. Papers disagree with their own released code. Sixty-two method rows released code that could be read against the paper, 38 of those record a discrepancy, and nine are contradictions where the shipped code states a different objective from the published one. PhysHOI [1] is the sharpest case. Its `compute_humanoid_reward` sets the object rotation errors to zero while the reward table in the paper weights them at 0.1 and 0.01, and its position-only success criterion could not have detected that. Closed-loop policies do not measure the quantity most specific to a hand. Eleven of the 96 method rows whose notes settle the question address interpenetration at all, four of them inside a closed-loop policy, and we found none that reports it for its own policy’s rollouts. Grasp synthesis and hand-object reconstruction have reported penetration depth and intersection volume comparatively for years, so the gap is specific to learned closed-loop control rather than to the field. IsaacGymEnvs already computes that depth and gates a policy update on it. Hardware and published work have come apart. 18 of the 33 hand rows appear in no method row, and 7 of those can be bought or built today.

This survey re-runs no method. It ranks nothing and publishes no leaderboard. On penetration it supplies a measurement method and a count, not a threshold. Every coverage statistic here is a floor over what this extraction captured, not a rate over what the literature reported.

Index Terms—Dexterous manipulation, multi-fingered hands, bimanual manipulation, reinforcement learning, imitation learning, physics simulation, contact modelling, benchmarking, reproducibility.

I. INTRODUCTION

A hand is dexterous when it can change an object’s pose without putting the object down. That property, and not the finger count, is what separates a hand from a gripper. Bicch [2] draws the same line in Sec. 1.1, between restraining an object and “manipulating objects with fingers, in contrast to manipulation with the robot arm”. Restraint is a static question about whether the contacts prevent motion. In-hand manipulation is a dynamic question about whether contacts can be broken and remade while the object stays held. An et al. [3] put the same idea in their abstract as the ability “to skillfully control, reorient, and manipulate objects through precise, coordinated finger movements and adaptive force

modulation”. Both definitions place the work in the fingers rather than in the arm.

The analytic theory answered the static question and stalled on the dynamic one. Form closure has a first-order test on the grasp matrix and known contact counts, four in the plane and seven in three dimensions for any polyhedron, per [2] Sec. 1.3.1. Force closure adds the wrench balance and the hand Jacobian. Neither delivers what a controller needs. The same chapter states in Sec. 1.5 that “force closure does not guarantee stability”, and its Sec. 1.8 names the reason the theory could not be pushed further, which is that “the nonsmooth nature of grasp dynamics, because of the unilateral constraints on displacements and forces, has made a thorough analysis very difficult”. That verdict is one architect’s, on one chapter, and this survey did not survey the tradition it judges. Learned control did not solve those modelling problems. It went around them by sampling a simulator instead of solving a model, and by scoring a rollout instead of certifying a configuration. Of the 112 method papers in this corpus, 53 train with reinforcement learning and 36 run in Isaac Gym, against 6 on its successor Isaac Lab.

Three things this survey measured are worth stating before the reader commits to 27,000 words. The first is that papers disagree with their own released code. Sixty-two of the 112 method rows released code that could be read against the paper, 38 of those record a discrepancy, and nine are contradictions where the shipped code states a different objective from the published one. The sharpest case is [1]. Its `compute_humanoid_reward` hardcodes the object rotation and rotation-velocity errors to zero, with the real computation commented out beside them, while the reward table in its own paper lists weights of 0.1 and 0.01 for exactly those terms on GRAB. Its position-only success criterion could not have caught that, and its headline 95.4 percent is cited as a baseline. Section 5.8 classifies all 38, and seven further accusations an earlier draft made were withdrawn under adversarial review and recorded beside the charge. Fig. 1 draws the narrowing.

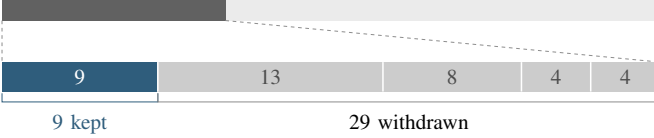
The second is that the quantity most specific to a hand is the one closed-loop policies do not record. Eleven of the 96 method rows whose notes settle the question address interpenetration at all, seven of the eleven do it outside a closed-loop policy in a grasp synthesiser, a trajectory optimiser or a contact model, and we found none that reports a penetration number for its own trained policy’s rollouts. The claim is about learned closed-loop control and not about the field. Grasp synthesis and hand-object reconstruction have reported penetration depth and intersection volume as comparative columns for years, and four rows of this corpus do it: OakInk [4] tabulates penetration

Corpus, notes and build at the repository accompanying this survey. Every claim in this paper traces to a structured note taken from a parsed source, and every count is recomputed from those notes at build time.

112 method rows in the corpus

62 released code that could be read against the paper

38 of those record a paper/code disagreement



kept: the shipped code states a different objective

withdrawn: 13 a limit of this survey's own parse, 8 the component was never released, 4 version skew, 4 a paper disagreeing with itself

Fig. 1: Papers disagree with their own released code, and most of the disagreements are this survey's own limits rather than theirs. Of the 112 method rows, 62 released code that could be read against the paper and 38 of those record a disagreement. The lower bar reopens those 38 at their own scale: 9 are contradictions, where the shipped code states a different objective from the published one, and 29 are withdrawn, most of them about this survey's own parse or about what a repository shipped rather than about the work. Further accusations an earlier draft made were withdrawn under adversarial review, each recorded in the accused row's `mismatch_review` field beside the charge. Section V-H classifies all 38 and names them.

depth, solid intersection volume and simulation displacement across methods, BiDexGrasp [5] prints penetration depth beside a prior method's, BimanGrasp [6] fails any grasp whose total penetration exceeds 1.5 mm, and TopoRetarget [7] reports a maximum penetration and a share of frames past 2 mm against a baseline retargeter. Every one of those numbers scores a pose or a reference trajectory rather than the behaviour a trained policy produced, and it is the rollout that is missing. The obstacle is not the engines. NVIDIA's own IsaacGymEnvs repository already computes a per-environment maximum interpenetration depth in Warp and gates the policy update on a 1 mm threshold. Section 7.3 has the file and the lines.

The third is that hardware and published work have come apart. 18 of the 33 hand rows in Table IV and Table V appear in no method row, and 7 of those can be bought today or built from published designs. Thirty-five method rows run on the Allegro, whose weight, joint torque, payload and price have no reachable source, because its product page returns HTTP 404 and everything Table IV confirms about it comes from its ROS driver.

None of those three findings is a first, and the audit behind the first of them is not a new idea. Collberg and Proebsting [8] examined 601 papers in computer systems research for whether the code behind them could be obtained and built at all. Xu et al. [9] align 48 bioinformatics projects with their publications at sentence-to-function granularity under expert annotation, and SciCoQA [10] collects 92 real paper-code discrepancies, mined from issue trackers and reproducibility reports, into a benchmark for detecting such discrepancies automatically. In reinforcement learning the phenomenon itself is a known result. Engstrom et al. [11] show that code-level optimisations present only in the implementation account for most of PPO's reported gain over TRPO, and Meta-World+ [12] finds undocumented changes accumulated across one benchmark's own versions,

which make comparisons between those versions unfair. Both establish it on a single codebase. The closest relative to the audit here is Knox et al. [13], who review nineteen reinforcement-learning publications on autonomous driving, characterise the reward functions of ten of them exhaustively in a standard form, apply eight sanity checks and report near-universal flaws in reward design. Their ground truth for what each reward was is the authors, obtained through correspondence with them rather than by reading a released repository, and what they establish is that published reward descriptions are incomplete: one of the ten described its reward, discount factor, termination conditions and timestep thoroughly. Reading the code instead needs no correspondence and supports a different charge, which is that where code exists it sometimes contradicts the description. Raff [14] took the opposite ground truth on purpose, reimplementing 255 papers from their text alone and never opening the authors' code, which is what makes the choice of arbiter a position rather than an accident.

So the claim here is narrow. We are aware of no prior work in robotics, and none in dexterous manipulation, that reads a field's released reward implementations against the rewards its own papers describe. Those seven comparisons are cited from outside this corpus and enter none of its counts. The exposure of the claim belongs beside the count above: the evidence is a repository at a fetched commit, each hash recorded in `corpus/code_manifest.json`, and a repository at a commit is evidence about that repository rather than about the run that produced a paper's numbers, because the commit may postdate, precede or diverge from it. Hora [15] states the problem in its own README, which directs a reader to tag v0.0.1 and not to the commit parsed here to reproduce the published numbers. The seven accusations an earlier draft withdrew are recorded in the accused rows for the same reason: they are the evidence that the charges left standing were checked rather than counted.

Fourteen corpus entries are themselves surveys or engine-comparison studies. Appendix D says what each covers, on one set of columns tabulated in the repository. Four of the fourteen could not be obtained, or were fetched too late to read, and are entered as such. Two of the three things this survey adds are columns none of the fourteen fills. The first is Table VI, which takes the engines [16] scored on documentation and usability and adds contact model, solver, iteration count, default timestep and penetration exposure, conditioned on what a hand does to a solver. Erez et al. [17] and Le Lidec et al. [18] do measure engines, on five engines and on four contact formulations, and neither surveys the field those engines are used in. The second is two hands on one object as its own problem, which none of the four field surveys gives more than a subsection.

The third is the penetration measurement, on an axis a predecessor had already named. Zhao et al. [19] state in their Sec. IV-C that the field assesses "at least two layers of performance: the quality of grasps or poses prior to execution, and the performance of policies or generators during downstream execution", and names "physical plausibility, including penetration" as the most common criterion for the first layer. That is the reference-versus-rollout split, named before this survey measured it, and the idea that contact quality

belongs on the evaluation axis is Zhao's. What Sec. IV-C does not give is a measurement method or a count, and this survey supplies those two. It does not supply a threshold. The 2 mm figure the field uses is taken from [7] with no independent justification, the captured human grasps in [20] sit above it at 3.25 mm, and Section 7.7 states plainly that 2 mm is a simulator convention rather than a physical bound.

The scope is single-hand and bimanual multi-fingered manipulation. Parallel-jaw manipulation enters only as a comparison, which matters because 12 of the 112 method rows name a parallel-jaw gripper among their own embodiments. Locomotion is excluded. Prosthetics are excluded except where a hand crosses over into robot use, as the Psyonic Ability Hand does. The corpus holds 221 bibliography entries, of which 218 carry a structured row read from a note. Those 218 rows are 112 method papers, 33 hands, 15 simulators, 15 datasets, 14 benchmarks, 14 surveys, 8 tactile sensors and 7 evaluation protocols. "Method row" throughout means one of the 112, and every headline count in this survey has one of those eight classes as its denominator.

Two limits apply to every number here. This is a corpus of the learned era, which is a selection effect and not a judgement: 138 of the 221 entries are dated 2024 or later, 16 predate 2018, and two of the 112 method rows predate 2018, so any statement about a trend over time is a statement about 2022 onward. The analytic tradition is represented by one readable chapter rather than surveyed, and the planning line that took up the dynamic question directly, finger gaing and rolling-contact manipulation and regrasp planning, is not in the corpus at all. Every coverage statistic is also a floor rather than a rate, because a value this survey failed to extract is indistinguishable from a value the paper never reported, and every such miss converts a reporting paper into a silent one.

Fig. 2 puts the field on one page. Section 2 sets out the task families and what makes each hard. Section 3 covers hands, their makers, and the gap between what is sold and what is run. Section 4 covers simulators and contact models. Section 5 covers how policies are trained. Section 6 covers bimanual work as its own problem. Section 7 proposes an evaluation frame rather than a leaderboard, and is the longest section. Section 8 states the gaps as claims with their evidence. Section 9 says what to do about them, addressed to someone publishing, running experiments or buying a hand. Appendix A is the method, and says where the tabulation this survey is built on lives, which is the repository rather than these pages. Appendix B carries the three tables Sections 3 and 4 read cells out of, Appendix C is the reward extraction, and Appendix D is the comparison with the existing surveys.

II. A TAXONOMY OF THE PROBLEM

A. Task families

The six families below are read off the rows rather than imposed on them, and Sec. 2.1's closing paragraph says what they miss. Grasping is the largest. Fifty-seven of the 112 method rows carry the grasp label, the labels are not exclusive, and one paper can sit in several families. Success is a lift that survives a hold, and the thresholds differ by more than an order of

magnitude. DexGraspVLA [21] requires the object "held 10 cm above the table for 20 s", while Omnigrasp [22] requires it "held at least 0.5 s in simulation". What makes grasping hard at scale is the continuum of starting configurations. UniDexGrasp++ [23] states it in Sec. 4.3, that "we are dealing with an infinite number of tasks considering the initial object pose can change continuously".

Functional and tool use is second with 45 rows. It is the family where a stable grasp can still be the wrong answer. Dexterous Functional Grasping [24] gives the case in Sec. 2.1, that "grabbing a hammer from the head or handle are both equally valid ways of using it", and only one of the two lets the tool be used. Success is defined against the tool's function, and the field has no shared way to state it.

In-hand reorientation has 31 rows and the most settled criteria, because the community inherited one number. OpenAI [25] declared the goal achieved below 0.4 rad of orientation error, DeXtreme [26] keeps 0.4 rad at test time against a 0.1 rad training tolerance, and Eureka [27] counts consecutive successes at 0.1 rad. A shared tolerance is not a shared protocol, because the stopping rule differs. Visual Dexterity [28] measures error "when the controller predicts it has reached the goal and stops", which makes the policy a judge of its own trial.

Tracking a human reference has 12 rows and inverts the problem. The target is a trajectory rather than an endpoint, so success is a per-frame error band. ManipTrans [29] requires all four of 30 degrees of rotation, 3 cm of translation, 8 cm of joint error and 6 cm of fingertip error. The difficulty is that the reference came from a human hand and was never dynamically feasible for the robot. DexMachina [30] warns in App. B.4 that scoring by timesteps inside the thresholds makes "the results highly sensitive to the threshold values".

Bimanual coordination has 43 rows and handover has 10. Handover is the smallest family and the one whose failure has a single moment, because the giver must release only after the receiver has the object. Dynamic Handover [31] reports a hit rate above its success rate and blames the gap in Sec. 5.4 on "occasional challenges encountered during the grasping phase of the catcher". Not every paper here learns both sides. In [32] only the receiver is learned and the giver is a scripted arm.

Those six families do not cover the corpus. Twenty-four of the 112 method rows carry a label from outside them and eight carry no label from the six at all. Twenty are labelled *other*, mostly generalist policies evaluated on a task suite rather than on a dexterous task family, among them [33], [34], [35], [36] and [37]. Three adjacent families are named here rather than absorbed. Locomanipulation is [38], [39] and [40], where the base is not fixed and the gravity argument below changes character. Piano playing is [41], [42] and [43], discussed in Section 6, where success is a per-timestep F1 against a MIDI score rather than an object pose. Deformable manipulation is [44], whose object carries its own state and its own physics. Ferrari and Canny's grasp-quality measure [45] carries no task label, because it is a measure rather than a task. Table I's six rows are the families with enough papers to compare, and not a partition of the corpus.

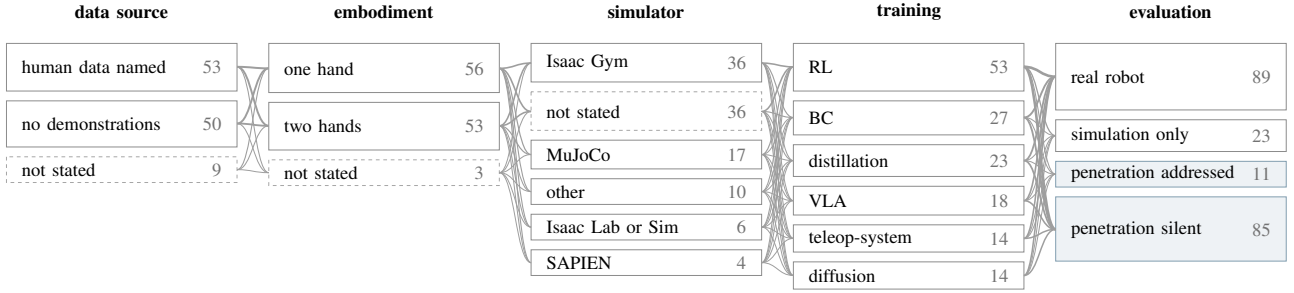


Fig. 2: The field on one page. Data sources, embodiment, simulator, training paradigm and evaluation, with nodes sized by how many corpus papers sit in them and edges drawn only for routes the corpus actually contains. The routes converge: almost everything ends in a policy trained in a GPU simulator and distilled to vision.

B. What makes dexterous control hard

Contact is non-smooth, and that is a property of the problem rather than of any solver. Bicchi [2] states in Sec. 1.2 that contact constraints are unilateral, and that losing a contact “involves an abrupt change of the structure of the model under consideration”. Its Sec. 1.4 gives the sharper version, that a rod sliding on rough ground has configurations with no consistent solution and configurations with more than one. Simulators inherit the difficulty. The re-implementation study [18] concludes in Sec. V that “there is no fully satisfactory approach at the moment, as all existing solutions compromise either accuracy, robustness, or efficiency”. Pang et al. [46] treat non-smoothness as the thing to remove, curating every contact pair in Sec. III-D so that contact points and normals change smoothly.

The hand has more actuators than the object has degrees of freedom and is still short of authority over it. The object moves only through contacts, and the contacts are what cannot be commanded. Bicchi [2] states in Sec. 1.4 that in multifingered grasps “the number of independent contact forces is much larger than the number of actuators. Thus, from a controllability standpoint, not all the contact forces are controllable”.

Vision is occluded by the hand doing the work. Bai et al. [47] name it in Sec. 4.3, that “occlusion hampers object tracking”. DexPoint [48] keeps vision and adds imagined hand points to the point cloud. Yin et al. [49] remove vision entirely and rotate objects from 16 binary touch sensors over the palm, links and fingertips.

Gravity direction changes the task rather than scaling it. Visual Dexterity [28] states that reorientation with the hand below the object “is much easier”, because “with a downward-facing hand, the hand must manipulate the object while simultaneously counteracting gravity”. AnyRotate [50] makes the direction a randomised variable by “randomly initializing hand orientations between episodes”, and its Sec. 5.3 measures the cost, with performance dropping progressively from palm up and palm down, through base up and base down, to thumb up and thumb down.

C. Single hand versus two

Three counts describe two hands and they measure different things. Fifty-three of the 112 method rows record two hands on the robot, which is the `bimanual` field. Forty-three carry

the `bimanual-coord` task label, the narrower claim that coordinating the hands is the task. Section 6 narrows again, to the 28 rows whose notes place a learned closed-loop controller on two multi-fingered hands, and its opening paragraph names every exclusion that takes the 53 down to the 28. That 28 is the denominator for every architecture count in this survey. Every bimanual claim here names which of the three it uses.

Four things genuinely change when the second hand arrives. Contact stays non-smooth, occlusion stays, and gravity stays the same problem.

Role asymmetry is the first. AsymDex [51] assigns a dominant hand with full finger and wrist control and a facilitating hand with 6-DoF base pose only, so that “the facilitating hand repositions and reorients one object, while the dominant hand performs complex manipulations”.

The second is that two hands on one object close a kinematic chain through the object. In [2] Sec. 1.4 the hand and object dynamics are separate and are “linked through the n rigid-body contact constraints”. A second hand adds a second constraint set on the same object rather than a second independent problem. That is where physical plausibility fails first. Bi-manGrasp [6] is the corpus row that measures it, rejecting a synthesised grasp when “total penetrations exceeded 1.5 mm” and reporting in Sec. IV-C that “penetration remains the primary cause of grasp failure”.

The third is two-arm collision, which does not exist for one hand. DyDexHandover [52] resets an episode when “any unintended arm contact with the environment or self-collisions” occurs. Bunny-VisionPro [53] pays for it in the controller, where adding a sphere-approximated self-collision cost raises motion-control time from 0.74 ms to 7.85 ms. Bidex [54] solves it in hardware, mounting the hands so the human arm and the teacher arm “are perpendicular to each other and do not collide”. DexMimicGen [55] states plainly that it does not handle inter-arm collision at all.

The fourth is the doubled action space. Bi-DexHands [56] runs 20 tasks on two Shadow Hands and finds single-agent PPO beating multi-agent RL on most of them, offering in Sec. 5.2 that “PPO algorithm is able to use all observations for training the policy, while MARL can only use partial observations”. AsymDex [51] attacks the dimensionality from the other side, halving the observation and action dimension through its role split and a frame relative to the facilitating hand’s object.

Two cells say the field has no agreed criterion, and both

TABLE I: Task families against the properties that define success.

task family	what is held	what moves	what gravity does	what counts as failure	shortest honest success criterion
In-hand reorientation	one rigid object, held by the fingers for the whole episode	the object's orientation in the palm frame, while contacts break and remake	a disturbance whose direction is set by hand orientation, and a load to counteract with the palm down, [28]	the object leaves the hand, e.g. the fall reset at goal distance 0.24 m in [26]	orientation error below a stated tolerance under a stated stopping rule, 0.4 rad in [25] and [26], 0.1 rad in [27]
Grasping	nothing at the start, the object rests on a support	the hand closes, then the object is lifted clear	the test itself, since the grasp exists to resist it	no lift, or a drop inside the hold window	a stated lift height held for a stated duration, 10 cm for 20 s in [21], 10 cm to episode end in [57], 0.5 s in [22]
Functional and tool use	a tool, held in the pose its function requires rather than the most stable one	the tool's own joint, or the tool acting on a second object	secondary, the function dominates	a stable grasp in the wrong place, [24]	no agreed criterion. Criteria range from never dropping the object, [24], to a four-point rubric, [58]. The one mechanical threshold is holding the tool joint past a commanded angle for the episode, [59]
Tracking a human reference	whatever the human held in the captured clip	hand and object follow a recorded trajectory under physics	full, and the reference never obeyed it, so the reference is not dynamically feasible	per-frame error leaves the band, or the object drops	all stated per-frame errors below threshold for the whole clip, 30 deg and 3 cm and 8 cm joint and 6 cm fingertip in [29]. Thresholds are not comparable across papers, and DexMachina [30] calls the results "highly sensitive to the threshold values"
Bimanual coordination	one object by two hands, or one object per hand	both hands, the object, and the relative pose of the hands	can be carried by one hand while the other acts, [51]	the object drops, the arms collide, or the hands work against each other	no agreed criterion. Task-specific thresholds dominate, e.g. 0.035 m block-to-cup distance in [51]. The one criterion with a physical-plausibility gate is 2.0 s of hold with total penetration under 1.5 mm, [6]
Handover and in-hand transfer	by the giver at the start, by the receiver at the end	the object crosses between hands, released only after it is regained	unopposed during the exchange, and total in a thrown transfer, [31]	a drop at the moment of release, and the catcher's grasp phase, [31] Sec. 5.4	the receiver holds the object and moves it clear, more than 10 cm from the first hand in [60], or holds it to episode end in [52]

are in families where a second body is involved. A criterion naming a distance, a duration and a trial count can be re-run by someone else. A rubric cannot, and Section 7 takes up what follows from that.

III. HANDS AND WHO MAKES THEM

A. The design axes

No degree-of-freedom, force, weight or price figure in Table IV or Table V was measured by anyone outside the maker. Six rows are the exception, from peer-reviewed papers with stated protocols: ILDA, Pisa/IIT, ORCA, RUKA, LEAP and BiDexHand. Table IV dates every other claim.

The degree-of-freedom count is the first number a vendor states and the least comparable one. Shadow's specification of 4 December 2024 reads "20 actuated DOF and a further 4 under-actuated movements for a total of 24 joints" [62]. Table IV prints joints first and actuated DoF second, and the gap is the informative number. Where a vendor's own DoF figure differs from the joint count, the joint count is what the table prints, as with Inspire's "Degrees of freedom 6, Numbers of joints 12" [63]. An actuator count is neither of those columns and has its own. DexHand states no joint count at all, only 16 finger micro-servos and two wrist servos, so 18 servos is the only count quotable for it [64]. Daxo states 120 actuators, no DoF figure, and a tendon structure with no rigid joints, which has far fewer kinematic degrees of freedom than actuators [65].

The actuation ratio is the real decision. The Pisa/IIT SoftHand drives 19 joints from one motor and its authors state the cost plainly: "No in-hand dexterous manipulation is required for this prototype" [67]. Tendon drive moves the motors off the fingers and the mass follows them. Shadow's hand plus forearm weighs 4.3 kg, against 1.1 kg for the ILDA

hand alone with every motor in its palm [62], [66]. ILDA states an 18 kg payload against Shadow's 4 kg in a power grasp, and Shadow states no fingertip force at all. What tendons cost is state estimation. The Faive Hand estimated joint angles from tendon length through a Kalman filter and its cube reorientation failed on hardware, which its authors blamed on "poor joint angle measurement from the EKFs, especially when there is contact" [68]. Ruka-v2 adds detachable AS5600 encoders to measure that problem rather than close the loop on it, and those encoders put its open-loop linear joint-to-motor map at 8.26 degrees of error over seven joints [69].

Direct drive trades torque density for transparency. LEAP's Dynamixel joints resist 19.5 N in a pull-out test against the Allegro Hand's 8.5 N, at 595 g for the LEAP hand [61], and the Allegro's own weight has no reachable source. Linkage drive is the argument against the forearm. ILDA puts 15 motors, drivers and ball screws inside the palm and reports 34 N at the fingertip in the bent pose and 28 N stretched [66]. Linkages buy coupling for free, as in BiDexHand's four-bar that drives each DIP off its PIP, whose paper claims 16 actuated DoF and 21 joints while its README describes 15 servos driving 15 joints [70].

Fingertip force is the number a hand buyer reads first, and across these rows it is six different measurements. Table IV puts the quantity beside the figure: pull-out resistance for LEAP's 19.5 N, pinch for RUKA's 2.74 N and BrainCo's 15 N, fingertip normal force under a 1 cm indenter for Unitree's 10 N, a calibrated five-trial fingertip force for BiDexHand's 2.14 N, a peak vendor claim for 1X's 45 N, and two unlabelled spec columns for Sharpa's "20 N | 12 N". BrainCo's whole-fist grip of 50 N is a separate figure and is no longer in the same column as a pinch. ORCA's 19.6 N was none of these. It is the

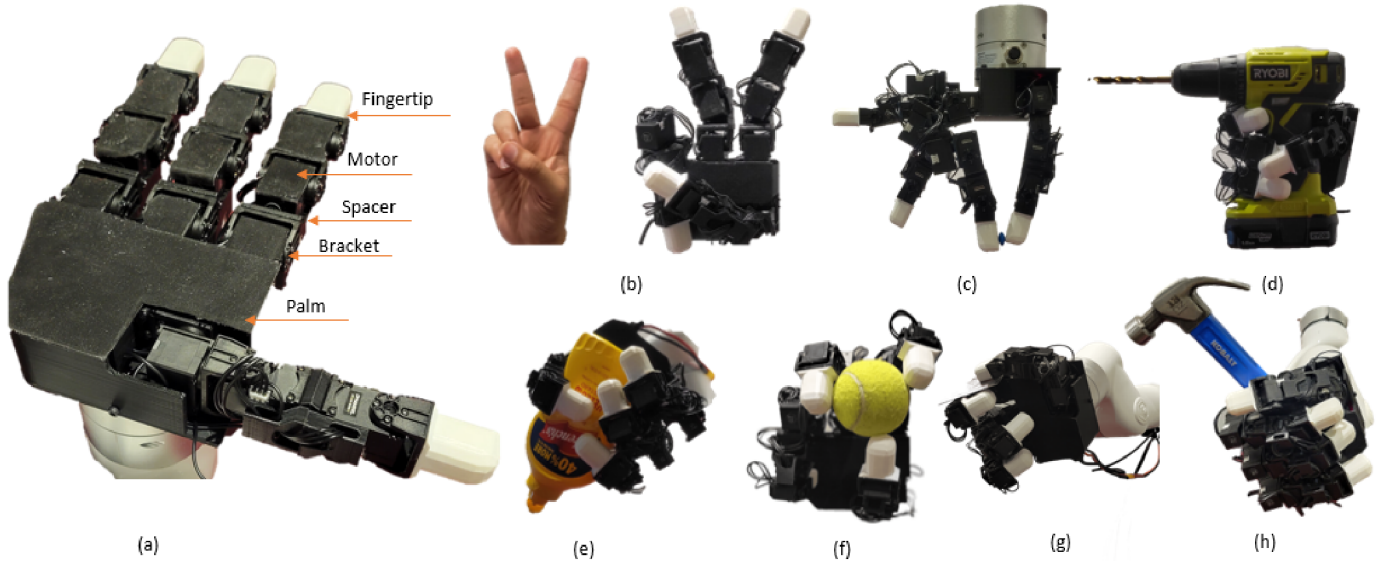


Fig. 3: A low-cost hand in which each actuator is a joint: the motor bodies are the phalanges, and the bracket-and-spacer stack sets the kinematics. The same hand in power and precision grasps on everyday objects.

Reproduced from [61]; permission not yet sought, see the register.

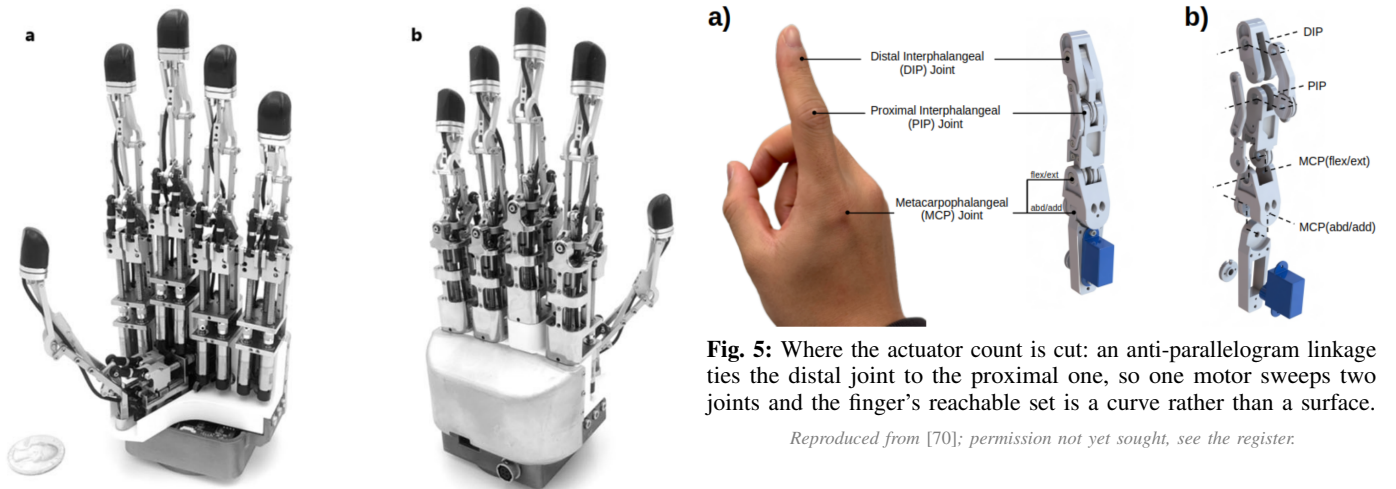


Fig. 4: A fully actuated linkage hand with the transmission left exposed: every joint is driven through its own metal four-bar and lead screw, with a coin for scale. Greyscale; the source photograph is near-monochrome [66]. The journal’s CC BY licence excludes anything marked there as third-party material; none is flagged on this figure, but the author should confirm.

Reproduced from Fig. 5 of U. Kim et al., “Integrated linkage-driven dexterous anthropomorphic robotic hand,” Nature Communications 12:7177 (2021),

DOI 10.1038/s41467-021-27261-0, licensed under CC BY 4.0; panels (a) and (b) only, converted to greyscale.

newton equivalent of a 2 kg index-finger payload at a fixed 600 mA motor current, so its force cell is empty and its payload cell carries that protocol [71]. No two hands in Table IV report payload under the same test either.

Compliance is reparability seen twice over, and both are mechanical fuses: the Pisa/IIT SoftHand’s rolling-contact joints return to assembly after over-extension and ORCA’s “poppable” pin joints dislocate rather than break [67], [71]. ORCA also reports tendon drive as maintenance: “Prolonged use requires

Fig. 5: Where the actuator count is cut: an anti-parallel linkage ties the distal joint to the proximal one, so one motor sweeps two joints and the finger’s reachable set is a curve rather than a surface.

Reproduced from [70]; permission not yet sought, see the register.

manual re-tensioning to maintain performance”. What fails is rarely the motors. Its durability run found silicone skin degrading on two fingertips after about 2,000 to 4,000 grasp cycles and sensor wires snapping on three after about 4,500 to 7,000. The sensing wore out an order of magnitude sooner than the hand.

Two columns matter before any of the above and were missing. The first is the rate a loop can be closed at, which spans 250 Hz on a Tesollo to 1 kHz on a Shadow, a Unitree or a Wuji, with the Allegro at 333 Hz on its own CAN clock and Inspire stating no rate at all. LEAP’s 500 Hz is a serial query ceiling and its own sim-to-real policy runs at 20 Hz, which answer different questions [61]. A reader deploying torque control is bitten by that first. The second column says whether a URDF or MJCF exists and who ships it, which is the bridge to section 4 and to this survey’s recommendation to pick a hand a simulator already carries. Shadow, Sharpa, Wuji, the Ability Hand, RUKA, Faive and the Allegro driver ship one. LEAP is the warning: its paper says the URDF is released and

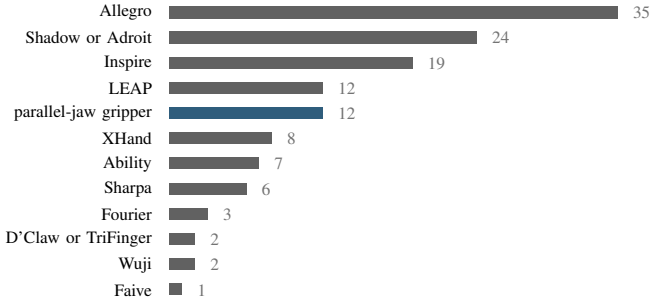


Fig. 6: Which hands the literature actually runs on. Per hand, the number of method papers whose own experiments use it, over the 103 of the 112 method rows that name a hand; a paper using two hands is counted under both. The accented bar is the parallel-jaw gripper, which is not a dexterous hand, and only three dexterous hands carry more method rows than it. Only hands with at least one method row appear, so the nine company-announced hands of Table V and the hands of Table IV that no paper runs on are absent from the chart.



Fig. 7: Five research hands photographed to scale beside an adult human hand and a ruler. A count of actuated degrees of freedom says nothing about the size of the hand that carries them.

Reproduced from [61]; permission not yet sought, see the register.

the released API repository contains none [61].

B. The hands the research literature actually runs on

The two oldest designs in Table IV carry 49 of the 103 method rows that name a hand at all, and seven rows use both. Neither design's date is confirmed by its own sources, so 2005 and 2016 are the bibliography's. Fig. 6 counts, per hand, the method papers whose own experiments use it. Of 112 method rows, 103 name a hand. The Allegro accounts for 35, Shadow for 21, the Inspire RH56 family for 19, a parallel-jaw gripper for 12 and LEAP for 12. Seventy-five of the 103 name an Allegro, a Shadow or Adroit model, LEAP or an Inspire.

The Allegro's position is the uncomfortable part. Its product page at allegrohand.com/v4 returned HTTP 404 while the Wonik wiki timed out [72]. Everything Table IV confirms about the most-used hand in the corpus comes from its ROS driver: 16 joints, four fingers, a torque interface, a 333 Hz CAN clock, 12 V, no tactile sensing. Its weight, joint torque, payload and price have no reachable source. The field's most common platform cannot be specified from a page a reader can open.

Concentration would matter less if the hand did not move the result, and it does. On the same simulated cube rotation, LEAP reaches 0.2288 rad/s against the Allegro's 0.0828 rad/s [61], and RUKA reports a 2.74 N pinch against the Allegro's 1.60 N under the same three-trial pinch test [73]. A method compared only on Allegro hardware is compared at one point in a space where a single axis moves the headline number two or three times over. Two further patterns follow. Inspire, XHand and Sharpa take 29 of the 103 rows between them, four rows use two of the three, and none is earlier than 2024. ORCA, RUKA, Ruka-v2, BiDexHand, DexHand, the Tesollo DG-5F and the Unitree Dex5 take none at all.

C. Open hardware and the collapse in cost

Seven rows in Table IV are open hardware. Five state a dollar cost in the price column, ORCA states a material cost below 2,000 CHF that the column leaves unconverted, and BiDexHand states none [71], [70]. The Faive Hand states neither a cost nor a licence that could be read, so it is not counted as open hardware here [68]. Two of the stated bases need saying. DexHand's \$300 is "additional total cost of components", excluding the printing and the wrist servos [64], and ORCA's own figure sits against Ruka-v2's table listing ORCA at about \$3.5K [69].

The expensive end of the collapse is secondhand throughout. The only six-figure numbers on disk are RUKA's comparison table at \$100,000 for a Shadow Hand and Faive's "steep price tag of 110k GBP" [73], [68]. Shadow's own page says to discuss pricing and Table IV's price cell for it is empty. The collapse is real at the cheap end and secondhand at the expensive one, and it has barely moved the literature. Twelve of the 103 hand-naming method rows use an open-hardware hand, and all twelve are LEAP.

What the cheap hands give up is sensing. LEAP has none and names touch sensors as future work [61], RUKA states its design "lacks tactile sensing" [73], and BiDexHand's sources never mention touch [70]. ORCA is the exception, with binary FSR fingertips whose threshold was measured as low as 0.05 N on a fresh fingertip, against the sensor's rated 0.29 N, and 6.38 N on a degraded one [71].

D. Announced and unreleased hands

Behind Table V's rows sit three kinds of evidence, and conflating them is how a DoF figure with no source ends up in a survey. Vendor prose carrying numbers is the strongest, as on 1X's page of 9 July 2026 [74]. Video is second and carries none. Press or bibliography assertion is third, as with Tesla's V3 hand, known here only through a paraphrase of patents because the USPTO PDF parsed empty [75]. Table V blanks

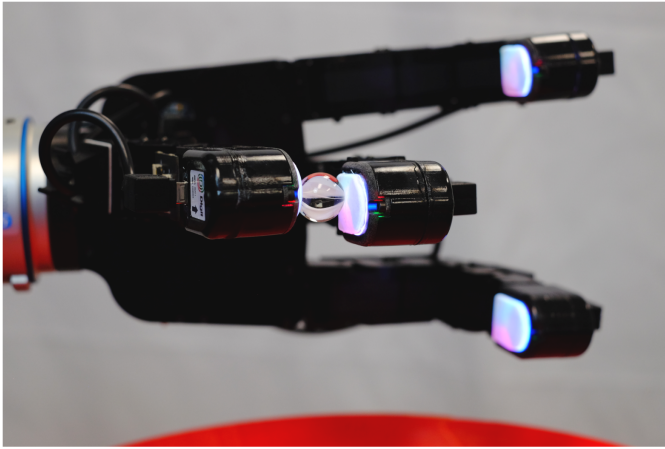


Fig. 8: Vision-based tactile sensors used as the fingertips themselves: each unit is a camera and an illuminated gel, and the sensor body sets the fingertip’s size and shape.

Reproduced from [80]; permission not yet sought, see the register.

the cells resting on the third class and footnotes who did the arithmetic.

A survey can go one step past recording that a claim is unverified, which is to say which claims are implausible on their face. Daxo’s 120 actuators in 750 g is about 6 g per actuator including structure, tendons, routing and skin [65]. Clone’s 27 degrees of freedom under 2 pounds excludes a 500 W pump the source does not confirm is excluded [76]. Figure 03’s “Degrees of freedom, hands | 20” sits on the same tracker page as “Number of fingers | 10”, so it is almost certainly the pair [77]. Tesla’s repeated 22 is one article’s arithmetic of four DoF on each of five fingers plus two at the wrist, and Gen 2’s own figure was 11 [75].

Scepticism belongs to the evidence class, not to which table a row lands in. Sharpa’s 22 of 22, Wuji’s 20 of 20 and Tesollo’s 20 of 20 are vendor claims about unmeasured hardware and they sit in Table IV, where Sharpa’s page footnotes “Specifications may vary between products” and Wuji’s says the spec “will continue to iterate” on a Beta1 product [78], [79]. Even the best-specified vendor page in the corpus leaves its fingertip-force columns unlabelled.

E. Tactile sensing as part of the hand

DIGIT set the cost floor. It is 20 by 27 by 18 mm, weighs about 20 g, streams 640x480 at 60 fps, and its paper states a “total estimated manufacturing cost is approximately 15 USD per sensor ... when manufactured in a batch of 1000” [80]. Durability was as much the contribution as price: its gel degraded 0.3 percent over 15 abrasion passes, against 805 and 918 percent for the two gels compared with it.

Digit 360 is the same lineage at a different operating point, claiming about 8.3 million taxels, spatial features to 7 μ m, normal and shear force resolution of 1.01 mN and 1.27 mN, and on-device processing that cuts event-to-action latency from 6 ms to 1.2 ms [81]. It is also not for sale, and no corpus paper’s parsed text names it.

XELA’s uSkin is the non-camera alternative and the one that bolts onto hands the field already uses. Its curved fingertip kit

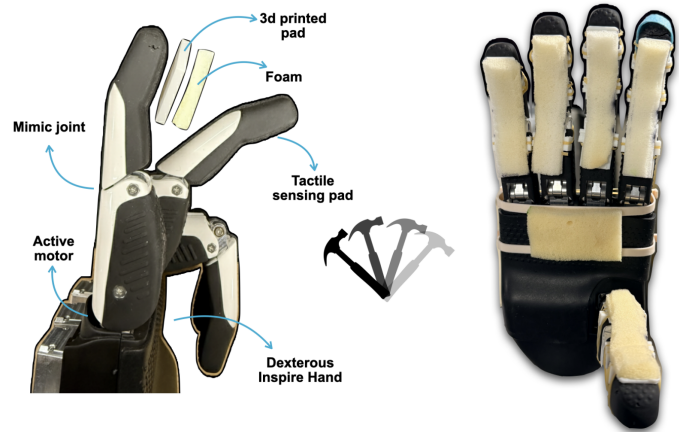


Fig. 9: Compliance added after the fact: a printed pad over a foam layer, fitted to a commercial hand to spread contact that the rigid shell concentrates. The right panel shows the whole hand once padded [59].

Reproduced from Fig. 4 of S. Atar, D. Huang, F. Richter and M. Yip, “In-Hand Manipulation of Articulated Tools with Dexterous Robot Hands with Sim-to-Real Transfer,” arXiv:2509.23075, licensed under CC BY 4.0; cropped to the right half of the page region.

for the Allegro V4 and LEAP carries 30 three-axis sensing points, a full Allegro integration reaches 368, resolution is 0.1 gram-force, and the kit runs at 275 Hz [82]. The 500 Hz figure often quoted belongs to the product family, not the curved fingertip. One corpus paper names it, [83], whose self-supervised encoders are pre-trained on 462.7k tactile images and beat matched end-to-end models by 95.1 percent when both see 33 to 50 percent of the labels. Its own bead-maze policies never complete the maze on the real robot.

The usage number is the one to keep, and it needs its inclusion rule stated: a sensor physically on the hand, read by the deployed policy. Eight of 112 method rows meet it: [50], [59], [84], [85], [60], [86], [87] and [49]. The rule excludes [88], whose 20 binary contacts are simulated on an Allegro that has no tactile hardware and whose released config sets `enable_tactile: False`, and it excludes [89] and [90] for the same reason. Sixty-five of the 112 method papers mention tactile sensing somewhere, which is the gap worth quoting. Thirty-five is the count over this survey’s notes.

Almost none of the eight uses a high-resolution sensor. Yin et al. [49] remove vision entirely and rotate objects from 16 binary touch sensors over the palm, links and fingertips, deployed zero-shot to a real Allegro, and Robot Synesthesia uses 16 force-sensing resistors read as binary [86]. Two papers exclude touch deliberately, HORA reporting rotation “even without the usage of vision and tactile sensing” [15], and ORCA’s authors dropping it from their reinforcement learning “due to the additional complexity involved in accurately modeling them” [71]. Taxel counts have risen by three orders of magnitude while the policies consuming them have stayed at binary contact.

F. What is sold against what is published on

The hands missing from Fig. 6 carry the finding. None of the nine company-announced hands in Table V appears in a single method row whose own experiments use it. Tesla, Figure,

1X, Sanctuary, Boston Dynamics, Xiaomi, Clone, Daxo and PaXini account for zero of the 112 method papers’ experiments. The nearest thing to a counterexample is [91], a Figure blog post claiming a 35-DoF whole-upper-body action space at 200 Hz that includes individual finger control. It never names the hand, gives no per-hand DoF count, and reports no success rate or trial count for any task. It is the maker describing its own unreleased hand, which is the evidence class the finding is about. The four research prototypes in Table V are in a different position, since the Faive Hand and LEAP Hand v2 Advanced account for three method rows between them [57], [54].

Table V’s emptiness is measurable and part of the same finding. Over the twelve specification columns, 61 percent of its cells are values no source stated, against 39 percent for the hands that can be bought. The hands with the highest advertised DoF counts have the least specification behind them.

The one hand that crosses the gap crosses it because its maker published rather than announced. ByteDexter reaches a method row through a ByteDance technical report stating 21 DoF per hand, 16 piezoresistive fingertip channels in the action vector and real-robot success rates [92], not through a product page. That mechanism is available to every vendor in Table V and none has used it.

The gap runs the other way too, and the last column of Table IV shows it. Seven hands that are sold and documented take zero method rows each: the Unitree Dex5 with 94 pressure sensors on its P variant [93], the fully actuated Tesollo DG-5F [94], ORCA, RUKA, Ruka-v2, BiDexHand and DexHand. Cheapness is established only for RUKA, Ruka-v2 and DexHand, because Table IV’s price cell is empty for Unitree, Tesollo, ORCA and BiDexHand, and the only Dex5 price on disk is Ruka-v2’s secondhand “~\$25K” [69]. A reader choosing a hand on this corpus’s evidence has two well-precedented options, an Allegro or an Inspire, and a third in LEAP if eleven papers is enough. Everything else is a press release or a hand nobody has published on.

IV. SIMULATORS AND THE PHYSICS UNDERNEATH

A. What a dexterous simulation must get right

The clearest demonstration that hands are the hard case came from a benchmark that was not about hands. Erez et al. built a 35-DOF arm modelled on the Shadow Hand, closed it around a capsule with fixed spring-dampers, and asked four of the five engines for the largest timestep at which the object was still in the hand. MuJoCo held the grasp at 16 ms, PhysX at 2 ms, ODE at 0.25 ms and Bullet at 0.03 ms, a spread of a factor of 500 [17, Sec. IV-D]. Havok is the fifth and was excluded for want of a working PD controller. Three caveats travel with that spread. The engines run a deliberately restricted common model, hinge joints with sphere and capsule geometry, no boxes and no meshes (Sec. II). The authors call it “an open question whether this model system can be tuned to work better in the gaming engines” (Sec. IV-D). The timesteps are log-spaced and good only to a factor of two, and the authors wrote MuJoCo and disclose it.

A grasp is hard for nameable reasons. It is many contacts at once, all persistent, all near stiction, on an object much lighter



Fig. 10: The object the policy touches is not the object it sees. Left, the visual mesh; centre, its single convex hull; right, the convex decomposition that restores the concavities contact needs.

Reproduced from [96]; permission not yet sought, see the register.

than the mechanism holding it. Persistence and multiplicity make the problem hyperstatic, and Le Lidec et al. show that per-contact solvers of the projected Gauss-Seidel family, and RaiSim’s, then inject spurious jamming forces at stiction that vanish only once the object slides. The mass ratio makes it ill-conditioned, and in their stacked-cube test at a 10^3 to 10^{-3} kg ratio those same methods fail to converge. Global methods with proximal regularisation stay robust in both cases [18, Sec. IV-A]. What degrades a truncated solve is therefore conditioning and redundancy, not stiffness, and a grasp supplies both by construction. Locomotion is forgiving by comparison. On flat ground the contact model and solver choice “hardly affects” their quadruped’s tracked base velocity, and only on rough, slippery terrain do RaiSim and CCP deviate. A walking robot makes and breaks a few contacts against ground far heavier than itself. A hand does neither.

Speed enters by the same door. MuJoCo Playground reports that contact time scales with the number of possible contacts rather than the active ones, because JAX requires static shapes, which is why its tasks carry hand-tuned `max_contact_points` and `max_geom_pairs` overrides [95, Sec. VI]. Every override printed in that paper is a locomotion port. A hand’s bound would have to be set the same way, for the worst case, but no hand configuration is printed.

Fig. 11 sets out the stages of one simulation step. Three of them make overlap, and they do not answer to the same knob. In an engine that enforces non-penetration at the velocity level, integration turns any residual approach velocity into overlap of order v times Δt ; an engine that enforces the gap at the next configuration carries no such term, which is why Dojo’s hard-contact NCP keeps its feet above the floor at every timestep it was tested at. Constraint assembly fixes the compliance a loaded contact then rests at. A truncated solver leaves a residual that grows with conditioning. Which of the three dominates in a grasp is not measured anywhere in this corpus, and the three are not ordered here. A fourth item on the figure is not a source of overlap at all. It is a mismatch between the geometry the solver uses and the geometry the renderer draws, and it runs in both directions.

B. Contact models and solvers, engine by engine

Table VI is the engine-by-engine comparison, one row per simulator, built only from what a note confirmed. Le Lidec et al. supply the taxonomy that organises it, checking each



Fig. 11: One simulation step, and where engines differ. The stages every rigid-body engine runs, and under each the choices that separate the engines used for hands. The dashed arrows mark the three stages that make overlap. They answer to different knobs, and which of them dominates in a grasp is not measured anywhere in this corpus.

formulation against the Signorini condition, Coulomb’s law, and the maximum dissipation principle. Linear complementarity, the family of Bullet, ODE, PhysX and DART, satisfies Signorini alone, because linearising the friction cone to a pyramid biases friction toward its corners. The cone complementarity problem satisfies the other two but relaxes Signorini, so contact acts at a distance of size $\Delta t \cdot \mu \cdot \|c_T\|$. The full nonlinear problem satisfies all three and is non-convex [18, Table II]. A contact model is not the algorithm that solves it. PhysX linearises the cone, which places its model in the LCP family, and it does not solve an LCP. Its Temporal Gauss-Seidel scheme folds substepping into the Gauss-Seidel sweep and exposes position and velocity iteration counts separately [97, Sec. 3].

MuJoCo is inside that taxonomy rather than outside it. Le Lidec et al. classify it as CCP-MuJoCo, a convex relaxation solved by a Newton method on the primal QCQP, in the same family as CCP-Drake, the SAP-style scheme (Table III). The relaxation makes two errors of opposite sign in one engine. A loaded contact carries a violation, and a sliding contact carries force at a positive gap. Drake’s SAP inherits the same pair, and its gliding effect at distance $\varphi \approx \delta t \cdot \mu \cdot \|v_t\|$ “unfortunately does not go away as $\delta t \rightarrow 0$ ” [98, Sec. V-A]. Le Lidec et al. call that compliance a “numerical trick designed to circumvent the issues due to hyper-staticity or ill-conditioning at the cost of impairing the simulation”.

The depth a loaded contact carries is a chosen number. In a regularised formulation the steady-state violation at a contact is the normal load times the compliance. It is zero at zero load and it grows with the load carried. MuJoCo drives that violation coordinate with a critically damped stabiliser parameterised by ε and κ , and for an object resting under gravity the steady-state depth has a closed form independent of the object’s mass [99, Sec. V]. The closed form did not survive the parse of that paper, so the cancellation is quoted and the algebra is not. Neither MuJoCo paper states why mass cancels, and the explanation this survey offers is its own inference rather than a cited one: the regulariser is scaled by the inverse effective inertia at the contact, so compliance falls as $1/m$ exactly as the gravity load rises as m . The nearest support in a parsed source is for a different solver of the same engine, MuJoCo’s diagonal solver, a “mass-aware spring-damper” that uses the diagonal of the A

matrix to keep contacts critically damped [100, Sec. II-E]. This is not a penalty spring, and depth is not always non-zero. The impulse solves a regularised convex program over the whole contact set rather than a per-contact function of the gap, both MuJoCo papers reject spring-dampers by name, and the 2012 ball-drop figure is captioned “there is no penetration” [100, Fig. 2].

The first of two concrete measurements comes from the other end. Dojo solves a hard-contact nonlinear complementarity problem with an exact second-order friction cone, by an interior-point method converging within 15 iterations on the three robots of its convergence study. Its Table II drops an Atlas humanoid and reports foot-floor penetration against the timestep. MuJoCo penetrates -28 mm at $\Delta t = 0.01$ s and -46 mm at $\Delta t = 0.001$ s, while Dojo stays above the floor at every step tested [101, Sec. V-A]. The MuJoCo column is not a trend. A ten-times-smaller step produces more overlap, which no timestep-independent stabiliser does. Either that configuration ties the compliance to Δt , or the quantity is an impact transient on a drop. Neither reading is a steady-state grasp depth. Drake’s SAP quotes 2.5×10^{-5} m at $\delta t = 10^{-2}$ s and 2.5×10^{-7} m at $\delta t = 10^{-3}$ s, three and five orders of magnitude below Dojo’s two MuJoCo cells at the same steps, on an engine that is also compliant [98, Sec. V-B]. Those two figures are analytical bounds for a single point mass at rest on a plane under that paper’s near-rigid stiffness rule, not measured depths, and they are not a Drake grasp-penetration number: a point-mass bound and a humanoid drop transient differ in load, effective inertia and regime, so the distance between them is not a measurement of anything. What the pair of engines does show is that in a compliant formulation the depth follows from a stiffness that someone chose.

Dojo’s Table V times 1000 steps of forward simulation with gradients, at a matched $\Delta t = 0.01$ s, for engines that are not all computing gradients. MuJoCo is fastest on every system, 0.335 ± 0.001 s against Dojo’s 1.159 ± 0.077 s on a Franka Panda, and the authors call the comparison difficult because Dojo is stable at five times the step size (Sec. VI-B). What the regularisation buys, on Le Lidec’s reading quoted above, is conditioning on a hyperstatic problem, and the depth it costs is tuned separately. Whether that is also what holds Erez’s

grasp at 16 ms is a question his data do not answer. His planar chain is contact-free, so its numbers speak to the coordinate formulation and not to contact: MuJoCo runs at 243.2 kHz there against Bullet’s 22.8 and PhysX’s 6.4, and Bullet’s articulated Featherstone mode at 81.4 kHz beats every Cartesian-coordinate engine [17, Sec. IV-B]. Those are throughput in evaluations per second, not a largest-stable-timestep result, and the articulated Bullet mode was never run on the grasp test at all, being usable only in tests without contact (Appendix). Joint coordinates explain the contact-free speed advantage. The grasp timestep is a contact result, and the paper attributes it to nothing: it reports that the other engines go unstable and “effectively simulate a different physics model which can no longer hold the object” (Sec. IV-D), without an experiment that separates the coordinate formulation from the soft absorption of penetration.

The GPU era moved the compliance knob rather than removing it. ComFree-Sim resolves contact in closed form in the dual cone of the friction cone, so penetration becomes an explicit tuning parameter and the paper reports it. On a drop test of convex primitives about 5 cm across at $dt = 0.002$ s, MuJoCo Warp penetrates 1.7 ± 4.9 mm and ComFree-Sim runs from 3.9 ± 6.9 mm at $k_{\text{user}} = 0.1$ down to 0.9 ± 1.5 mm at $k_{\text{user}} = 0.5$, though its prose describes the opposite direction from its own table [102, Sec. IV-A]. Its impedance acts on the signed gap through the “identical” default hyperparameter API as MuJoCo’s solver, so the knob it is credited with exposing is one MuJoCo users already set. Those are 5 cm primitives under their own weight, not a fingertip loaded by a grasp, and depth scales with the normal load. Nobody has published the fingertip number. The baseline it measures against is MuJoCo Warp, which Newton builds on and which MuJoCo Playground names as its intended replacement for JAX [95, Sec. VI].

That brings Table VI’s most important column, and the claim it does not support. The column records whether a parsed source reported a penetration depth, not what an engine can compute. Two of the 15 engines report one, Dojo and ComFree-Sim, and both are engines whose paper is about contact accuracy. Four are recorded as not reporting it: Brax, Isaac Gym, Isaac Lab and Orbit. Nine rows are blank. The word “penetration” appears nowhere in the Isaac Gym paper, and Isaac Lab’s contact sensor reports force, duration and an average contact point with no contact-quality metric. Those two simulators carry 42 of the 112 method papers in this corpus.

The tooling exists and the number is still not recorded. NVIDIA’s own IsaacGymEnvs repository computes interpenetration depth in simulation. Its IndustReal tasks load plug and socket meshes into Warp, sample points on one, query them against the other, and reduce to a per-environment maximum interpenetration distance (code/md/isaacgym_2021.md, lines 8809 to 8862). The policy update is gated on that number. Environments are split on `max_interpen_dists <= interpen_thresh` and the surviving reward is scaled by `1 - tanh(max_interpen_dists / interpen_thresh)`, with `interpen_thresh: 0.001` commented as the maximum allowed interpenetration between plug and socket (lines 3630 and 3753). That is a shipped Isaac Gym task measuring simulated interpenetration

per environment at a millimetre threshold, during RL, and acting on it. Table VI records Isaac Gym as not exposing penetration and corpus/rows/isaacgym_2021.json carries `penetration_exposed: false`, which the code parse in the same corpus contradicts. Tactile Genesis makes the point from the other side, shipping penetration depth as a sensor on an analytic SDF backend and a BVH backend [103, App. A.1], while Table VI leaves the Genesis cell blank. The depth is computable from the poses and the meshes in a few lines of Warp, and the field’s own benchmark repository already does it. Interpenetration in a dexterous rollout is a setting nobody records and a measurement nobody takes.

A policy will exploit what nobody looks at. DexTrack’s configs carry PhysX’s `max_depenetration_velocity` at 10.0 or 1000.0 depending on the task variant, with no explanation in the paper or in a config comment [104]. The same parameter appears across unrelated stock IsaacGymEnvs tasks at 5.0, 10.0, 100.0 and 1000.0 (code/md/isaacgym_2021.md, lines 897, 1924, 2083, 2448 and 3113), so DexTrack inherited a template rather than choosing per variant. The parameter caps the rate at which the solver pushes overlapping bodies apart, so it sets how long an overlap persists and how violently it is undone, not how deep the overlap gets. The one knob here that governs interpenetration behaviour is being copied without being read. DexTrack’s paper defines a maximum hand-object penetration depth, applies it only to its input kinematic references, and presents tolerance of “severe hand-object penetrations” as evidence of robustness (App. B). TopoRetarget [7] is the one corpus method that reports the number carefully, and it reports it on retargeted references rather than on a rollout, which Section VII takes up.

The rest of Table VI is largely empty, and the emptiness is a result. Sixty-eight of its 165 cells are values no parsed source stated, which is 68 of the 150 cells outside the engine-key column, or 45 percent. No engine paper states a default physics timestep. Three report one for a named experiment, and the timestep column reports those experiment settings. Isaac Gym’s cell is its Shadow Hand step, from the only per-task timestep table any engine paper here publishes, which runs 1/120 s for Shadow Hand and Allegro, 1/200 s for ANYmal and TriFinger and 1/60 s for Franka [97, Table 4]. MuJoCo’s 0.01 s is the 27-DoF humanoid test’s step and ComFree-Sim’s 0.002 s is its benchmark step, against a stated stability limit near 0.02 s. Four engines state a solver iteration count and seven ship any dexterous hand at all. The licence column answers a question a reader choosing an engine actually has, and thirteen of fifteen rows do not answer it.

C. The GPU-parallel turn

Isaac Gym set the pattern. Physics, observations, rewards and actions stay on the GPU, and PhysX resolves contacts with the Temporal Gauss-Seidel sweep described above. Its per-task timesteps are published, which is rare: the Shadow Hand runs a 1/120 s physics step under a 1/60 s control step, or 1/20 s in the OpenAI variant. The result that reorganised the field is that reproducing OpenAI’s Shadow Hand cube reorientation took under an hour on one A100, against 30 hours on 6144

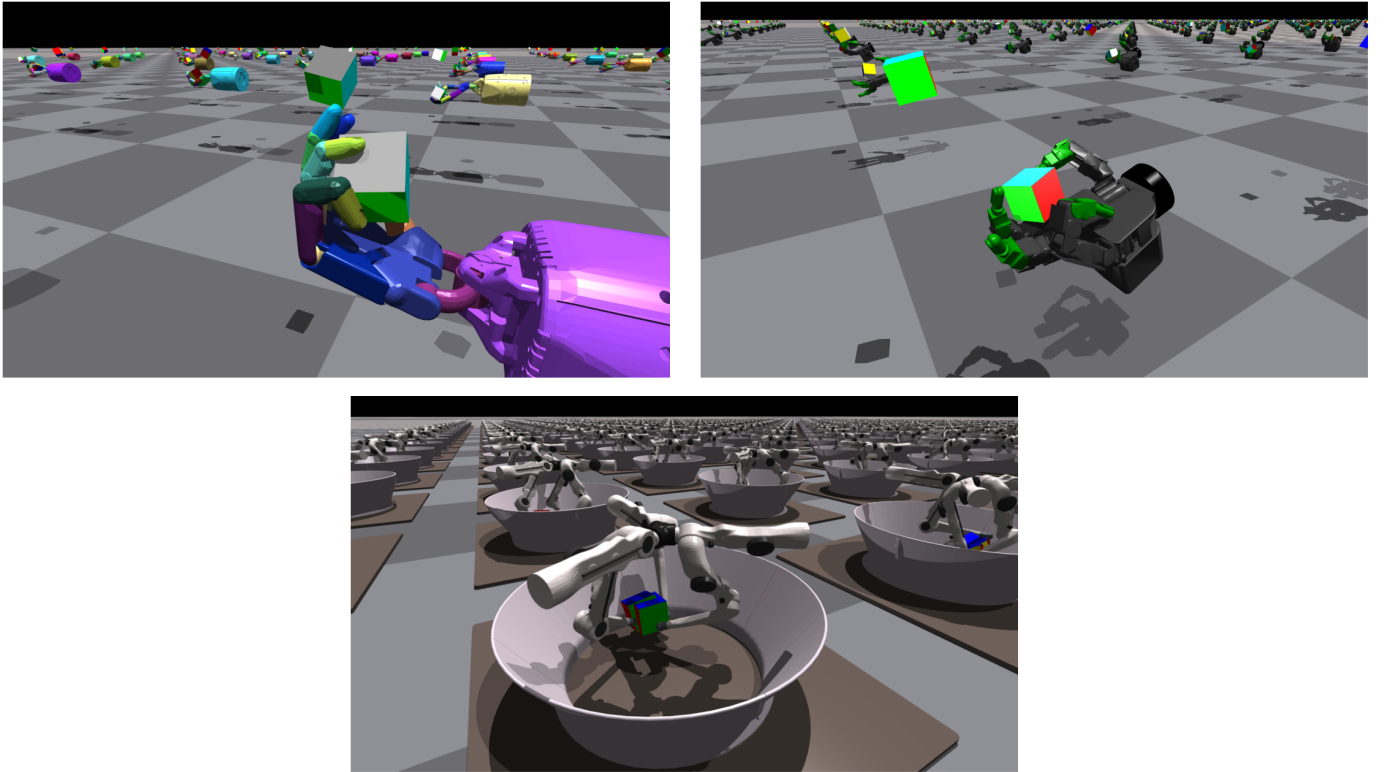


Fig. 12: Three in-hand environments as the GPU simulator actually presents them. The copies receding into the background are the other parallel environments sharing the step; the foreground scene is one of thousands [97]. The plate shows NVIDIA product imagery; CC BY 4.0 conveys no trademark rights.

Reproduced from Fig. 13 of V. Makovychuk et al., “Isaac Gym: High Performance GPU-Based Physics Simulation For Robot Learning,” arXiv:2108.10470, licensed under CC BY 4.0; re-rendered, not otherwise changed.

CPU cores and 8 V100s [97, Sec. 6.4.1]. Thirty-six of the 112 method papers in this corpus run on it.

Orbit and Isaac Lab moved the stack to PhysX 5, and the dexterous offering is thinner than the predecessor’s: the first-party suite is lifting, grasping and reorienting with the KUKA Allegro hand, while Shadow and Ability hands appear only through third-party grasp evaluation [105, Sec. 7.2.3]. MJX and MuJoCo Playground took the other route, putting MuJoCo’s own solver on the GPU through JAX at the cost described above. MuJoCo Warp ports the same solref and solimp soft-constraint rows to NVIDIA Warp, and its README states that the legacy PGS solver and the noslip pass are unsupported and that differentiability “is not yet available” [106]. GPU MuJoCo is not a differentiable MuJoCo. Genesis is the broadest, pairing a rigid constraint solver with FEM, MPM, PBD and SPH solvers and a differentiable backward path, and it ships a Shadow Hand. Its README quotes no throughput number at all, so the large FPS figures that circulate for Genesis trace to nothing in [107].

Newton changes how these names should be read. It is explicitly multi-solver: SolverMuJoCo wraps MuJoCo Warp, SolverKamino is a proximal-ADMM and dual-variational-inequality contact solver, and XPBD and VBD do position-based constraint projection [108]. Its contact model is solver-dependent, not a property of the engine, which is how Table VI records it. Isaac Lab’s code already exposes a `-physics newton_mjwarp` backend switch in its hands demo, and

its roadmap announces Newton integration. An experiment reported as Isaac Lab may be running PhysX 5 with TGS, or MuJoCo’s soft constraint rows under Warp, and those two make different contact errors. The name also fails to fix the physics inside one engine. MuJoCo’s convex solver is a family, interior point or projected Newton, conjugate gradient or Gauss-Seidel [100, Sec. II-D], and MuJoCo Warp supports neither PGS nor the noslip pass. PhysX 4 and PhysX 5 differ in whether non-convex rigid bodies get SDF collision, which is a contact-geometry difference between Isaac Gym and Isaac Lab under one vendor name [109], [105]. Naming a simulator no longer names its physics. Papers should report the backend and the solver beside the framework.

D. Throughput, and why the reported numbers do not compare

Four headline figures measure four different quantities. Isaac Gym reports parallel environment steps per second with physics, observations and rewards on device: 150,000 for the Shadow Hand at 16,384 environments on one A100. Isaac Lab reports frames per second in training, which includes the learning update: over 900,000 for the DextraH teacher task at 16,384 environments on eight RTX Pro 6000 GPUs, with no single-GPU dexterous number anywhere in the paper. ManiSkill3 reports “up to 30,000+ FPS” with RGBD and segmentation on one RTX 4090, measured over 1000 random actions with reward and termination removed from the timing, and does

not state the environment count behind the figure [110]. Orbit reports a 125,000 FPS physics-only ceiling on an RTX 3090, also with no environment count [109, Sec. VII].

The one number reported cleanly enough to reuse is MuJoCo Playground’s, in PPO steps per second on a single A100 over five seeds. LeapCubeReorient runs at $76,354 \pm 143$ and PandaRobotiqPushCube at $487,341 \pm 4,346$. Same hardware, same measurement, same codebase, a factor of 6.4 between two environments. It is not a factor from the task in isolation. Playground tunes solver iterations, line-search iterations, timestep and contact bounds per environment, with values as far apart as one and four solver iterations [95, Table III], and it does not print the two configurations side by side. That omission is the reporting failure this subsection is about. The same confound sits inside Isaac Gym’s paper on one A100: 700,000 environment steps per second for Ant, 200,000 for Humanoid, 150,000 for the Shadow Hand. A training-loop FPS is also mostly not a measurement of physics. Playground breaks the fractional cost down on an RTX 4090: for CartpoleBalance, physics is 0.02, rendering 0.06, inference 0.01 and the policy update 0.91, and the policy update still dominates on the Franka task.

Cross-framework comparisons add further free parameters. ManiSkill2’s PickCube table takes the best result over 16 to 512 environments for each system [111, Table 1a], giving ManiSkill2 with a render server 2487 ± 24 FPS at its optimum of 64 environments against Isaac Gym’s 865 ± 35 at its optimum of 512. The env-count tuning is the second problem, not the first. The two systems are not the same kind of thing. ManiSkill2 runs rigid-body physics on CPU worker processes behind a shared GPU render server, and Isaac Gym runs physics on the GPU, a difference its own paper states plainly. The measured quantity includes 128×128 rendering at 500 Hz simulation and 20 Hz control for both, so this is a visual sample-collection loop rather than two physics engines. Its authors add the fidelity caveat themselves, and Playground is equally explicit that its cross-simulator plot borrows its Isaac Lab and ManiSkill3 numbers from the ManiSkill3 paper. Several sources give no number at all: MuJoCo Warp’s README points to an external nightly dashboard, and Newton’s and Genesis’s READMEs contain no FPS or speedup anywhere.

The failure reaches past the engine papers into the methods. Of the 47 corpus method papers that name a GPU-batched simulator, 19 state no environment count anywhere, and Isaac Lab’s own paper states no physics timestep, no decimation and no solver iteration count for any task. A throughput figure is interpretable only with the environment count, the GPU, the timesteps, the solver iteration budget, and a statement of whether rendering and the learning update sit inside the measurement. Almost nobody reports all five.

E. Tactile simulation and what it is calibrated against

The three tactile simulators in this corpus calibrate against three different things, and none of them is a manipulation outcome. TACTO is a rendering layer over a host engine, by default PyBullet’s rigid contact model. It reads post-solve link poses and the engine’s reported normal force and maps that

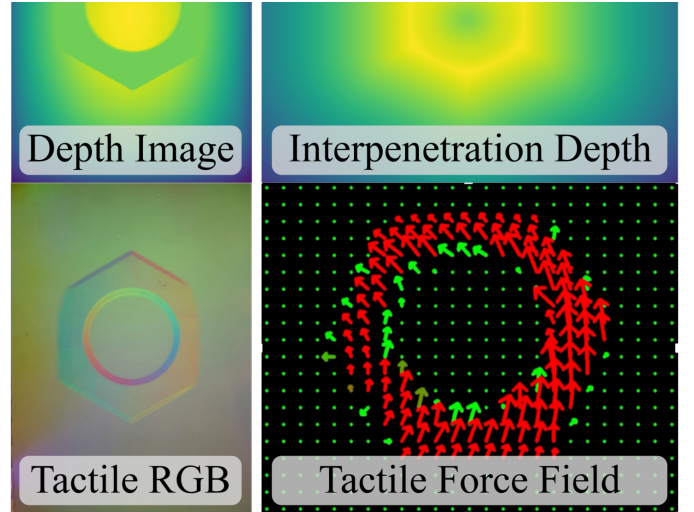


Fig. 13: Interpenetration rendered as a field. A compliant contact model turns the depth by which object and sensor overlap into the tactile image and the distributed force the policy reads; the overlap is the signal, not an artefact to be removed [105]. The plate shows NVIDIA product imagery; CC BY 4.0 conveys no trademark rights.

Reproduced from Fig. 10 of M. Mittal et al. (NVIDIA), “Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning,” arXiv:2511.04831, licensed under CC BY 4.0; cropped to the four labelled panels.

force to gel-mesh deformation at the rendering level, so it contributes no contact physics of its own. Its only sim-to-real number is a tactile pose-estimation task, at 1.66 ± 0.16 mm with colour-jitter augmentation against 0.76 ± 0.07 mm for a model trained on 128 real datapoints [112, Table II].

Taxim is example-based rather than simulated, with an optical model calibrated from 50 real indentations, and it beats TACTO on every optical-similarity metric against real images. Its marker-motion model is calibrated against an ANSYS FEM, and the errors are 1.00×10^{-2} mm real against FEM, 1.02×10^{-2} mm real against Taxim, and 3.96×10^{-3} mm FEM against Taxim. Taxim tracks its own reference more tightly than that reference tracks reality, so its residual is bounded below by the calibration target. The authors also state that they simulate quasi-static contact only, with slip left as future work [113, Sec. V]. Slip is what a hand needs.

Tactile Genesis inverts the survey’s thread. It implements two penetration-depth backends on the collision geometry, an analytic SDF query and a BVH raycast, then binarises the result with Schmitt hysteresis at 5×10^{-4} m on and 2×10^{-4} m off. Here the penetration depth is the sensor signal, so the quantity every other engine treats as an error sets this sensor’s operating point. Against a real GelSight it reports relative marker RMSE of 0.329 in dilation and 0.174 in shear, against HydroShear’s 0.403 and 0.217, with each simulator tuned to match the real image first [103, Fig. 2]. Its own sim-to-real check is a matched success count rather than a fidelity measurement. The real XHand1 SDK reports a per-taxel raw pressure field as well as an aggregate contact pressure, but documents no taxel positions or response characteristics, so the raw field cannot be registered to the simulated probe layout and only the aggregate is comparable (App. C). The undocumented

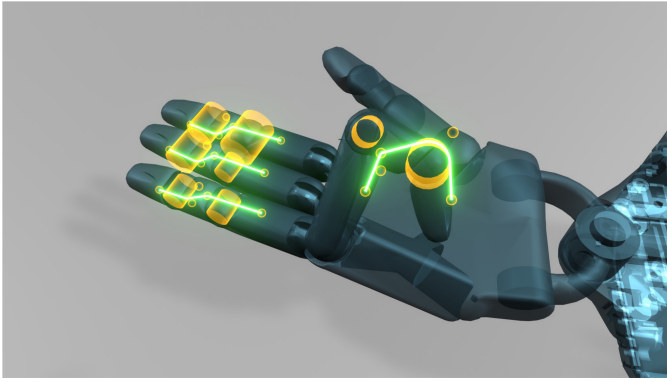


Fig. 14: A tendon-driven hand as the physics engine sees it: spatial tendons (green) routed over wrapping cylinders (yellow) inside the transparent shell. The routing geometry, not the joint limits, is what a sim-to-real gap on this hand is made of.

Reproduced from [114]; permission not yet sought, see the register.

calibration is the barrier, not a missing signal.

F. The sim-to-real gap for hands

Most of what is written about this gap is attribution without measurement. PenSpin asserts that the pure physics gap “cannot be bridged by extensive domain randomization alone” while reporting no experiment that isolates it [88]. MuJoCo Playground attributes its LEAP hand failures to physical flex in low-cost hardware and names more accurate collision geometries as the fix, without measuring either. DeXtreme lists four candidate causes for its shortfall, including a malfunctioning Allegro thumb used in most trials, and disambiguates none by experiment [26].

What has been measured is narrower, and Section VII collects the transfer numbers. The one result that links a quantified model error to transfer is a humanoid recipe paper, which ranks its system identification runs by dynamics-model MSE and pairs each with real success over 10 trials: the lowest-MSE model grasps 8 out of 10, the median-MSE model 3 out of 10, the highest-MSE model 0 out of 10 [115, Table 1]. Where a cause has been pinned down elsewhere it is usually perception or actuation, not contact. OpenAI’s pose estimator has 3.12 mm error on rendered images and 9.27 ± 4.02 mm on 992 real ones, and PDDM reports a camera tracker with 5 mm average error and 20 ms latency as the unmodelled source in its real numbers [116, App. C].

The contact side stays unmeasured, and the one paper that looks at it is usually read backwards. DeXtreme’s real-to-sim replay interpenetrated because the replayed poses carried the pose estimator’s error, not because the contact model failed. The paper says so: “there is still some sim-to-real gap in pose estimation. This is manifested when we played back the real states in sim (real-to-sim) with physics enabled, which sometimes resulted in interpenetrations. Therefore, we were not able to easily calibrate physics parameters of the cube” [26, Sec. 5]. A replayed trajectory is a placement, so the overlap it shows bounds the state estimate rather than the physics. That is why it could not calibrate the cube, and it is why calibrating contact against hardware still has no worked example for a

hand. The only direct measurement of simulator fidelity against hardware in this corpus is Dojo’s, an average final-position gap of about 0.5 cm over 5 box-pushing trials. It is a parallel-jaw arm pushing a box, and there is no equivalent number for a hand.

V. HOW POLICIES ARE TRAINED

A. The design space

A dexterous policy is defined by what supervises it, and four sources are in use across the corpus’s 112 method rows: a reward function supervises reinforcement learning, a human demonstration supervises imitation, a human reference trajectory supervises a physics-based tracker, which is imitation with a simulator in the loop, and nothing supervises a model-based planner, which is handed a cost and a model instead.

The labels do not partition the corpus. Of the 112 method rows, 53 carry the tag RL, 27 BC, 23 distillation, 18 VLA, 14 teleop-system, 14 diffusion, 10 data-collection, 8 flow, 6 RL+demo, 6 trajopt, 5 MPC, 4 grasp-synthesis and 2 world-model. The tags sum to far more than 112 because most methods published since 2024 sit on two branches at once. Fig. 15 draws the tree and the cross-links. The repository carries the row-by-row version of the same thing, one row per method, and that is what to scan when looking for work comparable to your own.

What the deployed policy looks like once training is done matters more than the name a paper gives itself, and on that axis the field has converged hard. Section V-G shows why.

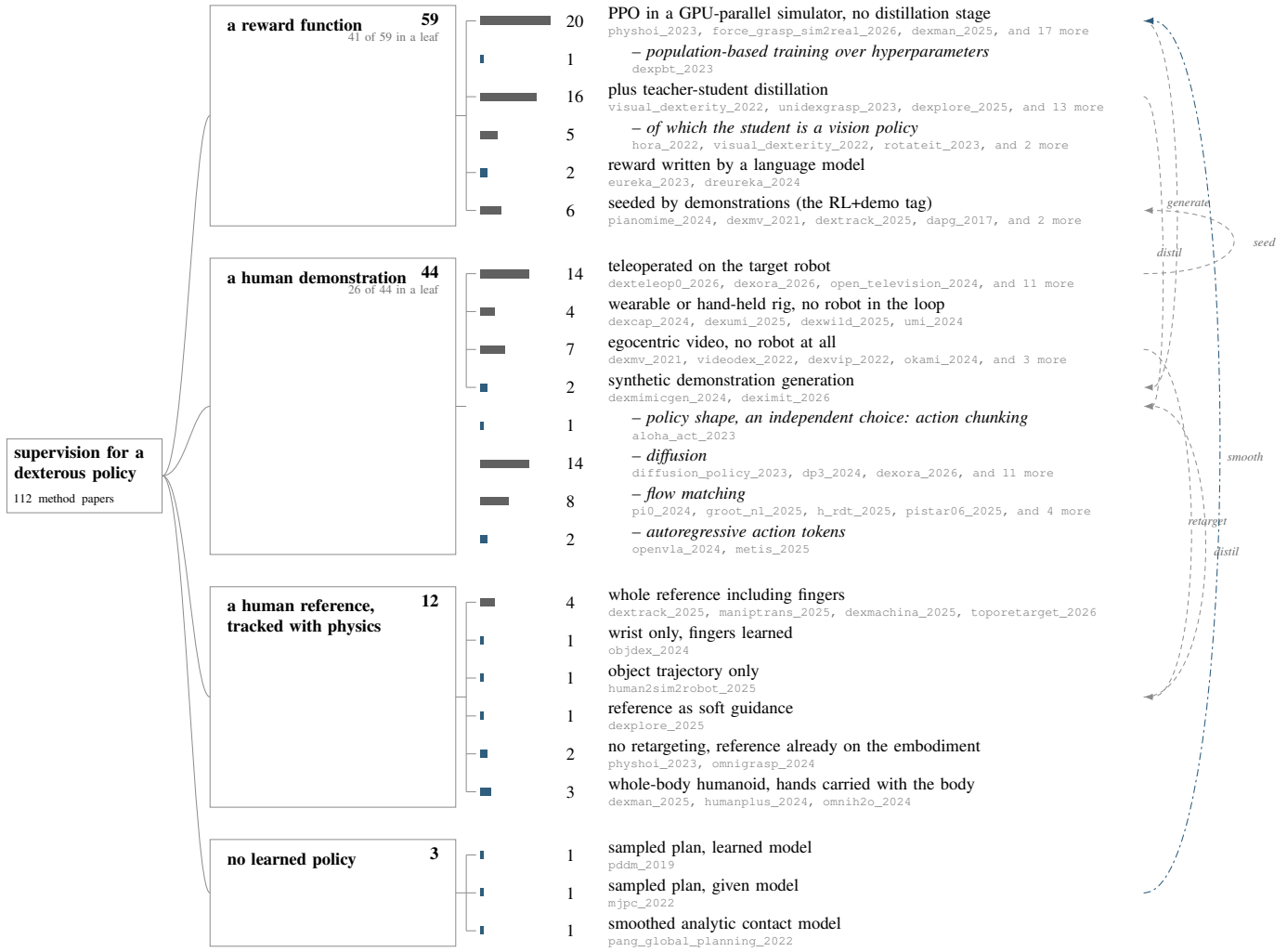
B. Reinforcement learning

1) *The standard recipe:* Fifty-nine of the 112 method rows learn from a reward. Forty-eight name PPO as the algorithm, and the next most frequent, DAPG, appears four times. Forty-three run their own experiments in a GPU-parallel simulator from the Isaac family or Genesis, and 36 do both. Only 31 rows state an environment count, with a median of 8192 and a maximum of 64000. Twenty-three rows distil a privileged teacher into a deployable student, and 16 combine all three of PPO, a GPU simulator and distillation. That 16-row intersection is the recipe as it is actually practised.

The hand is almost always an Allegro. It appears in 35 of the 112 method rows and in 17 of the 31 reorientation rows, ahead of Shadow at 21 and 6 and LEAP at 12 and 4.

Privilege is handled two ways. OpenAI [25] gives the value network object and target orientation, joint angles, joint velocities and object velocities the policy never sees, with a footnote recording that current object orientation was left out of the policy inputs by accident, and DeXtreme [26] uses the same asymmetric critic with a 2048-unit LSTM against the actor’s 1024. Hora [15] instead compresses nine privileged object properties into an eight-dimensional vector regressed from 30 steps of proprioception, and Visual Dexterity [28] distils twice, from a state teacher to a synthetic point cloud and then to a rendered one, for a fivefold speedup.

Domain randomisation is the part of the recipe with the least discipline. OpenAI [114] made it adaptive, pushing each range boundary out when performance at that boundary exceeds



Eight further rows carry a trajectory-optimisation or MPC tag inside a learned pipeline and are counted on the branch that learns. The double-dashed edge is the one overlap that is a theorem rather than a pipeline: pang_global_planning_2022 proves that a policy gradient and an analytic log-barrier relaxation compute the same local model of contact.

Fig. 15: What supervises a dexterous policy. Four kinds of supervision, subdivided to the level at which the 112 method papers actually differ, with the corpus keys at each leaf and the count of method rows the leaf’s stated predicate selects. Bars scale with that count, so the scarce leaves read as scarce. The branches are not exclusive: the dashed curves in the right-hand gutter are the overlaps, and the double-dashed one is a theorem rather than a pipeline.

20 successes and pulling it in below 10, and DeXtreme [26] reproduced the mechanism with a 256-sample queue and 40 percent of environments dedicated to boundary evaluation. Both publish the discovered ranges. Below that standard, two shipped configurations disagree with their own papers about what was randomised: Hora [15] states a joint-noise range of $U(0, 0.005)$ against a shipped `jointNoiseScale` of 0.02, in a commit its own README says is not the one that reproduces the paper, and PenSpin [88] zeroes the disturbance force its appendix describes. DexPBT [117]’s `randomize: False` is not a third case: it is the IsaacGymEnvs default and agrees with the paper’s statement that randomisation was not used here.

2) *Reward engineering:* The repository puts the 21 in-hand reorientation methods against nine recurring term families and marks each cell by where the term was found: in the

paper, in the code, in both, or in the code with every shipped configuration setting its weight to zero.

The families are not equally popular. Nineteen of 21 methods have a goal or rotation tracking term, which is the task. Thirteen penalise effort as torque, work or joint velocity, 13 pay a sparse success bonus, 11 penalise object velocity and 10 penalise a drop. Eight penalise deviation of the hand from a canonical grasp pose, a family the plan for this table did not anticipate and which had to be added. Six penalise action rate or magnitude, five reward closing the distance from fingertips to the object, and four carry a contact or force mark at all.

Three cells an earlier draft marked `code print code (0)` instead, because in each the term is in the released code with every shipped configuration setting its weight to zero: DeXtreme [26]’s `timeout_rew` and DexPBT [117]’s fall

penalty through `fallPenalty: 0.0`, and PenSpin [88]’s `action_penalty_scale: 0.0`. Marking them code would tell a reader the code optimises something the paper does not state, and leaving them blank would hide a term that is in the file. The same mark carries the config key and value in `corpus/reward_matrix.json`. Section V-H says why the distinction matters.

Three of the four contact marks are the finding, not four. AnyRotate [50] scores good and bad fingertip contacts, Lin et al. [118] reward a friction-cone wrench margin, and Zhao et al. [119] track a commanded grasp force. The fourth, [28], penalises the *object* touching the table, a task-shaping term against using the table as a third finger rather than a hand-object term, and its `pen_tb_contact` flag defaults to `False` with no shipped config setting it true. Three of 21 in-hand reorientation methods, then, put a hand-object contact or force quantity in the reward, and none puts interpenetration in it. Li et al. [120] penalise interpenetration with a differentiable signed-distance term, but during offline reference construction, not in the policy’s reward. Across all 112 method rows, 85 do not address penetration at all, 5 constrain it, 3 measure it, 3 penalise it and 16 say nothing either way. The physical quality of the contact is not something this literature optimises.

Term counts range from one to ten. DemoStart [121] gives a single binary terminal reward of 1 and lets a curriculum over demonstration states do the shaping. AnyRotate [50] and ViserDex [122] each publish ten weighted terms. ViserDex is the cleanest specification in the table, with every term, weight and equation in one appendix table, and a statement that the same weights are used for every object.

Weights are frequently unrecoverable. Three of the 21 rows give no numeric weight that survives PDF conversion, because the equation blocks are images, which is a limit of this survey and is recorded as such. Robot Synesthesia [86] is a worse case. Its six coefficients are described only as “tuned hyperparameters” and no number is printed anywhere in the paper, so the reward it reports cannot be reconstructed by anyone. DexReMoE [96] names an angular-velocity penalty weight in prose with no entry in its hyperparameter table, and DexTrack [104] names an affinity reward that is missing from its weight table.

3) *Curricula, populations and machine-written rewards:* Curricula in this literature relax physics or tighten tolerances. DexPBT [117] tightens a success tolerance from 0.075 to 0.01 by a factor of 0.9 every 3000 environment steps once three successes are logged, and ManipTrans [29] starts at zero gravity and high friction and restores both while narrowing a fingertip threshold from 6 cm to 4 cm. DexMachina [30] attaches virtual spring-damper controllers to the object, lets them do the task at first, then decays their gains to zero. The two that ablate the curriculum report it is load-bearing: RotateIt [87] sets its rotation-penalty weight to zero and ramps it to -0.1 , because applying it from the start makes the policy learn only to hold the object still, and ViserDex [122], whose performance-based curriculum runs over regularisation weight, action latency and time between successes, fails completely without the regularisation component.

Population-based training appears once. DexPBT [117] runs

populations of 8, 16 and 32 agents, splits them 30/40/30, mutates the middle and replaces the bottom with mutated copies of the top, each float hyperparameter multiplied or divided by a factor drawn from $U(1.1, 1.5)$ with probability 0.2. It reports 30 hours on a single V100 for a five-billion-transition single-arm run, and 0.32 trillion environment steps for its largest population.

Eureka [27] has GPT-4 write the reward function directly, constrained to return a total and a dictionary of named components, and feeds per-component statistics back between iterations. Its appendix gives an example whose reward correlates at -0.26 with the human-written one and still scores 1.45 on the human-normalised metric, the strongest argument in the corpus that hand-tuned weights are not a ceiling. DrEureka [123] moves the safety requirement into the prompt instead of a penalty term, asking in words for a cube rotating at about 0.25 radians per second with fingers penalised for leaving their initial pose. Neither can be checked against a file. Eureka’s generated rewards are written at run time to a gitignored directory and never committed, so the reward behind any reported number cannot be recovered from the repository, though its README documents the paper’s budget of five iterations and sixteen samples as the default and marks the shipped one-iteration block a fast-test preset. DrEureka’s repository contains only the locomotion and globe-walking trees, so its LEAP-hand reward has no code at all.

4) *Where reinforcement learning stalls:* Reinforcement learning stalls on objectives that are not learnable as stated. AnyRotate [50] found the angular-velocity objective unlearnable in the multi-axis setting and replaced it with a moving keypoint target. RotateIt [87] reports that its multi-axis policy “does not converge when training with only reinforcement learning” and adds an imitation loss against single-axis oracles. Eureka [27] cannot spin a pen from scratch and needs a pretrain-then-finetune split, with the from-scratch ablation failing to complete one cycle. Section V-D’s contact-graph term exists because of the general shape of the problem: contact moves the object off the reference, so return goes down, so the policy learns not to touch the object.

It stalls on geometry too. Hora [15] fails on objects under 4 cm across because the fingers collide with each other. What it does not stall on is publication: the real-trial counts behind these headlines are small enough that Section VII-A treats them as the reporting problem they are.

C. Learning from human data

1) *Teleoperation and retargeting:* Thirty corpus rows describe a system for getting human motion onto a robot hand; the repository lists them with the retargeting objective compressed to one clause each.

Retargeting splits four ways. The dominant family matches task-space vectors between the human and robot hands. DexPilot [124] set the pattern with a weighted squared distance between fingertip vectors, switching weights of 1, 200 and 400 by whether a pair is within threshold, a minimum-separation constant of 3 cm between primary fingers, and a regulariser toward the open hand, solved online with SLSQP; AnyTeleop

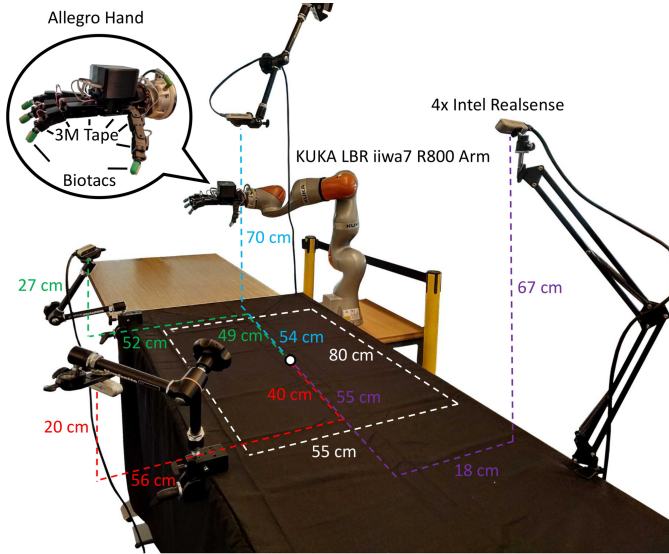


Fig. 16: A camera-based teleoperation cell, dimensioned. Four depth cameras around a marked table, a seven-axis arm and a four-fingered hand with tactile fingertips: the working volume in which every demonstration from this rig was recorded [124].

Reproduced from Fig. 3 of A. Handa et al., “DexPilot: Vision Based Teleoperation of Dexterous Robotic Hand-Arm System,” arXiv:1910.03135, licensed under CC BY 4.0; re-rendered and resampled.

[125], Open-TeleVision [126], Bunny-VisionPro [53] and ACE [127] all use the same formulation. The second copies joint angles where the kinematics allow it, as Holo-Dex [128] does for index, middle and ring while solving the thumb by inverse kinematics and discarding the pinky the Allegro does not have. The third learns the map offline: Geometric Retargeting [129] trains a per-finger network in three to five minutes against motion-direction, coverage, flatness, pinch and collision criteria, then runs at 1 kHz against the 60 to 100 Hz of online optimisation, raising configuration-space coverage from 38 to 90 percent and one-time grasp success from 55 to 87.5 percent. The fourth removes retargeting from the loop: DexUMI [85] builds an exoskeleton whose fingertip workspace is optimised to match the target hand, then regresses encoder values straight to motor values.

The table’s empty cells are the point. Only 12 of the 30 rows carry a latency cell at all, three of those give no number, and only 7 state a rig cost. Where costs are stated they are low and falling, from [128]’s \$399 headset through the \$600 exoskeleton of [127] and the \$600 haptic glove of [130] to the \$4,000 backpack of [131] and the \$6,000 rig of [54]. Latency is not one quantity: DexPilot [124]’s “about one second” and Holo-Dex [128]’s “under 100 milliseconds” are end to end, while Geometric Retargeting [129]’s 1 kHz and AnyTeleop [125]’s 9 to 10 ms retargeting are single stages and several cells give a control rate. The column mixes the two, so its spread is not a range. Only 13 rows report how many trajectories were collected, and only 7 report hours.

Where throughput is measured, human collection beats teleoperation by a wide margin. Qiu et al. [132] time a grasp demonstration at 3.79 seconds for a bare human and 19.72 seconds through a teleoperated humanoid, and a pour at 4.81

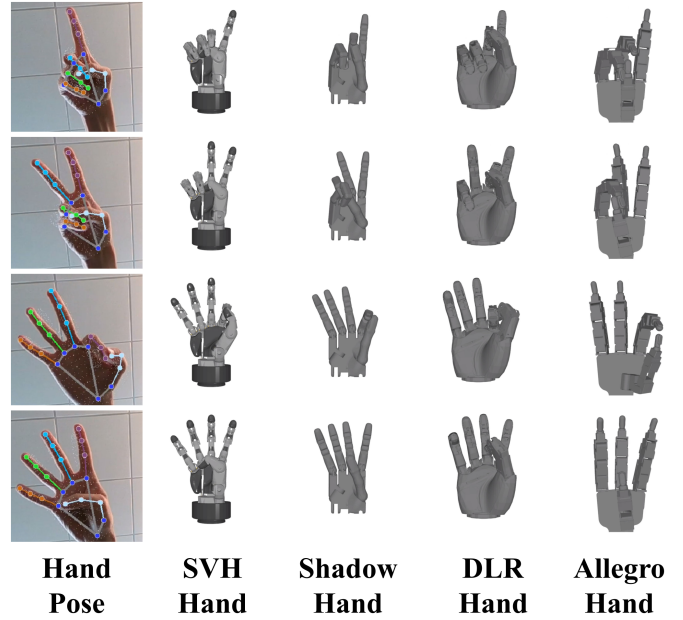


Fig. 17: One human hand pose, four robot hands. The same tracked keypoints retarget to visibly different postures on each mechanism, which is the size of the embodiment gap a shared demonstration corpus has to cross [125].

Reproduced from Fig. 10 of Y. Qin et al., “AnyTeleop: A General Vision-Based Dexterous Robot Arm-Hand Teleoperation System,” arXiv:2307.04577, licensed under CC BY 4.0; re-laid out from a row of panels into a stacked column: the four gesture rows composited above the hand-label strip.

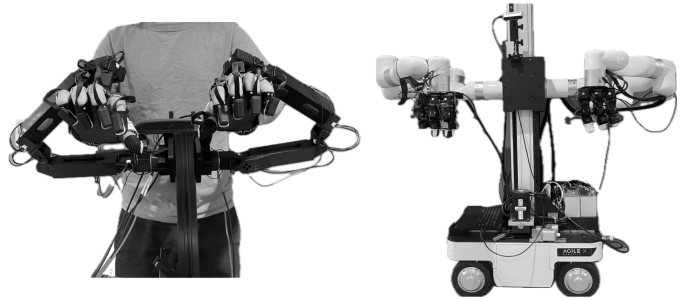


Fig. 18: The other end of the same problem: a worn bimanual exoskeleton that measures the operator’s own finger motion directly, and the mobile two-armed robot it drives. Greyscale; colour carried no information here.

Reproduced from [54]; permission not yet sought, see the register.

against 37.31 seconds. It attributes the gap to retargeting latency and to the workspace of a 7-DoF arm rather than to anything about the hand.

2) *Imitation architectures:* Four architectures cover the corpus. Action chunking came from [133], which predicts a chunk of joint targets with a CVAE and combines overlapping chunks by exponentially weighted ensembling, reaching 80 to 90 percent on fine bimanual tasks from about 50 demonstrations each. Diffusion came from Diffusion Policy [134], which denoises an action sequence and reports a 46.9 percent average success improvement over LSTM-GMM, IBC and BET across 15 tasks. 3D Diffusion Policy [135] swaps the image encoder for a small point-cloud encoder and reports 74.4 percent against 59.8 percent across 72 simulated tasks, and 85.0 against 35.0

percent on four real tasks from 40 demonstrations each. Flow matching is the 2024-onward default for large models and carries 8 of the 112 rows. Discrete action tokens are the fourth, used by [36] and [136], and are the only one of the four that needs no continuous head at all.

Data generation is a separate lever and is underrated. DexMimicGen [55] turns 60 human source demonstrations into 21,000 simulated ones across 9 tasks and 3 embodiments, and a real Fourier GR1 with two Inspire hands reaches 90 percent from 40 generated demonstrations against 0 percent from the 4 source ones. Dex1B [137] iterates optimisation, a CVAE proposal model and a simulator filter into roughly a billion synthetic grasps, and its CVAE baseline beats the prior best by 22 points.

3) *Human video without a robot*: Seven rows train a dexterous-hand policy from video with no teleoperation at any stage. DexMV [138] retargets 700 self-recorded demonstrations by matching palm-to-fingertip task-space vectors and estimating actions by inverse dynamics, VideoDex [139] mines 965 trajectories from EpicKitchens to pretrain a neural dynamic policy, and DexVIP [140] goes further into the reward, clustering 715 curated HowTo100M frames into one consensus grasp pose per object category and adding it as a reward term. OKAMI [141], Lum et al. [142] and Guzey et al. [143] work from a single video each, and Goswami et al. [144] pretrain a world model on 829 hours of EgoDex.

4) *The embodiment gap, and what closing it is worth*: Three papers measure the gap directly rather than asserting it. OKAMI [141] runs the same reference-plan and warping pipeline with and without human body and hand retargeting, and the version without it loses 75.0, 42.1, 82.0 and 74.0 points across four task and setting pairs. That is the largest clean embodiment-gap number in the corpus.

Qiu et al. [132] ablate two mitigations separately on the same vertical-grasp task over 10 trials. The full method scores 4 out of 10. Removing the interpolation that slows human trajectories by a factor of four drops it to 1 out of 10. Removing the unified 54-dimensional state space while keeping the slowdown drops it to 0 out of 10, because the policy is handed a shortcut for telling the two embodiments apart. EgoZero [145] reports the sharpest version of the same effect. Removing 3D augmentation drops every one of its seven tasks to 0 out of 15, and substituting monocular depth for triangulated depth does the same, because the best metric depth models it tested carry more than 5 cm of error. It also quantifies what its hand-pose pipeline costs, at 1 to 2 cm of action-label error.

Chen et al. [146] supply the counterexample. Their method retargets only the wrist and lets reinforcement learning discover finger motion, and their ablation that adds a fingertip-matching reward “does not yield benefits and even leads to lower performance”. Richer human correspondence is not uniformly better, and this is the one result in the corpus that says so with an ablation.

D. Tracking a human reference with physics

Twelve method rows carry the `track-human-ref` task family. Eight track a human hand on an object and are covered

here; the other four sit at the edges, Lum et al. [142] tracking only the object’s trajectory, DexMan [147] retargeting bimanual video onto a full humanoid, and HumanPlus [38] and OmniH2O [39] tracking whole-body motion. A reference is retargeted onto the robot and a policy trained to make the simulated body follow it: the reference supplies the shaping reward engineering would otherwise have to invent, and the simulator the physical consistency pure imitation lacks.

PhysHOI [1] is the origin of the reward form. It multiplies a body term, an object term, an interaction-graph term and a contact-graph term, and reaches 95.4 percent success on GRAB against 27.0 percent for a DeepMimic baseline. The contact-graph term exists to stop the policy learning not to touch the object. Omnigrasp [22] replaces the action space with a pretrained latent motion prior and reports 94.6 percent grasp success and 84.8 percent trajectory success on GRAB, while stating outright that it omits penetration metrics.

DexTrack [104] adds an imitation loss to PPO and mines its own demonstrations through a homotopy search over easier neighbouring trajectories, reaching 46.70 and 65.48 percent on GRAB at loose and strict thresholds against 38.58 and 54.82 for the best baseline. It has a penetration-depth formula and applies it only to input references, never to its own rollouts, and presents tolerance of “severe hand-object penetrations” as robustness. ManipTrans [29] freezes a generalist hand imitator and trains a per-task residual on top, reaching 58.1 percent single-hand and 39.5 percent bimanual success on OakInk-V2. Its stance on contact is to raise the friction coefficient above the real value rather than model skin deformation, and its real deployment is open-loop replay.

DexMachina [30] moves to Genesis with 12,000 environments and six hand URDFs, and drives the object with virtual controllers that decay to zero. Its re-implementation of [29]’s curriculum does not beat no curriculum on its own setup, which is a direct disagreement between two papers worth quoting in both directions. Dexplore [90] drops the explicit retargeting stage and treats the reference as soft guidance, with termination thresholds derived from the failure rate, reaching 87.7 percent on GRAB with an Inspire hand.

TopoRetarget [7] is the only corpus method that quantifies penetration as an evaluation quantity, reporting maximum depth and the fraction of frames past 2 mm, and constraining it during retargeting with a 1 mm soft tolerance and a 30 mm hard bound. It reaches 87.5 percent on its own pen-spinning set against 46.9 for the best baseline. It does not re-measure penetration after the tracking policy runs, so the property it constrains is a property of the reference and not of the behaviour.

That is the pattern across all eight, and it is the reference-versus-rollout split in its sharpest form. Penetration is handled at the reference, if at all, and never at the rollout. Chen et al. [146] complete the set with the only real-robot numbers among them, from 100 percent on a microwave and a laptop down to 41.2 percent on a ketchup bottle over 20 trials each.

E. Generalist and vision-language-action policies

The finding is the size of the hand. Eighteen method rows carry the `VLA` tag. Fourteen of them settle whether the reported

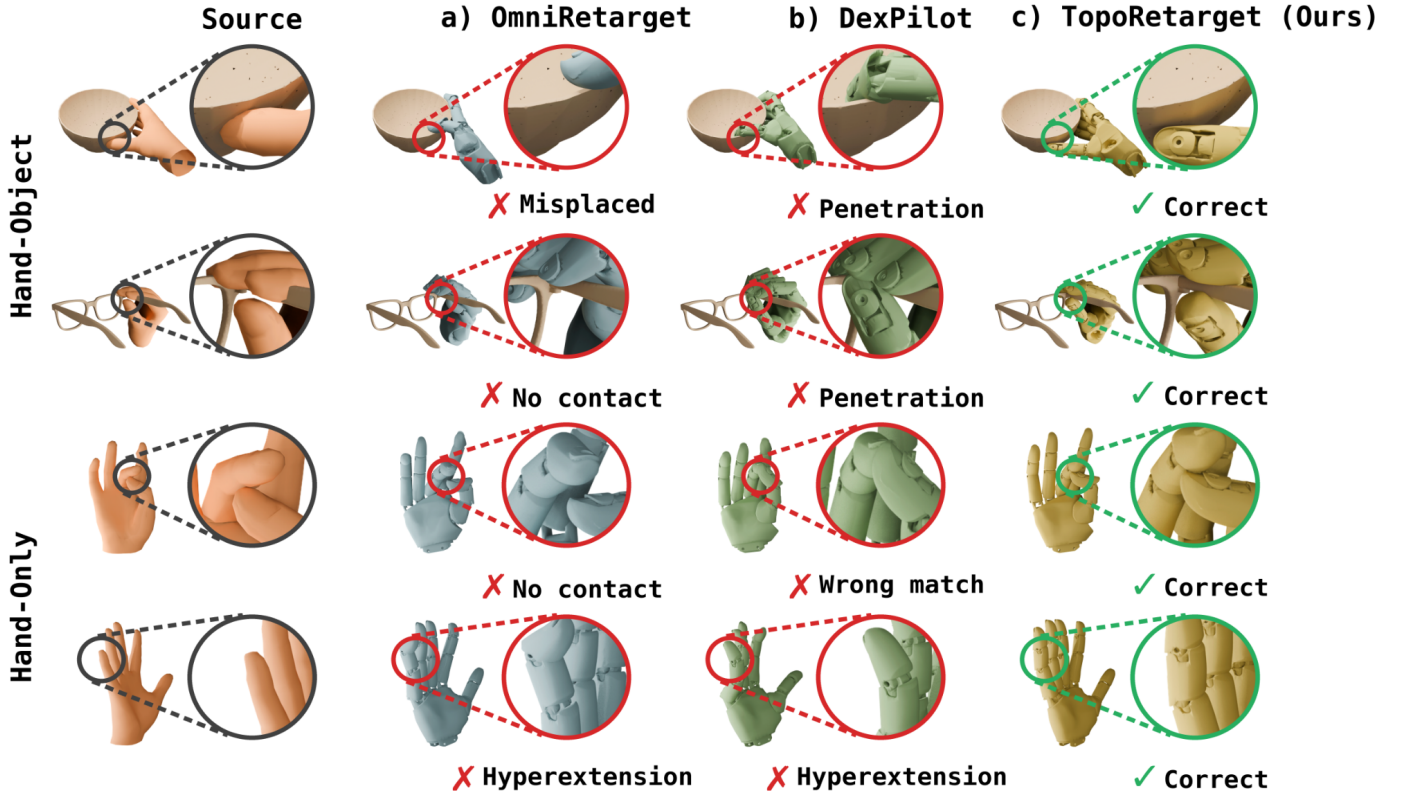


Fig. 19: What retargeting gets wrong, magnified. Against the same source motion, two established methods place the fingertip off the object, drive it through the surface, or hyperextend a joint; the failure is in the contact set, not in the joint-angle error a retargeting method usually reports.

Reproduced from [7]; permission not yet sought, see the register.

evaluation ran on a multi-fingered hand. Eight of those fourteen did, six report no hand result at all, and the remaining four never say, GR00T N1.6 [40] naming no end effector anywhere on its page. Section VIII-D counts the same eighteen the same way.

Five of the eight state a hand size, and that comparison has to be made in actuated degrees of freedom rather than joints, for the reason Section III opens with. Four are six, the Inspire RH56DFX among them actuating six of its twelve joints, and one is twelve, [148]’s XHAND. Their median is 6. Two of the four rows that never settle the hand question do state a size, and both are large: 21 for [92]’s ByteDexter V2 and 22 for [149]’s Sharpa Wave. Of the 59 reward-learning rows, 48 state a count, 38 of those are 16 or above, and their median is 16: an Allegro and a LEAP actuate 16, and a Shadow actuates 20 of its 24 joints. So no row that settles the question reaches the band the reinforcement-learning literature of Section V-B works in, and the two rows that reach it on paper are the two that never say whether the hand was in the evaluation. An earlier version of this claim counted eleven rows as evaluating on a hand, which mixed the settled eight with rows the notes leave open.

The six with no hand result are parallel-jaw throughout (π_0 [33], whose released code encodes each ALOHA gripper as one scalar, $\pi_{0.5}$ [34], $\pi_{0.6}^*$ [35], OpenVLA [36], Helix [91], which claims individual-finger control of a 35-DoF upper body with no success rate or trial count for any task, and Gemini

Robotics [150], whose Apollo hand appears in a figure caption with no number attached). Its successor [37] closed that gap with Apollo progress scores, not success rates, of 0.74 down to 0.62 across five generalisation axes. DexVLA [58] is the borderline case the other way: a hand in one real-robot task family, grippers in the other five and in all 91 pretraining tasks.

The released code also drifts. GR00T N1 [151]’s repository is a later N1.7 generation with a different backbone, a different action horizon and embodiments the paper does not describe, and the $\pi_{0.5}$ repository [34] supports only the flow-matching head, not the hybrid recipe the paper describes.

F. Model-based control as the non-learning baseline

Three corpus methods solve dexterous tasks without learning a policy, and they are the control group the rest of this section lacks.

PDDM [116] learns an ensemble of dynamics models and plans through it with an MPPI-style optimiser, replanning every step. It needs 1 to 2 hours of data in simulation and 2 to 4 hours on a real 24-DoF Shadow Hand. Predictive Sampling [152] removes the learned model too and samples ten rollouts per step through MuJoCo itself, planning in 1 to 20 milliseconds. It reorients a cube with a Shadow Hand in real time from scratch, and reports no success criterion, no trial count and no real-robot result.

The work of Pang et al. [46] is the most substantial of the three. It proves that the randomised smoothing implicit in

reinforcement learning and an analytic log-barrier relaxation compute the same local linear model of contact, then uses the analytic version inside a trajectory optimiser and an RRT. Allegro in-hand rotation takes 19.59 seconds to optimise and its plate-pickup task 117.16 seconds of planning on a 16-core CPU, against the GPU-days of Section V-B, and it is the only method here that imposes non-penetration as a hard constraint. Its own limitation is the honest part: its 3D systems transfer to hardware far worse than its 2D ones, because the quasi-dynamic assumption breaks and planned grasps miss contacts under a second-order solver.

G. Hybrids, and where the field has converged

The recurring shape is reinforcement learning in simulation distilled into a policy that looks like an imitation policy, and it appears in four variants. The first distills a privileged teacher into a vision student inside one paper, which is [15], [28], [87], [86] and [122]. The second uses reinforcement learning as a demonstration factory: DexTrack [104] mines demonstrations with per-trajectory RL and trains a generalist on them, ManipTrans [29] does the same to build a 3.3K-episode dataset, and DexterityGen [153] trains a diffusion controller on ten billion simulated grasp-to-grasp transitions and projects a human teleoperator’s coarse command onto it, taking four reorientation tasks from 0 out of 20 under raw teleoperation to 12, 13, 10 and 9 out of 20. The third generates data without reinforcement learning at all, which is [55], [137] and [154]; Dex1B’s row is classed as a dataset, so Fig. 15’s leaf, which counts method rows, holds the other two. The fourth wraps an RL policy in a residual, which is [155] at 88.8 percent over 3,200 objects in 12 GPU-hours.

The consequence is worth stating plainly. In the variants that dominate 2025 and 2026 work, the reward is no longer the objective of the deployed policy. It is the objective of the process that produced the deployed policy’s training data. Reward engineering has moved upstream into data curation, and every reward-shaping pathology in Section V-B now reaches the shipped policy through a dataset rather than a gradient.

H. What the released code says

Thirty-eight of the 112 method rows record a discrepancy between a paper and the code it released, and all 38 released code, so they sit inside the 62 rows that released anything (Fig. 1). They are not one kind of thing. Nine are contradictions, where paper and code state different values or different terms. Thirteen are limits of this survey’s own parse, where the body or config that would settle the question was never recovered and the row says so. Eight released code without the described component in it, four are version skew against a later repository, and four are a paper disagreeing with itself. The fourth version skew is [40], which ships a main branch one generation later than the checkpoint its paper describes.

Nine is the number to quote, eight at high confidence and one, PenSpin [88], held at medium pending a direct read of the code. Nine of 62 is 15 percent, bounded on both sides: a floor, because the census covers method rows only and RoboPianist [41], whose row is a benchmark, sums five

reward terms against the three its Table 2 documents; a ceiling, because eight accusations an earlier draft of this section made were withdrawn, seven of them under adversarial review and an eighth, OmniH2O [39], once writing to its authors sent someone back to the evidence, each with its reason recorded in the accused row’s `mismatch_review` field. Among the 21 reorientation methods scored term by term, seven released a repository: four disagree with their paper, one [123] ships no cube-rotation environment at all, one [15] is a later generation its own README flags, and one [27] was a default-value question the same README settles.

The most consequential case is [1]. Its `compute_humanoid_reward` hardcodes the body position-velocity error and both object rotation errors to zero, with the real computation commented out beside them, and does so unconditionally rather than per dataset, while its Table 4 lists non-zero weights of 0.1 and 0.01 for those rotation terms on GRAB. The reward that produced the paper’s numbers never tracked object orientation: a method presented as tracking a 6-DoF reference was, in the code that ran, tracking the object in position only, with body rotation and body rotation-velocity still live. Omnigrasp [22], in the same file family, keeps its object rotation term live and fails the other way, internally: `compute_pregrasp_reward_time` takes weights as arguments and then hardcodes `w_pos, w_rot = 0.9, 0.1`.

Zeroed terms recur, and are not the same failure. A term present and zeroed is worse than a term missing, because it survives a reader’s check of the file, but only when the paper claims it. DexPBT [117] sums eight components against the paper’s four and multiplies `hand_delta_penalty` by zero with the comment “currently disabled”, a term the paper never claims, and its five named weights are present at their stated values. PianoMime [43]’s Table 3 states two weighted terms while its environment sums roughly five unweighted ones, two of them inherited stubs returning zero and a third, forearm collision, the paper never lists. PenSpin [88] ships `forceScale: 0.0` against the disturbance force its appendix describes, which is the charge that survives. A second half of the original charge, that the same 96-dimensional observation disables the paper’s tactile channel, is withdrawn: those dimensions are proprioception-only, consistent with the student policy the released config runs rather than the tactile-and-point-cloud oracle, and the paper never claims the student has tactile input. The disturbance-force charge alone is why the row still reads medium rather than high.

Weights drift. DeXtreme [26] states an action-delta penalty of -0.25 in Table 2 and ships -0.2 and -0.01 in its two DR yamls, neither matching. Visual Dexterity [28]’s Eq. 8 penultimate-joint penalty is absent from `dexenv/envs/rewards.py`, and its two configs disagree about the fall distance. UniDexGrasp [156] and DexPoint [48] ship rewards structured differently from their equations, and PDDM [116]’s Baoding reward carries a -10 wrist-height term Table 2 omits.

In 13 rows the repository does not settle the question, which is this survey’s limit and not an accusation. The reward code is C++ and outside the parse in [57], whose configs expose four

velocity coefficients where the paper prints two, and in [157], whose two weight sets are the two phases of a curriculum the paper documents. DexTrack [104] ships several unreconciled coefficient sets and which produced its headline table cannot be identified, and ManipTrans [29]’s learning rate and environment count match its own config, the differing values being a README example’s override and an unused fallback. Hora [15] is the one repository that discloses its own gap, telling the reader to check out tag v0.0.1. Elsewhere a paper disagrees with itself: ALOHA [133]’s Algorithm 1 says MSE and its Section IV.C says L1, [135]’s prose says the network predicts noise while its config sets `prediction_type: sample`, and DexMachina [30]’s multiplicative task reward, charged to its code in an earlier draft, is printed in the paper.

A reward table is a claim about a training run and the code is a claim about a repository. Here the two contradict each other in ten cases, in the other 28 the released artefacts do not settle the question, and in exactly one, [15], the repository says so itself. Read the reward function before the reward table, and treat a printed weight as a hypothesis about the code.

I. The master table

The emptiest columns of the method tabulation are the ones a reader most needs: only 31 rows state an environment count, only 70 state how many real trials are behind the headline number, and only 39 state how many unseen objects were tested. The trial and unseen-object figures are the audited ones, after Section VII-A recovered 15 trial counts and 7 unseen-object counts that the notes carried and the extraction had dropped. A mostly empty row is not a weak method, but it is one that cannot be compared with any other row here.

VI. BIMANUAL DEXTEROUS MANIPULATION

A warning first. Fifty-three corpus method rows record `bimanual: true`, and the flag says only that the robot has two end effectors. Thirteen of the 53 put no dexterous hand on the robot at all. ALOHA, RDT-1B, EgoMimic and H-RDT [133], [158], [159], [160] and UMI [161] are parallel-jaw throughout; π_0 , $\pi_{0.5}$ and $\pi_{0.6}^*$ [33], [34], [35] and Diffusion Policy [134] name no hand and run grippers on every embodiment; Gemini Robotics [150] and Gemini Robotics 1.5 [37] report every bimanual number on grippers and show five-fingered hands only qualitatively; and Helix [91] and GR00T N1.6 [40] name no end effector anywhere on the page. Three more run a gripper on one embodiment and a hand on another: ACE and 3D Diffusion Policy [127], [135] and Open-TeleVision [126], whose Unitree H1 carries 6-DoF Inspire hands and whose Fourier GR-1 carries a 1-DoF jaw.

The denominator for this section is 28: the method rows whose notes place a learned closed-loop controller on two multi-fingered hands. It is the 53 less those 16; less two static grasp-synthesis methods, BiManGrasp [6] and BiDexGrasp [5]; less three systems with no learned policy, Castro et al. and DexTeleop-0 [98], [84] and Pang et al. [46]; and less four rows where no learned policy holds both hands: OmniH2O [39], whose fingers are mapped open-loop from the Vision Pro and sit outside its 19-DoF policy; OKAMI [141], whose

headline pipeline is open-loop retargeting with a learned policy only in a side experiment; DexDeform [44], a skill model refined by trajectory optimisation rather than a closed-loop controller; and Omnigrasp [22], a simulated human body with no bimanual task. Frau-Alfaro et al. [32] never enter, because their row records `bimanual: no`. Benchmarks and datasets are outside the 28 by class, Bi-DexHands and Bench2Dex [56], [162] and RoboPianist [41] being benchmarks and RP1M [42] and HumanoidGen [163] datasets; they are quoted here as evidence and never counted. Every paper below runs two multi-fingered hands unless said otherwise.

A. Why two hands is not twice one hand

When both hands hold the same object, the object closes a kinematic loop between them. Neither hand can move without changing what the other must do. BiDexGrasp [5] reports that the coupled objective “often yields imbalanced solutions, where one hand dominates stability while the other contributes marginally.” ArtiGrasp [157] reports simulation speed scaling “roughly quadratically with the number of contacts,” and trains each hand alone before pairing them.

Role asymmetry is an assumption almost everyone makes silently. BiDexHD [164] states it outright: “we assume the robot to be right-handed by default, i.e., the left hand handles the target object and the right hand handles the tool,” and Lin et al. [165] bake the same split into their reward, putting reference contact keypoints on the bottle base for the left fingertips and on the lid for the right. The bias is in the data first: TACO [166] recruited 14 right-handed subjects and measures the right hand moving consistently faster.

The arms collide, and the corpus handles this by construction rather than by control. RoboPianist [41] ships a forearm-forearm collision term that its own reward table never lists. Two papers name arm collision as unsolved. DexMimicGen [55] “does not explicitly handle collisions,” and DexMan [147] says its control parameterisation “overlooks full arm posture,” which it needs to avoid them.

The action space doubles and the observation grows faster. Bi-DexHands [56] uses a 52-dimensional action for 18 of its 20 tasks against observations of 398 to 446 dimensions. GR-Dexter [92] emits an 88-dimensional chunk of arm joints, end-effector poses, hand joints and fingertips. What the larger vector does not carry is any statement of who does what. The usual answer is one scalar reward summed over both sides, and the cost is a compounding failure rate. ManipTrans [29] scores success only if both hands succeed, and its OakInk-V2 success rate falls from 58.1 percent on single-hand sequences to 39.5 percent on bimanual ones.

B. Coordination architectures

Fig. 20 sets the four architectures side by side. Every row of the 28 is assigned to exactly one of them, read from its note.

Twenty-one of the 28 put one policy over both hands, which is three quarters of the set: the trackers and tracking-adjacent methods [30], [147], [29], [146] and [38]; the reinforcement-learning tasks [117], [165], [27], [43] and [115]; the demonstration pipelines [54], [131], [167], [55], [60] and [132]; and the

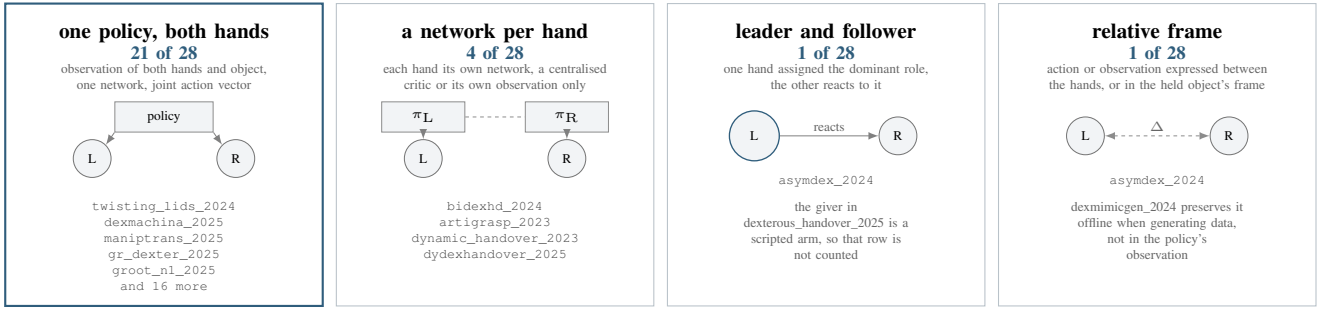


Fig. 20: Four ways to control two dexterous hands. Counts are over the 28 corpus rows that put a learned closed-loop controller on two multi-fingered hands, each read from its note. The four architectures do not partition the 28: *asymdex_2024* is counted in both of the last two panels, being the only corpus paper that puts either mechanism inside a policy, and *bunny_visionpro_2024* and *deximit_2026* are unassigned because their notes describe the rig and the data pipeline rather than the policy’s decomposition. The field is concentrated in the first panel; the two on the right are where the structure of the problem is actually used.

generalist policies [92], [151], [148], [136] and [149]. In every one of them a single network takes a concatenated two-hand observation and emits a two-hand action. Two rows do not say which they are, *Bunny-VisionPro* [53] and *DexImit* [154], whose notes describe the rig and the data pipeline but never the policy’s own decomposition. The concentration is not the outcome of a comparison that was won.

Four of the 28 give each hand its own network, and the two papers that compare the choice disagree. *Bi-DexHands* [56], a benchmark row and so outside the 28, ships the MARL baselines and finds PPO over the full observation beats HAPPO and MAPPO “in most cases,” with the gap narrowing on tasks that need both hands, because PPO “can use all observations” where MARL sees only part. *BiDexHD* [164] concludes the opposite. Its independent PPO teachers score 74.59 percent stage-two tracking rate on trained tasks against 53.88 for a centralised policy over both observations. Both are simulation only, on different tasks and hands, so neither settles it. *ArtiGrasp* [157] trains one PPO policy per hand, and *Dynamic Handover* [31] and *DyDexHandover* [52] use MAPPO with one agent per arm-hand system.

One of the 28 assigns explicit leader and follower roles, and the same one is the only policy expressed in a relative frame. *AsymDex* [51] is that paper, and the only corpus paper that puts either mechanism inside a policy. It gives the dominant hand full finger and wrist control, restricts the facilitating hand to a 6-DoF base pose, and writes the dominant hand and the object in a frame attached to the object the facilitating hand holds. That cuts the observation from 176 dimensions to 88 and the action from 52 to 26. It ablates the two mechanisms separately, which no other corpus paper does. On Block in cup the full method scores 0.7701 over five seeds, against 0.1086 for relative frames without asymmetry and 0.0164 for asymmetry without them. *Twist Lid* transfers zero-shot at 18 of 20 real trials. The real rig pairs a 16-DoF *Allegro* with a 6-DoF *Ability Hand* because only one *Allegro* was available, so the roles are confounded with the hardware. Two other rows use a relative quantity without giving either hand a role: *DexWild* [167] appends the inter-hand pose to its observation, and *DexMimicGen* [55] applies one shared SE(3) transform to both arms’ source segments when generating data, which is

the mechanism offline rather than in a policy.

C. Benchmarks and datasets for two hands

Two purpose-built bimanual dexterous suites exist in the corpus, four years apart.

Bi-DexHands [56] is 20 tasks on two *Shadow Hands* in *Isaac Gym*, ordered by the infant age at which humans acquire the skill, at 2048 environments and a reported 30,000-plus FPS. Its measurement discipline is weaker than its coverage. It reports reward and normalised score, never a success rate. The only success flag in the code is `goal_dist < 0.03`, a 3 cm object-to-goal test that ignores orientation and exists only in the four catching tasks. Any success rate later work attributes to *Bi-DexHands* comes from that flag or its own definition.

Bench2Dex [162] is the more instrumented of the two. It runs 26 long-horizon tasks in *Isaac Lab* across 12 arm-and-hand embodiments, with roughly 1.3K teleoperated demonstrations in eight modalities, including a ray-cast tactile image that maps all 12 hand geometries into one representation. Success needs the terminal predicate held for 0.5 s. Matched, *GR00T N1.5* leads at 631 of 1300 rollouts, 48.5 percent, against ACT at 29.5, $\pi_{0.5}$ at 27.3 and *Diffusion Policy* at 12.9. Under combined perturbation those first two compress to 19.8 and 19.7. Tasks are not crossed with embodiments, so no contrast isolates the hand, and the authors state the hand and tactile models are not calibrated against matching physical hardware.

Ten corpus rows are bimanual datasets, and five are two-hand interaction capture. *ARCTIC* [168] gives 339 mocap sequences of 11 one-DoF articulated objects held in both hands, and deliberately labels over-shooting vertices as in contact, so its ground truth records interpenetration as contact. *TACO* [166] gives 2.5K bimanual tool-use sequences fitted markerlessly under an attraction loss and a penetration loss the paper says trade against each other. *OakInk-V2* [169] is the only corpus dataset publishing a penetration number for its own annotations, a mean depth of 0.25 cm over its 627 long-horizon sequences. *HOT3D* [170] gives 833 egocentric minutes with mocap-grade poses for both hands and up to six objects, and *GigaHands* [171] gives 2,034 markerless minutes from 56 subjects, neither with a penetration metric. These five feed the tracking work here: *DexMachina* [30] and *ArtiGrasp* [157] from *ARCTIC*,

BiDexHD [164] from TACO, ManipTrans [29] from OakInk-V2 and DexMan [147] from both.

D. Methods on the same axes

The method tabulation carries the bimanual papers on the same columns as every other method here, and three of those columns are worth reading together.

Real trials. AsymDex [51] reports 20 per task, HATO [60] 10 per condition, DexMimicGen [55] 20 on a Fourier GR1 and BiDexGrasp [5] 260 across 30 objects. ManipTrans [29] reaches real Inspire hands by open-loop replay with no trial count and no success rate. Seven of the 28 have no real robot at all: [157], [164], [30], [147], [117], [52] and [43]. The two piano rows outside the denominator, RoboPianist [41] and RP1M [42], are simulation-only as well.

Penetration. Most bimanual rows read “not addressed.” BimanGrasp [6] gates on it, failing any grasp whose total penetration across hand-object, self and inter-hand exceeds 1.5 mm, and reports that “penetration remains the primary cause of grasp failure.” BiDexGrasp [5] reports it as a number, 0.15 to 0.20 cm maximum depth for its own grasps. DexImit [154] carries a named hand-hand penetration term in its grasp-synthesis objective. DexMachina [30] resolves it once, replaying retargeted joints against a fixed object “to eliminate object penetrations” and measuring nothing during rollout. ArtiGrasp [157] declines the metric outright, because its baselines “include a physics simulation which exhibits no interpenetration.”

Comparability. BimanGrasp [6] reports 54.03 percent success in Isaac Gym at friction 3. BiDexGrasp [5] re-runs the same grasps in MuJoCo at friction 0.6 and gets 26.80 percent, at 1.52 cm penetration depth. Neither number transfers, and Section V-D’s re-implementation result is the same lesson on the training side.

E. Handover and in-hand transfer

Handover is the one bimanual task where the hands are unambiguously asymmetric, because one gives and one receives. All three corpus handover papers use a single shared reward across giver and receiver. None defines separate objectives for the two roles.

Dynamic Handover [31] states one formula, $r = r_{\text{dis}} + r_{\text{linvel}} + r_{\text{torque}}$, and never says whether thrower and catcher receive different decompositions of it. DyDexHandover [52] is explicit that they do not: “Both hands share aligned objectives, forming a fully cooperative relationship,” with one weighted sum feeding both agents. Only the thrower gets an extra KL regulariser toward a policy pretrained on human throws, which constrains style and not role.

The third case is weaker still. In [32] only the receiver is learned. The giver is a UR5e with an Allegro hand that “holds the object without moving during the whole episode.” There is one agent, one reward and no second policy, its corpus row accordingly records `bimanual: no`, and it is one of the rows the 28 excludes. The 94 percent often attached to this paper needs its conditions. It is Total Success, which counts “Indetermination” cases where the simulator failed to resolve

collisions and the object clipped through the giver’s hand, on the short prism, in simulation, over 100 episodes, with no real robot in the paper.

Nothing in the handover literature measures contact quality. Contact appears only as a positive signal, a boolean per-phalange touch in [32] and a boolean contact reward in [52]. The two systems that do handover well sidestep the problem. HATO [60] reaches 10 of 10 on a real slippery handover by imitating teleoperation with fingertip touch sensing and no physics model. Lin et al. [115] stage their reward with a discrete variable switching which hand’s fingertips are scored, and still calls handover its hardest task at 52.5 percent real success.

F. What transfers from single-hand work

The training recipe transfers intact. PPO with thousands of parallel environments, an asymmetric critic on privileged state, domain randomisation and distillation into a vision policy is the same in [165] and [115] as in Section V-B. Reward forms transfer literally, and the contact-goal reward $r_{\text{contact}} = \sum 1/(1 + \alpha d)$ appears in both.

Multi-task learning does not transfer, and the bimanual benchmarks show it clearest. Bi-DexHands [56] finds multi-task PPO and ProMP fail on MT4, MT20, ML4 and ML20. RoboPianist [41] finds F1 on the shared training song drops “from roughly 0.7 F1 for 1 song to almost 0 F1 for 16 songs,” with no positive transfer “regardless of the size of the pre-training tasks.”

Single-hand evaluation does not transfer either. Success criteria compound, because ManipTrans [29] fails a trajectory “if either hand fails to meet these conditions.” And object-centric metrics cannot say which hand caused the failure. BiDexHD [164] scores the steps where both objects track their references, and DexMachina [30] averages ADD across object parts before an AUC. Both say whether the task worked and nothing about which hand lost it.

One failure mode has no single-hand counterpart. Two hands can penetrate each other, and the corpus almost never looks. Three papers carry an inter-hand penetration term, and all three are static grasp synthesis: [6], [5] and [154]. Not one learned bimanual controller in the corpus measures or penalises hand-hand penetration during a rollout. PianoMime [43] ships a flag that turns inter-hand collision off in the physics.

VII. EVALUATION: HOW WE WOULD COMPARE THESE METHODS

A. What the field reports, and why the numbers do not compare

Of the 112 method rows in the corpus, 89 report a real-robot experiment, 22 do not and one row is unsettled: 79 percent of all 112 rows, or 80 percent of the 111 the note settled. Among those 89, 70 state how many real trials produced the headline number, which is 79 percent of them and 62 percent of all 112 method rows. The 89 is the denominator that belongs to this statistic: the 22 rows with no real robot cannot state a real trial count, and counting them as silent turns a definitional impossibility into a reporting failure. Thirty-nine rows state a count of unseen test objects, 35 percent. Ninety-eight state

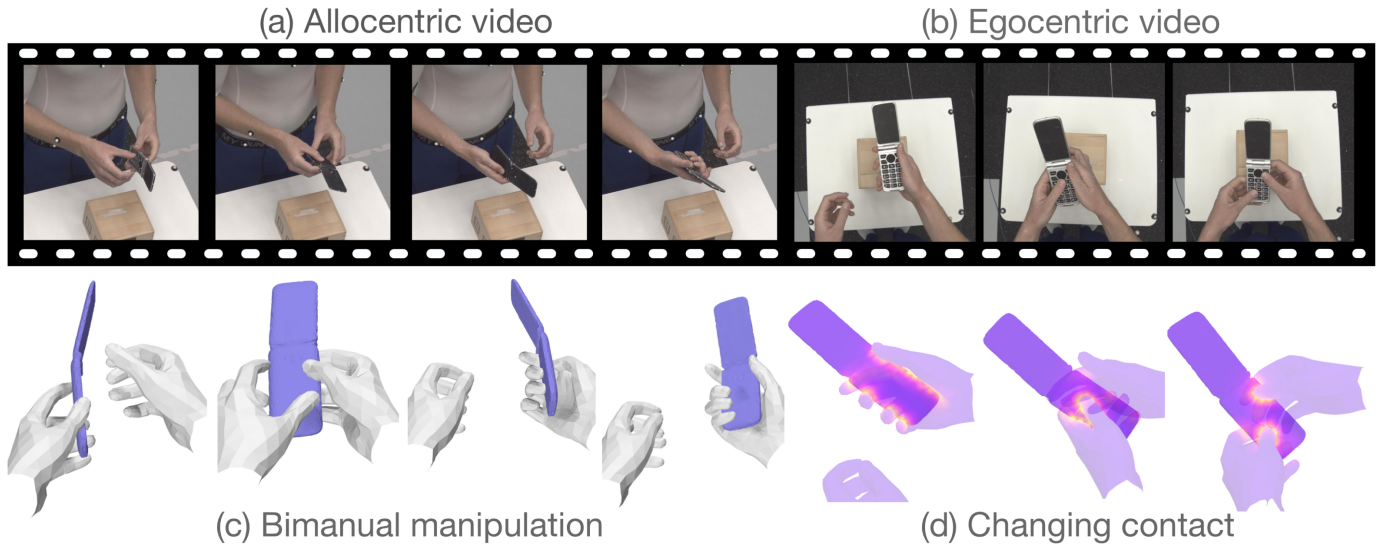


Fig. 21: A bimanual hand-object capture, from source video to the fitted meshes and the dense contact field between them. The contact annotation is inferred from tracked geometry, which is why its accuracy is bounded by the fit rather than measured at the surface.

Reproduced from [168]; permission not yet sought, see the register.

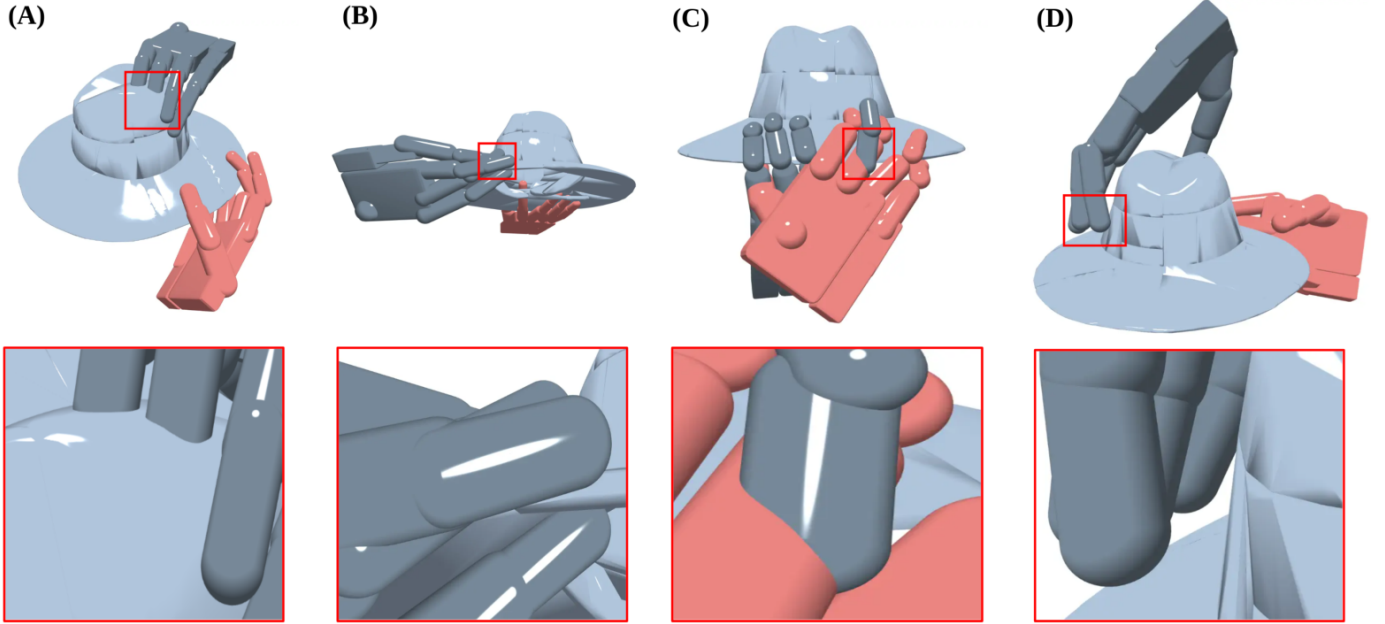


Fig. 22: The failure modes of synthesised two-hand grasps, with the contact region enlarged: fingers inside the object, fingers inside the other hand. At the scale of the whole grasp all four look plausible, which is why a success rate quoted without a penetration measure says little [6].

Reproduced from Fig. 9 of Y. Shao and C. Xiao, “Bimanual Grasp Synthesis for Dexterous Robot Hands,” arXiv:2411.15903, licensed under CC BY 4.0; cropped to Fig. 9 alone.

how a rollout is scored, 88 percent. Sixty-two released code and 46 did not, with four rows unsettled: 55 percent of all rows, or 57 percent of the 108 the note settled. Fig. 24 draws these six shares, each against the denominator that belongs to it.

These are counts of what this survey captured, not of what papers reported, and every one is a floor. A row in corpus/rows holds a scalar. A paper that reports ten trials on each of nine tasks, or a scoring rubric instead of a threshold, or a count spread over four tables, has nothing the extraction can reduce to one integer, so it produces a null, and a null is then indistinguishable from a paper that said nothing. The bias

runs one way: every miss converts a reporting paper into a silent one, and the survey’s argument is that the field reports badly, so the artefact flatters the argument. Section V-H makes the same disclosure about the paper/code count, subtracting the 13 disagreements that are limitations of this survey’s own parsing before declaring which number to quote, and the coverage statistics above need it more.

So the nulls were audited by hand against the notes they came from, and the numbers above are the audited ones. Of the 34 method rows with a real robot and no trial count, 15 had the count written in their own note. π_0 [33] at ten trials per task, RDT-1B [158] at 139 across seven tasks, UMI [161]

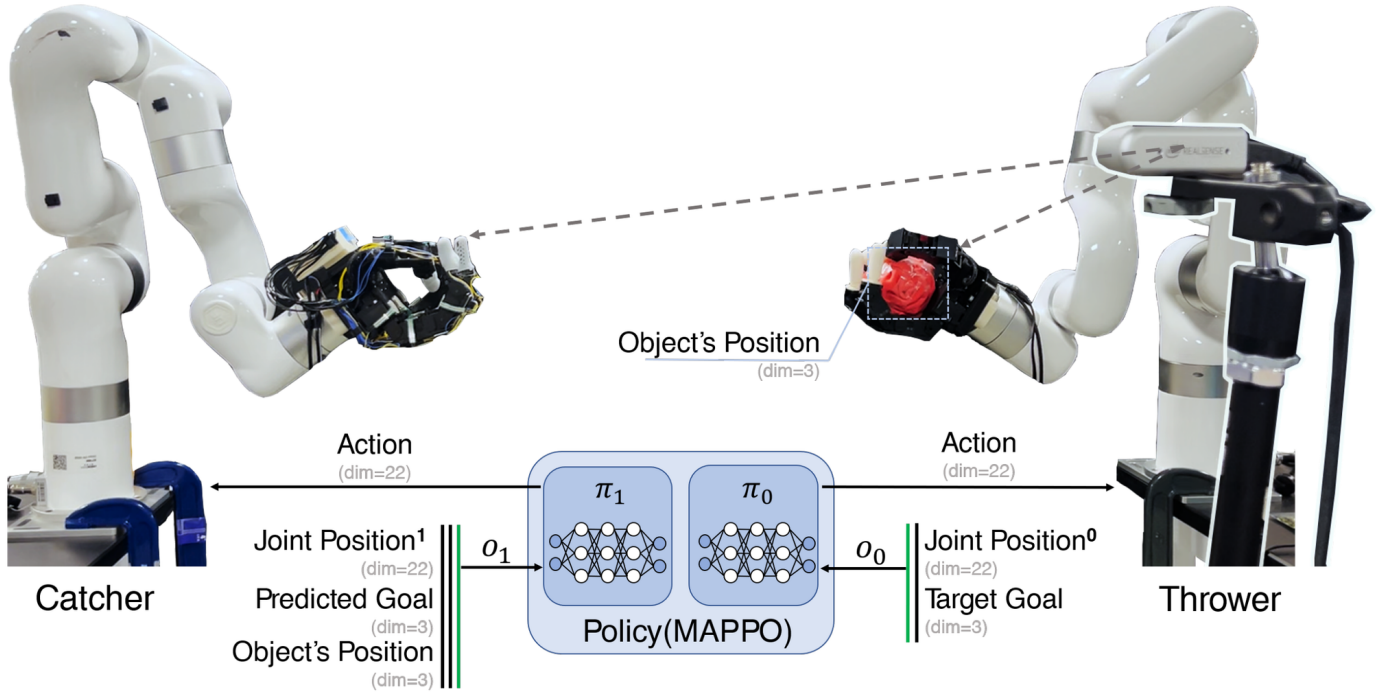


Fig. 23: Two arms and two four-fingered hands as thrower and catcher, with the observation and action dimensions each policy sees. The hands never touch: the coupling between them is the object’s flight.

Reproduced from [31]; permission not yet sought, see the register.

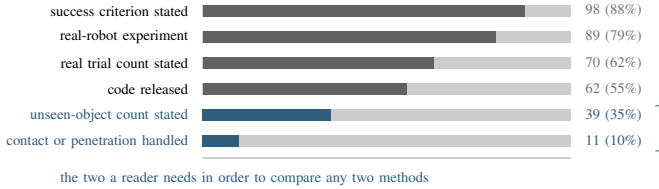


Fig. 24: What gets reported, and what does not. Each bar counts the corpus method rows that state the quantity, drawn against all 112 of them; the text gives the denominator that belongs to each share, which for the real-robot experiment is the 111 rows the note settled, for the trial count the 89 rows with a real robot, for released code the 108 rows the note settled, and for contact or penetration handling the 96 rows whose contact handling the note settled. Every bar is a lower bound, for the reason set out below. The two marked bars are the two a reader needs in order to compare any two methods: without a count of unseen objects a success rate has no generalisation denominator, and without contact handling it says nothing about whether the hand passed through the object. They are the two the field reports least.

at 260, $\pi_{0.6}^*$ [35] at 750, Gemini Robotics [150] at twenty per task, and ten more. That is 44 percent of the audited nulls, and it moved the headline from 55 rows to 70, from 62 percent of real-robot papers to 79 percent, and the “never says” figure from 34 of 89 down to 19. The success criterion moved further: of 33 null rows, 19 do state a criterion, so the count rose from 79 to 98. The unseen-object count moved least, 7 recovered from 80 audited nulls, 32 to 39. The audit only counted cases where the note itself carried the number, so a count that the note also missed is still uncounted, and 19 trial counts, 8 unseen-object evaluations whose object count the note never gives, and 4 criteria remain genuinely unsettled. Those nulls are now a defensible claim rather than an artefact. Every bar

in Fig. 24 should still be read as a lower bound.

The remaining gap is the one that matters. Nineteen of the 89 papers with a real robot never say how many times they ran it. A percentage with no denominator cannot be given an interval, so it cannot be compared with anything.

Where the denominator is stated it is small, and it is not one quantity. Some stored counts are per-cell, meaning ten trials on each task, or twenty per condition, or five per object. Others are grand totals over every cell. The rows now carry a `real_trials_kind` beside every value, and the two distributions are quoted separately. The 39 per-cell counts run from 5 to 100 with a median of 15 and quartiles at 10 and 20. The modal cell is 10 trials, in 16 rows, then 20, in 12. The 24 grand totals run from 12 to 750 with a median of 110. Pooling the two gives a median of 20 and a range of 5 to 1287, and that pooled figure is the one an earlier draft of this section quoted. It describes nothing, because Hora [15]’s 240 and Visual Dexterity [28]’s 20 are experiments of comparable size recorded on different bases. Seven further counts could not be assigned a basis at all. Fig. 25 is the per-cell distribution.

The per-cell count is the one a Wilson interval attaches to. For s successes in n trials, with $\hat{p} = s/n$ and $z = z_{1-\alpha/2} = 1.96$ at the 95 percent level, the bounds of that interval are

$$\frac{2n\hat{p} + z^2 \pm z\sqrt{4n\hat{p}(1-\hat{p}) + z^2}}{2(n + z^2)}, \quad (1)$$

which, unlike the normal approximation, stays inside $[0, 1]$ and does not collapse to zero width at $\hat{p} = 0$ or $\hat{p} = 1$. At 20 trials a reported 60 percent carries a 95 percent Wilson interval of 39 to 78 percent, and a reported 80 percent an interval of 58 to 92 percent. Overlapping intervals are not a test, so the

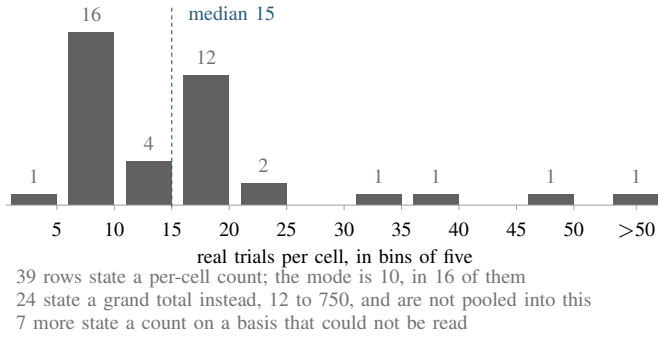


Fig. 25: Where a denominator is stated it is small. The 39 method rows whose stated real-trial count is per cell, meaning per task, per condition or per object, binned by fives and labelled on each bin’s upper edge, with the median of 15 marked. The modal evaluation is 10 trials in a cell, in 16 of the 39. The 24 rows that state a grand total over every cell are a different quantity, and pooling the two is the error this section withdrew: they are counted beneath the axis rather than drawn in the histogram.

point is made with one: 12 of 20 against 16 of 20 is $z = 1.38$, $p = 0.17$, and the difference is not established. Two methods separated by 20 points at 20 trials, the larger of the two modal cell sizes, are not separated at all.

The share of papers stating a count has risen, from 39 of the 65 rows before 2025, 60 percent, to 31 of the 47 rows from 2025 and 2026, 66 percent. The counts themselves have not. The median stated count is 20 in 2024, 22.5 in 2025 and 20 in 2026. Before 2024 the per-year medians rest on one to seven observations and should not be read as a trend.

The denominators also sit on different hardware. The 103 method rows that name their own hand give 78 distinct hand strings between them, and this survey applies no normalisation to those strings, so 78 is a count of strings and not of hand designs. Matching on the string, *Allegro* appears in 35, *Shadow* in 21, *Inspire* in 19 and *LEAP* in 12. A success rate on a 16-degree-of-freedom *Allegro* and a success rate on a 6-actuator *Inspire* hand are not measurements of the same thing.

Nor are the criteria, and the audit changed what that sentence can claim. *DexVerse* [172] counts *PickCube* a success when the cube is “lifted at least 0.20 m above its resetting height”. *Bench2Dex* [162] requires its terminal predicate to hold for a continuous dwell time of 0.5 s, to reject transient contacts. The *Colosseum* [173] counts an episode successful “if the model completes the task fully”. *DexTrack* [104] reports every success rate as a pair under two threshold sets, which on *GRAB* gives 46.70 and 65.48 percent for the same rollouts. Of the 14 rows that still state no criterion, ten have no success predicate at all. They report radians rotated or time-to-fall and never define a success, which is a fact about the paper rather than a gap in this survey. And four are unsettled by the note. What the audit found in the other 19 was mostly not a threshold: eleven score by rubric or staged partial credit, five judge binary completion against a task description by eye, two defer to a benchmark’s own definition, and exactly one, [35], states a verbatim numeric threshold. A rubric is a milder failure than silence and a worse one than a threshold, because it is reproducible inside a lab and not across two.

B. The axes that matter

Seven quantities dissociate in the published data, so they have to be reported separately.

Task success. Binary success discards the difference between near-misses and inaction. Snyder et al. [174] put it plainly: “a policy that completes 90% of the task is clearly better than a policy that is frozen the whole time, yet their success rates would be identically 0%”. In [175] policy C scores 17 percent overall on the pancake task while picking up the spatula and flipping the pancake in 23 of 23 attempts.

Robustness to perturbation. The *Colosseum* [173] measures a 30 to 50 percent success drop under single perturbation factors and at least 75 percent under all 14 together. In [162] *GR00T* N1.5 leads the matched condition at 48.5 percent and falls to 19.8 percent under combined shift, while $\pi_{0.5}$ goes from 27.3 to 19.7 and retains the most at 72.1 percent. The ranking at the anchor is not the ranking under shift.

Generalisation to unseen objects. Only 39 rows state a count and the median is 11 objects.

Physical plausibility of the contact. Eleven of the 96 rows whose contact handling the note settled address it, 11 percent, with 16 rows unknown. Section VII-C takes them apart.

Sample and wall-clock cost. Thirty-one of 112 rows state a parallel environment count and 18 a simulated episode count. *RoboPianist* [41] is the exception, at 5 million samples per song and roughly 5 hours per run on four Tesla K80 GPUs.

Real-robot transfer. Simulated rank order is not real rank order. *AutoEval* [176] scores *Open- π_0* on *put-eggplant-in-sink* at 6 of 50 in *SIMPLER* and 47 of 50 on the real *WidowX*. *Badithela* et al. [177] state the limit directly: “the simulation-to-real gap precludes rigorous statistical inferences about real-world outcomes from simulation results alone”.

Reproducibility. 62 rows released code that could be parsed against the paper, and 38 of the 112 rows record a disagreement of some kind between the paper and that code. All 38 released code, so the raw rate among code-releasing rows is 61 percent. That raw rate is not the finding, because the 38 are not one thing. Section VIII-A classifies them: 9 contradictions, 13 limitations of this survey’s own parsing, 8 components never released, 4 version skews and 4 inconsistencies internal to a paper. Only the contradictions are a finding about the work rather than about this survey, so 9 of 62 code-releasing rows, which is 15 percent, is the figure this section and Table II use. *PhysHOI* [1] is the clearest of the 9. It lists a non-zero object-orientation weight for *GRAB* in its Table 4, and its released `compute_humanoid_reward` hard-sets that orientation error to zero, so the reward that produced the published numbers tracked the object in position only.

C. Physical plausibility as a first-class metric

Eleven method rows handle interpenetration in any form: three penalise it, three measure it, five constrain it. The denominator is 96, not 112, because the `penetration` field is null for 16 rows, and a null there means the note did not settle the question, not that the paper ignored penetration. Eleven of 96 is 11 percent.

112 method rows in the corpus

96 of them whose note settles the question

11 of those 96 address interpenetration in any form

4 of those 11 inside a closed-loop policy



report a penetration number for the rollouts of their own trained policy

Fig. 26: The quantity most specific to a hand is the one closed-loop policies do not record. Each bar is drawn against all 112 method rows, the light track behind it. Of the 96 rows whose note settles the question, 11 address interpenetration in any form, and 4 of those are closed-loop policies: the other 7 are grasp synthesisers, trajectory optimisers or a contact model, which score a configuration rather than a controller. Of those 11, 0 report a penetration number for the rollouts of their own trained policy, which is the last row and the point of the figure: every one of them sits on the reference side of the reference-versus-rollout split.

That eleven is not a claim that penetration goes unmeasured in general, and it would be wrong read that way. Outside closed-loop control the quantity is a standard comparative column, and has been for years in grasp synthesis and in hand-object reconstruction: OakInk [4] tabulates penetration depth, solid intersection volume and simulation displacement across methods, and inside this corpus BiDexGrasp [5] prints a penetration depth beside a prior method’s, BimanGrasp [6] fails any grasp that exceeds 1.5 mm of total penetration, and TopoRetarget [7] reports a maximum penetration and a share of frames past 2 mm against a baseline retargeter. The finding is narrower than the field and concerns learned closed-loop control: all eleven score a pose or a reference trajectory, four of them are closed-loop policies, and we found none that reports the measurement for rollouts of its own trained policy.

Where in the pipeline those eleven act is the reference-versus-rollout split that Section I takes from [19]. A reference is a pose or a trajectory scored before execution, and a rollout is what the trained policy actually did. What this section supplies on that axis is a measurement method and a count, and it does not supply a threshold. The count is the eleven of 96 above, with four closed-loop policies inside it and none we found reporting a number for its own rollouts, which is Fig. 26. The method is the plausibility row of Table II: maximum and mean penetration depth over the evaluation rollouts, on a dense surface sample, computed by code that never entered the reward or the termination rule. The threshold is borrowed, and Section VII-G says from where and why it does not bind. Six of the eleven are grasp synthesisers or trajectory optimisers, namely [5], [6], [154], [46], [7] and [156], and Castro et al. [98] give a contact model rather than a controller. That leaves four closed-loop policies in the whole corpus: [178], [30], [104] and [120].

The reason the number is four is a measurement trap. A quantity a policy optimises cannot also judge it, because the policy learns the measure rather than the property the measure

stands for. PhysHOI [1] documents the failure in the clean direction. Its kinematic imitation reward was maximised by not touching the object at all, because contact perturbed the reference trajectory: “the policy may learn not to touch the object and falls into a local optimal”. A contact-graph reward closed the hole, and that reward reads a force threshold rather than geometry, so it constrains contact presence and says nothing about penetration depth.

DexTrack [104] shows the trap in the other direction. It defines a maximum hand-object penetration depth over all frames in its Appendix B, and applies it only to the input kinematic references, as one component of a reference-quality score. No penetration number appears for any of its own rollouts, in simulation or on the LEAP hand. Tolerance of the failure is then reported as a result: “Despite severe hand-object penetrations in Figure 4c and Figure 4a, the hand still interacts effectively with the object, highlighting the resilience of our tracking controller”.

TopoRetarget [7] is the strongest case in the corpus and still stops one step short on the same reference-versus-rollout line. It constrains penetration during retargeting with a 1 mm soft tolerance and a 30 mm hard bound, and it reports two numbers on 25 ContactPose grasps: a maximum penetration of 1.07 mm and 0.00 percent of frames above 2 mm, against 22.22 mm and 96 percent of frames for its GeoRT baseline. Then a PPO controller tracks those references, and its four reward terms and its five termination criteria govern object pose, link position, joint error and action smoothness, never penetration. The constrained quantity is the reference, and the rollout is not re-measured.

Definitions are not shared either. GRAB [20] estimates contact by proximity, because “contact cannot be directly observed”, with a 4.5 mm tolerance, and reports that “‘Use’ grasps have 3.25 ± 0.68 mm average penetration”, without saying whether 3.25 mm is a maximum, a mean or a median. OakInk [4] supplies the fullest published vocabulary: penetration depth, solid intersection volume and simulation displacement. Its Table 3 scores the GRAB GrabNet split at 2.53 cm penetration depth, against GRAB’s own 3.25 mm. The two differ by a factor of about eight, and neither source states its distance function precisely enough to reconcile them, and the two are not scored over the same grasps either, since GRAB’s figure is over its own captured “use” grasps and OakInk’s is a model’s output on the GrabNet split, so a difference of population and a difference of definition are confounded in the same ratio.

The analytic tradition scored a grasp without simulating it, and Ferrari and Canny’s measure [45] and Roa and Suárez’s review [179] are its wrench-space reference points. Neither could be obtained. The first DOI fetch returned HTTP 202 and the second a Springer JavaScript interstitial, so both are cited by metadata only and no definition here rests on their contents. The learned literature has not replaced that tradition with anything it measures on its own rollouts. Penetration is a quantity between meshes, so it needs a simulator or a mesh reconstruction, and the protocol below treats it as a simulation-only axis.

D. Statistics

Kress-Gazit et al. [175] give the field’s reference protocol, and it prescribes process, not numbers. Write the success criteria before the run and have someone other than their author score the runs. Match initial conditions with image overlays. Interleave the policies blind within one session. Report counts rather than percentages, alongside the initial conditions and the failure modes. Use a posterior over the Bernoulli parameter instead of a point estimate. It prescribes no minimum trial count anywhere and no frequentist confidence-interval width anywhere. Its own example report uses 10 initial conditions with two runs each, 20 evaluations per policy, and its worked case shows what 20 buys. Pancake success of 15 of 18 against 11 of 17, nominally 83 against 65 percent, leaves a 0.11 posterior probability that the worse policy is actually better. At 150 of 180 against 110 of 170 the same rates separate.

The TRI LBM Team [180] supplies the missing numbers by fiat rather than derivation: 50 rollouts per task per policy per condition on hardware, 200 in simulation, blind, with randomised policy order inside per-initial-condition bundles. It replaces confidence intervals with Beta-posterior violins and corrects its pairwise tests under Bonferroni, “unless otherwise noted”. Its own warning is the strongest sentence in this literature: “there is significant risk that many robotics papers are measuring statistical noise due to insufficient statistical power”.

The rest fall short of their own advice. RoboArena [181] runs 612 double-blind pairwise comparisons across seven institutions and 4284 rollouts, and reports no confidence intervals and no per-policy trial counts. The Colosseum [173] evaluates 235 test sets at 25 episodes each with “one training seed and one evaluation seed” per baseline and gives no intervals in simulation. AutoEval [176] runs 50 trials per policy per task and calls ± 10 percent “the natural variance of robot evaluations”, without giving the formula behind the intervals it plots.

Two papers do give usable numbers. Badithela et al. [177] pair real and simulated trials and de-biases the simulation with a rectifier, saving 20 to 25 percent of hardware trials at paired correlations of roughly 0.6 to 0.7, and nothing at all at a correlation near zero. The 25 percent figure is the better of its two reported settings, the DP case at $\rho = 0.702$. Its decision rule is exact: combining helps only when the rectifier variance is below the variance of the real evaluations. Snyder et al. [174] give the largest saving. On the LBM rubrics their sequential test on graded scores cuts simulated evaluation by about 70 percent and hardware by about 45 percent, 286 trials against a nominal 500, with per-task decisions landing in 12 to 36 paired trials. On RoboArena’s data a 30-point gap on continuous progress scores reaches significance in 18 trials, while a 20-point gap on binary success needs about 80.

E. A proposed protocol

Every count below is printed by `tools/make_eval_tables.py -derive`, and each axis is derived for the statistic that axis actually reports: a single rate takes a Wilson half-width, a matched comparison takes McNemar, a ratio

takes the standard error of the log ratio, a correlation takes the Fisher-z interval.

Fix the width first, then read off the count. Take a 95 percent Wilson interval on a single reported rate, at the worst case of $\hat{p} = 0.5$. Writing the half-width of (1) as

$$w(n, \hat{p}) = \frac{z\sqrt{4n\hat{p}(1-\hat{p}) + z^2}}{2(n + z^2)}, \quad (2)$$

the count a target width w_0 demands is $n_{\min}(w_0) = \min\{n : w(n, 0.5) \leq w_0\}$, which gives

half-width w_0 (points)	20	15	10	5
$n_{\min}$ (trials)	21	39	93	381

Ten points is the coarsest width at which a single rate is worth printing, so the absolute-rate minimum is 93, rounded to 100. At 100 trials a reported 80 percent has an interval of 71 to 87 percent, and a reported 50 percent has 40 to 60. A comparison is a different question and a harder one: two rates each carrying ± 10 points do not resolve a 10-point difference between them, because the difference’s standard error is larger by a factor of $\sqrt{2}$, so the width argument sets a floor on what is worth reporting and not on what can be compared.

For the A/B comparison the relevant calculation is power, and the design is paired. Table II matches initial conditions by image overlay and interleaves the two policies in one session, so the unit is a matched pair and the count follows McNemar, which depends on the discordance rate. The share of initial conditions on which the two policies disagree, and not on the two rates alone. Write $\delta = |p_1 - p_2|$ for the gap between the two success rates, π_d for that discordance rate, $z_{1-\alpha/2} = 1.96$ at $\alpha = 0.05$ and $z_{1-\beta} = 0.84$ at 80 percent power; then the number of matched pairs is

$$n_{\text{pairs}} = \frac{[z_{1-\alpha/2}\sqrt{\pi_d} + z_{1-\beta}\sqrt{\pi_d - \delta^2}]^2}{\delta^2}. \quad (3)$$

To separate 50 from 70 percent, which is $\delta = 0.20$, at $\alpha = 0.05$ with 80 percent power, (3) asks per arm for

discordance π_d	0.2	0.3	0.4	0.5
n_{pairs}	37	57	77	96

The protocol assumes 0.3 and asks for 57, and states the sensitivity rather than hiding it, because 0.5 is the discordance the same two rates produce when the pairing buys nothing, and at that value the paired count returns to the 93 per arm an unpaired test would need. The saving from pairing is real but smaller than the pair counts suggest, since a pair costs two rollouts: 57 pairs is 114 rollouts against 186. An earlier version of this section quoted 93, 169 and 387 per arm for gaps of 20, 15 and 10 points, which are correct for independent arms and are the wrong test for this protocol. That 93 was also the same integer as the half-width calculation in the paragraph above, which is a coincidence of the worst-case arithmetic and not a second derivation of the same number.

One hundred is a cap and not a bill, because on a graded score a sequential test reached its decision in 12 to 36 paired hardware trials in [174]. At 30 rollouts a continuous score already carries

a half-width of ± 0.36 standard deviations, which is why a graded score can stop where a binary one cannot. A cell that stops early does not report a Wilson interval. Optional stopping breaks the coverage of a fixed- n interval, which is the reason Snyder et al. [174] and Badithela et al. [177] use anytime-valid betting intervals rather than Wilson, so Table II asks a cell run to a fixed 100 for a Wilson interval and a cell stopped early for a confidence sequence, and never for both. Intervals are also marginal rather than simultaneous. Table II has seven axes and Table III twelve methods. At 84 independent 95 percent intervals, four excursions are expected by construction, so a paper comparing k policies on m tasks corrects its $k(k-1)/2$ pairwise tests to a global 95 percent level, as the TRI LBM Team [180] does, or says its intervals are not simultaneous.

Simulation is cheap, so simulated cells take 200 episodes, giving a 6.9-point half-width. Perturbation axes are screened rather than certified, and 40 per axis buys a 15-point half-width on each axis’s own absolute rate, which is enough to rank the axes and pick the two worst for hardware. It is not enough for the ratio to the anchor that an earlier draft asked each cell to report. At 40 trials in each arm, a fall from a 0.50 anchor to 0.30 is a ratio of 0.60 with a 95 percent interval of 0.34 to 1.06, which contains 1: at the screening count you cannot establish that the perturbation hurt at all. Certifying that same drop takes 101 per arm by the log-ratio standard error, so Table II now asks for absolute rates with their own intervals at 40, reports the ratio without an interval, and prescribes 101 before any claim that a named axis hurt.

For unseen objects the resampling unit is the object and not the trial, so 20 objects at 5 trials each gives 100 trials and an object-level half-width near 20 points. That 20 points is the Wilson width at $n = 20$ and it treats each object’s outcome as a single Bernoulli draw, which the five within-object trials are not. It is the right order of magnitude and the assumption belongs in the cell. A 10-point claim about an object distribution needs about 93 objects. Seven of the 39 rows that state an unseen count reach that: 225 in [6], 241 in [155] and [156], 360 in [21], 500 in [138], 2029 in [178] and 503409 in [57]. For a continuous score the half-width is 1.96 standard deviations over the square root of the count, $1.96 s/\sqrt{n}$, so 100 rollouts give ± 0.20 standard deviations, and the unit is the rollout because frames within one are correlated.

The transfer axis is the one where 100 is least defensible. On 100 matched pairs a measured correlation of 0.70 carries a Fisher- z interval of 0.58 to 0.79, a half-width of about 0.10 that 130 pairs would be needed to guarantee. That is enough to establish that a simulator tracks reality at all, and it is not enough to separate [177]’s useful regime from its marginal one, since those differ by about 0.11 in correlation. Both limits come within 0.05 of the estimate only at about 457 pairs. Table II states which of the two decisions each count supports rather than leaving a reader to assume the larger one.

F. Table III, an empty results matrix

The rows are the 12 most-mentioned dexterous-hand policy methods in the corpus, and the rule is the one `tools/make_eval_tables.py` implements, stated here in the

same words. A candidate is a method row with a non-null hand; its hand string must not contain “parallel” or “gripper”; it must carry at least one paradigm tag that produces a closed-loop policy and must not carry `teleop-system`, because an interface is scored on latency and operator effort rather than on a policy’s success rate. And its name must be at least four characters, so that a short string does not match everything. Candidates are then scored by the number of other corpus papers whose parsed text in `papers/md` contains the name, and the top 12 by count, ties broken by key, are the rows.

Two corrections to that ranking are worth stating, because both changed it. The match is on a whole word. Under the bare substring test an earlier version used, “UniDex” matched inside “UniDexGrasp” and “UniDexGrasp++”, and [182]. A 2026 paper. Sat sixth in a ranking over a corpus written mostly before it, on 34 mentions that belonged to a different work. As a whole word it has 3 and it is not in the table. And the interface rule is now applied to every row that carries the tag rather than only to rows that carry nothing else, which drops [125] at 34 mentions, [183] at 28 and [128] at 23, along with [124], which the earlier prose already excluded by hand. The mention counts are printed under the table so a reader can audit them.

The ranking is not one quantity even so. A method’s name is taken from the first line of its note, which yields an acronym for some works and a full title for others, and a title is matched mostly inside reference lists while an acronym is matched in running text. Those have different base rates, so the table marks which kind each row was matched on and the two kinds are not comparable with each other. Mention counts are counts of mentions and not of use, as `METHOD.md` records.

Every cell is empty. This survey re-ran nothing, and no cell can be filled at the denominator Table II asks for. DeXtreme [26] comes closest and is the reason the claim is stated that narrowly: it reports a criterion, a trial count and an interval. Object orientation within 0.4 rad of target, 27.8 ± 19.0 average consecutive successes with the $\pm$ a 90 percent confidence interval. On 10 trials. The method tabulation is not a counter-example either, though it looks like one: it carries `trials`, `unseen obj`, `penetration` and `code` columns for all 112 method rows, including all 12 of these. It records what each method reported. Table III asks instead for what Table II defines. A value with an interval, a stated denominator and a criterion written before the run. And no reported value meets that. The first row of Table III is a worked example so that the format of a cell is unambiguous. Every number in it is fabricated and labelled as such.

G. What would have to be true

The bill comes first. Per policy and per task the protocol asks for 57 matched trials on the anchor set, which is the paired count from §VII-E, and 100 on the unseen-object set, at 20 objects and 5 trials each, with robustness and plausibility absorbed by simulation. A two-policy comparison on three tasks is then 342 real rollouts on the matched set and 600 on the unseen-object set, 942 in all, and at one minute per rollout including the reset that is about 16 hours of robot time, before

TABLE II: The proposed evaluation protocol, this survey’s own contribution and the reason it is in the body rather than in the repository. No cell is –: every cell is a proposal rather than a reading of a source, and this is the one table in the paper that no corpus row can fill.

axis	what is measured	minimum trials	reported with it
Task success	fraction of episodes meeting a criterion written before the run, and a graded rubric score in $[0, 1]$	100 real and 200 sim per policy, task and condition for an absolute rate at a 10-point interval; a comparison alone needs 57 matched pairs per arm	raw counts, the criterion verbatim, the rubric, the initial-condition protocol, and an interval whose kind is named
Robustness	success under each perturbation axis, and the ratio to the unperturbed anchor	40 per axis per policy to rank the axes; 101 per axis and 101 at the anchor before any claim that a named axis hurt	both absolute rates with their Wilson intervals, the axis ranges, and the ratio without an interval unless the certifying count was run
Unseen objects	mean over held-out objects of per-object success	20 objects at 5 trials to screen; 93 objects for a 10-point claim	the object list, the per-object rates, a bootstrap interval over objects, and the within-object trial count
Physical plausibility	maximum and mean hand-object penetration depth per rollout, and the fraction of frames above 2 mm	100 rollouts, simulation only	an interval over rollouts, the sample density, the threshold, the solver’s depenetration settings, and at least one rendered rollout
Sample and wall-clock cost	environment steps and wall-clock to the checkpoint that produced the headline number	3 training seeds	the range over seeds, and the statement that 3 seeds is a range and not an interval
Real-robot transfer	paired real and simulated outcomes on matched initial conditions, their correlation, and the rectifier variance against the real variance	the 100 real trials paired to 100 sim; about 457 pairs to separate two correlations 0.1 apart	the correlation with its Fisher-z interval, the variance ratio with its bootstrap interval, and which decision the count supports
Reproducibility	the released config and checkpoints against the paper’s stated objective	not a trial count	the file and line of every disagreement, or an explicit statement that none was found

Two columns rather than one: four columns of clauses do not fit 252pt without breaking words. The *how* and *why* columns of the long form, which carry the design of each measurement and the reason for each count, are derived in Section VII and generated by `tools/make_eval_tables.py`; a paragraph of rationale does not belong in a float. The bill for two policies on three tasks is 342 matched plus 600 unseen real rollouts, against 1200 on the independent-arm count the paired design replaces.

TABLE III: The matrix, for someone else to fill: the 12 most-mentioned dexterous-hand policy methods against the axes of Table II. All 84 cells are empty. An empty cell here is not –: it is a measurement nobody has made, which is the point of the table, and no cell in it has a source to be missing.

method	task success	robustness	unseen objects	plausibility	cost	transfer	reproducibility
<i>worked example</i>	0.72 $\pm$ 0.09 (100)	lighting 0.41 (40), ratio 0.57	0.55 [0.41, 0.68], 20 obj	3.1 / 0.8 mm, 0.12	1.2e9 steps, 46 GPU-h	0.61 [0.47, 0.72]	train. yaml:88
OpenAI [25]*							
DexMV [138]							
DAPG [184]*							
DeXtreme [26]							
DexCap [131]							
Hora [15]*							
Visual Dexterity [28]*							
Yin et al. [49]*							
UniDexGrasp [156]							
PDDM [116]*							
HATO [60]*							
DexPoint [48]							

The first row is a worked example and every number in it is fabricated. Rows are ranked by the rule of Section VII, on whole-word matches over `papers/md`: OpenAI 64; DexMV 50; DAPG 49; DeXtreme 44; DexCap 40; Hora 34; Visual Dexterity 32; Yin et al. 27; UniDexGrasp 27; PDDM 25; HATO 23; DexPoint 22. A mention count is a count of mentions, not of use. * marks a work matched on its title rather than on a short name; a title is matched mostly inside reference lists and a short name in running text, so the two kinds of count are not comparable. Each column is filled in the units Table II asks for.

failed resets, repairs and scoring. The same bill computed from the independent-arm count, which an earlier draft used, was 1200 rollouts and 20 hours. The pairing removes about a fifth of it rather than the four fifths the pair counts suggest, because a pair is two rollouts and only the anchor set is paired. AutoEval [176] ran about 850 episodes in 24 hours on a WidowX with three human interventions, and had to pause 20 minutes every 6 hours once the motors overheated. A tendon-driven multi-finger hand is more fragile, so 16 hours of rollouts is most of a week of calendar time.

That bill is large but not unprecedented. AutoEval [176] records that evaluating OpenVLA against its baselines took

more than 2500 rollouts and more than 100 hours of human labour across three institutions, and the TRI LBM Team [180] analysed about 1800 real rollouts across nine hardware stations. What is unprecedented is paying it for a single dexterous-hand paper, where the modal per-cell count is 10 trials and the median per-cell count 15, and where the median stated count has been 20 or 22.5 in each of the last three years.

Four things would have to change. Reviewers would have to reward 57 matched trials on one task over 20 unmatched trials on five, and nothing in the corpus suggests that is happening. The loop would have to be automated, and AutoEval [176] shows it is buildable for a gripper at one to three hours of



Fig. 27: The apparatus behind a headline in-hand reorientation number: a caged rig carrying sixteen motion-capture cameras and three RGB cameras, beside the simulated scene the policy was trained in. Reproducing the result means reproducing the left panel.

Reproduced from [25]; permission not yet sought, see the register.

setup, while stating that it supports binary success only and no robustness axes. Somebody would have to run the penetration measure on their own rollouts, which is a choice rather than a capability: Section IV-B shows the depth is computable from the poses and the meshes in a few lines of Warp, and IsaacGymEnvs already ships a task that does it every step. An earlier draft of this section made the engine the barrier, and that claim is withdrawn, because the released code refutes it. And the comparison would have to be sequential, because the savings in [174] are the only reason 100 is a cap rather than a cost.

Four limits apply to the proposal itself. This survey re-ran no method, so every count in Table II is derived from an interval width, a power calculation or another paper’s measurement, and Table III is empty because we filled no cell. The counts are worst-case at $\hat{p} = 0.5$, so a method near 90 percent needs fewer trials for the same width and a method near 50 percent needs all 100, and the paired count additionally rests on an assumed discordance of 0.3, which no paper in this corpus reports. The perturbation axes are borrowed from The Colosseum [173] and SIMPLER [185], which run parallel-jaw grippers on rigid objects, where SIMPLER found physical parameters moved success rates by at most 15 percent. That is the sensitivity expected to grow with multi-finger contact, and nobody has measured it. The 2 mm penetration threshold is taken from [7] with no independent justification, and the captured human grasps in [20] sit above it at 3.25 mm, which makes 2 mm a simulator convention rather than a physical bound. So the three things this survey adds to the reference-versus-rollout axis are a count, a measurement method and a protocol slot for them, and a threshold is not among them. It is borrowed from one paper and reported as borrowed, and it will stay a convention until somebody measures penetration on rollouts across hands and engines and finds a value that separates behaviour a physicist would accept from behaviour they would not.

VIII. GAPS

A. What opening the repositories showed

This is a result rather than a gap, and a result about publishing practice in robot learning: the dexterous corpus is its sample, not its subject.

Sixty-two method rows released code that could be parsed against the paper, and 38 carry a recorded discrepancy, drawn in Fig. 1. The classification is the finding: nine contradictions, where the paper states one value and the shipped code demonstrably states another; thirteen limits of this survey’s own parse, which captured signatures or a truncated body rather than the component; eight cases where the code was never released; four version skew; four inconsistencies inside a paper with no code involved. Nine is the number to quote, eight at high confidence, with PenSpin [88] held at medium against an innocent reading a direct code read would settle.

The first count was sixteen. An adversarial re-reading withdrew seven accusations. ManipTrans, Eureka, Open-TeleVision, DexMachina, ArtiGrasp and GraspXL [29], [27], [126], [30], [157], [57], and the domain-randomisation half of the charge against DexPBT [117]. Two refuted by the accused repository’s own README, two resting on reward code never in the parse, one charging the code with structure the paper prints, one against a paper with no reward function. That left ten, and ten held until the letters to the authors were drafted.

Drafting the letter to OmniH2O [39] meant reading its accusation once more before it went out, and that second reading is what broke it. Four of its five reward-weight comparisons match the paper’s own table to the digit once a systematic $\times 1.25$ curriculum factor is applied, and only the stumble weight differs, by a factor of about a million: a typo signature in the paper’s own table, not a policy trained on a different objective. The work’s hands are also driven open-loop from a VR pose, outside the policy and outside the reward, which made it a poor fit for a dexterous-manipulation reward census whatever the weight said. The charge is withdrawn and the row reclassified as an internal inconsistency, which is where the count above already places it. **The survey has now withdrawn eight accusations in total: seven of them under adversarial review, and the eighth at the point of writing to the authors, because someone sat down to write the letter and looked at the evidence again.** PenSpin [88] is narrowed rather than withdrawn, the same way the charge against DexPBT was narrowed above: the half of it that said the released code disables the paper’s tactile channel is dropped, because the configuration read has proprioception-only observation dimensions consistent with the student policy rather than the oracle, and the paper never claims the student has tactile input; the half that stands is that the appendix states a randomised disturbance force and the shipped configuration sets its scale to zero. Each retraction and narrowing is recorded in its row’s `mismatch_review` field: a survey that names people should carry its corrections beside its accusations, in print and not only in the corpus.

PhysHOI [1] survived every attempt to break it. Table 4 weights object rotation at 0.1 for GRAB, and the released `compute_humanoid_reward` sets the object rotation and

rotation-velocity errors to `torch.zeros_like`, unconditionally, so the dataset exemption the paper states does not cover it. The reward behind its 95.4 percent tracked the object in position only, and its own position-only success criterion could not have caught that. Robot Synesthesia [86], at the other end, prints its six weights as symbols and released nothing, so that objective exists in no machine-readable form. Hora [15] alone discloses its own gap, in its README.

Nine is a floor, since forty-six method rows released nothing to check and four more are unsettled.

What would close it: publish the reward table generated from the released config at a named commit, so a reviewer diffs two artefacts instead of reading two documents.

B. No closed-loop policy records interpenetration for its own rollouts

Eleven method rows handle interpenetration at all, of the ninety-six whose notes settle the question, and only four of them are closed-loop policies (Fig. 26). Every one of the eleven sits on the reference side of the reference-versus-rollout split, scoring a pose or a trajectory before execution rather than the behaviour that followed. The gap is therefore specific to learned closed-loop control rather than general. Grasp synthesis and hand-object reconstruction report penetration depth and intersection volume comparatively, and [4], [6], [5] and [7] do so within this corpus; what none of them scores is the behaviour a trained policy produced. The engines are not the obstacle: IsaacGymEnvs ships a task that computes a per-environment maximum interpenetration depth in Warp and gates its policy update on a 1 mm threshold, and Tactile Genesis [103] offers two penetration-depth backends on Genesis geometry as sensors. The depth is computable by anyone from poses and meshes, and we found no paper that reports it for a dexterous rollout.

What would close it: maximum and mean penetration depth over the evaluation rollouts, on a dense surface sample, from a measure the policy never optimised.

C. The evaluation-methodology literature contains no dexterous hand

Seven corpus rows are evaluation protocols, a small denominator, and not one uses a dexterous hand: Badithela et al. [177] run a parallel-jaw gripper and the other six state no hand. Kress-Gazit et al. [175] set no minimum trial count and no confidence-interval width. *What would close it:* run the Table II protocol once, on one in-hand reorientation task with one 16-DoF hand, and release the rollouts as the first row of Table III.

D. Generalist policies run on a fraction of the degrees of freedom

The gap is size, not absence. Eighteen rows carry the generalist tag. Fourteen of them settle whether the reported evaluation ran on a multi-fingered hand, and eight of those fourteen did. The remaining four do not say, which is itself the smaller half of this gap. Of the eight, five state the hand's degrees of freedom: four at 6 and one at 12. The median is 6,

against 16 over the 48 reinforcement-learning rows that state one. So the generalist policies that do touch a hand run it at roughly a third of the actuation the reinforcement-learning literature assumes. The stronger claim, that no generalist reaches the 16-to-24 band, holds over those five stated counts and no further, which is what section 5.5 says. GR-Dexter [92] and EgoScale [149] name hands at 21 and 22 actuated degrees of freedom, so they do reach the band on paper, and neither settles whether the reported evaluation ran on that hand.

What would close it: one dexterous-hand task in the standard generalist suite, with the hand's DoF and vendor beside the number.

E. The hands that can be bought go unused

7 documented hands that can be bought or built from published designs take zero method rows between them: Unitree Dex5, Tesollo DG-5F, Proception ProHand, ORCA, RUKA, RUKA v2, BiDexHand. Only three of the fifteen simulator rows name a real hand at all, and the ones they do name are the field's defaults. 18 of the 33 hands in Table IV and Table V appear in no method row, but 11 are neither sold nor open and appear in none for that reason, so 33 is not the denominator for a software-lag claim. The rule that decides used from unused is one regular expression per hand against the method rows' own hand field, in `tools/hand_usage.py`, so the partition can be recomputed rather than argued about. Bench2Dex [162] compares 12 hands and DexVerse [172] six without stating a DoF count for any.

What would close it: a conformance suite for hand models, one URDF or MJCF per hand, with fixed joint-limit, mass and collision checks and a published pass or fail per engine.

F. Bimanual work runs on one coordination architecture, and it has been ablated once

Twenty-one of the 28 rows that Section 6 counts as putting a learned closed-loop controller on two multi-fingered hands run one policy over a concatenated two-hand observation. That choice has been compared twice, with opposite outcomes, and ablated once: AsymDex [51] scores 0.7701 on Block in cup over five seeds, against 0.1086 with relative frames but no role asymmetry and 0.0164 with asymmetry but no relative frames. All three handover papers share one reward across giver and receiver.

What would close it: one handover task, three architectures, the same hand and the same seeds, with separate giver and receiver returns.

G. Human data does not port across hands, and the map is usually unstated

Fifty-three method rows use human data and name 45 distinct hand strings between them. Twenty appear among the 30 teleoperation systems with a stated retargeting objective. The other 33 do not say how the human motion reached the hand, and the stated objectives do not converge, from a fingertip keypoint-vector energy in [125] to a per-joint regression from exoskeleton encoders in [85] to no retargeting at all in [145].

More correspondence is not better: Chen et al. [146] retarget the wrist only and beat both finger-joint and fingertip mapping, and their added fingertip-matching reward “does not yield benefits and even leads to lower performance”.

What would close it: a retargeting benchmark in TopoRetarget’s form, reporting contact precision, contact alignment, maximum penetration and share of frames past 2 mm, per hand and per dataset.

IX. CONCLUSION

The binding constraint on this field is not ideas. It is verification. Sixty-two method papers released code that could be read against the paper, 38 of those record a discrepancy, and nine are contradictions where the shipped code states a different objective from the published one. The first count was sixteen. An adversarial re-reading withdrew seven accusations, and an eighth fell later still: writing to OmniH2O’s [39] authors sent someone back to its evidence, and the reward-weight discrepancy it rested on turned out to be a typo signature in the paper’s own table, not a different trained objective. Each withdrawal is recorded in the row beside the charge. Nine is a floor, because forty-six method rows released nothing to check. A reward table in a paper is a claim about a document, not about a run. PhysHOI [1] is the case to remember, because the term its table weights at 0.1 is set to zero in the code, and its own success criterion could not have detected that.

The second finding is that the quantity most specific to dexterous manipulation is the one closed-loop policies do not record. Contact is what separates a hand from a gripper. Eleven of the 96 method rows whose notes settle the question address interpenetration at all, only four inside a closed-loop policy, and we found not one that reports a penetration number for its own policy’s rollouts. Grasp synthesis and hand-object reconstruction have reported penetration comparatively for years, so what is missing is not the measure but the measurement of a trained policy’s own behaviour. The obstacle is not the engines. IsaacGymEnvs ships a task that computes a per-environment maximum interpenetration depth against meshes and gates the policy update on a 1 mm threshold, and Tactile Genesis [103] offers penetration depth on Genesis geometry as a sensor. The tooling sits in the field’s own benchmark repository and the number is still not reported. DexTrack [104] has the formula and points it at its inputs.

The third is that hardware and software have come apart, on a narrower claim than the hand count first suggests. Table IV and Table V hold 33 hands and 18 appear in no method row. 11 of those 18 are neither sold nor open and appear in none for that reason, which leaves 7 hands that can be bought today or built from published designs and that take zero method rows between them. Eight of the fourteen generalist policies that settle the question do evaluate on a dexterous hand, at a median of 6 degrees of freedom against 16 across the reinforcement-learning rows.

What this survey cannot establish is which method is better than which. It re-runs nothing, and Section 7 argues that the published numbers do not compare. Five works are cited by metadata only and no claim rests on them. Every coverage

statistic here counts what this survey’s extraction captured rather than what the literature reported. Each is a floor and not a rate, because every miss converts a reporting paper into a silent one.

Three things to do next week, cheapest first.

If you are publishing, generate the reward table from the config that trained the reported run, print it, and cite the commit. State the trial count, state the success predicate, and release the per-trial outcomes. None of that needs a GPU.

If you are running experiments, measure penetration depth over your evaluation rollouts on a dense surface sample, and never let that measure become a reward. TopoRetarget [7] shows what the number looks like when someone takes it seriously, and what the widely used retargeters look like when nobody does.

If you are choosing hardware, the corpus names four hands and no more. The Allegro carries 35 method rows, the Shadow 21, the Inspire 19 and LEAP 12, and every other hand in Table IV and Table V carries eight rows or fewer. An announced hand has no URDF, no datasheet that can be checked, and no paper in this corpus that used it.

APPENDIX A METHOD

The survey is written from a corpus that lives on disk. No claim about a paper is made from memory. Each claim traces to a note, each note to a parsed markdown file, and each markdown file to a PDF or repository with a recorded hash or commit.

A. Selection

Six topic-specific bibliographies were assembled in parallel (simulators and physics, hands and vendors, reinforcement learning, imitation and human data, bimanual, benchmarks and evaluation), seeded with the canonical works in each area and extended by search up to 2026-09-17. Every arXiv identifier was checked by fetching the abstract page and matching the title, and entries that could not be checked that way are marked. The six lists were merged with deduplication on arXiv identifier and normalised title, giving 221 entries.

Seven further bibliography entries are not part of that corpus and are not counted anywhere in this survey. They are the prior audits and case studies Section I positions this survey against, they carry the topic related in `corpus/bib_related.json`, and none of them carries a structured row. No source for them was parsed either, so each is quoted only from its abstract and its stated method.

B. Acquisition

PDFs were downloaded from arXiv or, for work without a preprint, from the publisher or vendor page recorded in the bibliography. Vendor pages for hands without any paper were fetched as HTML and converted to markdown, so that a specification quoted in this survey is quoted from a stored copy of the page and dated. Repositories were shallow-cloned and parsed, then deleted. What remains is one markdown per

repository holding the README, a pruned file tree, the task and reward configuration files, and the bodies of reward and observation functions.

C. Parsing

PDFs were converted with `pymupdf4llm`, falling back to raw text extraction when the layout parse returned too little. The corpus manifest records, per entry, the source URL, page count, sha256 of the PDF and the converter used. Two consequences matter for reading this survey. Equations rendered as images do not survive conversion, so where a reward weight exists only inside a figure it is recorded as unreadable rather than guessed. Tables are flattened, so a number taken from a table is quoted with the table it came from and, where the flattening is ambiguous, the ambiguity is stated.

D. Notes

Each entry was read into a structured note under a fixed template: embodiment, learning method, objective, contact handling, evaluation, reproducibility, stated limitations, and quotable claims. Notes were written only from the parsed files. Where the source is silent the note says “not stated”, and nothing was inferred from the reviewer’s prior knowledge of the work. For method papers the reward or loss was quoted from the paper and, separately, from the released code, so that disagreements between the two are visible rather than smoothed over. Those disagreements turned out to be common enough to become a finding in their own right.

E. Sources that could not be obtained

Six works are cited by metadata only and no claim in this survey rests on their contents. Five are behind publisher paywalls with no author-hosted copy found on 2026-09-18: Okamura et al. 2000, Bicchi 2000 (IEEE T-RO), Piazza et al. 2019, Roa and Suarez 2015, and Butterfass et al. 2001 on DLR-Hand II. The sixth is Hwangbo et al. 2018 on RaiSim, whose hosted PDF returned no body. The freely circulating PDF often taken for Bicchi 2000 is a different work, a book chapter, and is listed separately.

Ma and Dollar 2011 is a seventh case with a different cause. The fetch that failed when its note was written succeeded on 2026-09-18, so a seven-page PDF and its hash are in the manifest, but no note has been read from it. It is cited by metadata only for that reason and not because the source is unavailable.

Three of the 221 bibliography entries carry no structured row. Two are the paywalled Bicchi 2000 and DLR-Hand II entries. The third is [2], which was read into a note and quoted throughout but is a book chapter rather than a work with an embodiment, a method or a result to record in a row.

F. What this method cannot do

Mention counts over the corpus are counts of mentions, not of use: a related-work sentence counts the same as an experiment. Vendor specifications are manufacturer claims and are labelled as such throughout, and where a page has since gone offline

the note says so. The corpus is large but not exhaustive, and selection by search favours work that is indexed, in English, and posted as a preprint. It also favours recent work. Of the 221 bibliography entries, 138 are dated 2024 or later and 16 predate 2018, so this is a corpus of the learned era and any claim here about a trend over time is a claim about 2022 onward.

Every coverage statistic in this survey measures what this survey’s extraction captured, not what the literature reported. A structured row holds a scalar. A paper that reports a per-task count, a rubric, or a total spread across several tables produces a null, and a null is then counted as silence. The bias runs one way. Every miss converts a reporting paper into a silent one, so the field is made to look worse at reporting than it is. The size of the effect was measured on the statistic the survey leads with. Of the 34 method rows that had a real robot and no recorded trial count, 15 carried a count in plain text in their own note, dropped because the paper reports it per task and the field takes a single integer. Those 15 have since been re-extracted, which moved the stated-trial-count row from 55 to 70. The success criterion and the unseen-object count were audited the same way, rising from 79 to 98 and from 32 to 39. The same mechanism reaches the penetration, code-release and failure-mode fields, none of which has been audited that way. Read every coverage statistic in this survey as a floor rather than as a rate, and read the bars in Fig. 24 the same way.

G. Where the tabulation lives

Most of this survey’s tabulation is not printed in this paper. The 112 method rows, the 21 reward-term objectives scored against their released code, the 30 teleoperation systems, the 33 hand specifications with the source sentence behind each one, the 15 contact-model and solver descriptions verbatim, and what each of the 14 predecessor surveys covers: all of it is reference material, consulted a row at a time and never read through. Printed, it ran to more pages than the argument it supports. It is in the repository instead.

Read it there in one of three forms. `corpus/rows/` holds one JSON record per work, with every field at full length and no cell cut; this is the form to query, and it is the form every number in this paper is computed from at build time. `paper/tables/` holds the same content as markdown tables, one file per table, for reading rather than querying. `papers/notes/` holds the structured note each row was read from, with the quotation that supports each field, so that a cell can be traced to a sentence in a parsed source. `tools/` holds the generators: every table in this paper and in the repository is written by a script there from `corpus/rows/`, so a reader who disagrees with a classification can change it and rebuild rather than argue with a PDF.

The repository is public, and the archived release this paper is submitted against will carry a DOI, which is the copy to cite for any number quoted here. A survey that argues for checkable artefacts should point at its artefact rather than reprint it, and a table a reader cannot recompute is worth less than the record it was compressed from.

Three tables are exceptions and are printed, because the prose reads individual cells out of them and does not stand

up without them: the two hand tables and the engine table, in Appendix B. Two more are the survey’s own proposal rather than a reading of a source, and are in the body where they are argued for: the evaluation protocol and the empty results matrix.

H. Reproduced figures

Some figures in this edition are not this survey’s own artwork. They are photographs of hardware and screenshots of results reproduced from the papers and vendor pages they discuss, rendered from the stored PDFs in the corpus, and each one states its source in its own caption, keyed to the corpus entry it came from and to the page it was taken from. No permission to reproduce any of them has been sought yet, from either an author or a publisher, and arXiv’s own licence does not by itself grant reuse. Every reproduced figure is therefore to be read as permission pending, and the register that records, per figure, the licence or policy it would rest on and whether the figure should instead be redrawn or dropped, is `tex/PERMISSIONS.md`. The build does not enforce that register.

APPENDIX B

THE HARDWARE AND ENGINE TABLES

Sections III and IV argue over individual cells of three tables: which column is emptiest, which cell is blank and why, which figure rests on a press release. Those three are set here rather than in the body, because a reader consults them rather than reading them through. Every other tabulation in this survey is in the repository, and Appendix A says where.

Nothing in these tables is a measurement by this survey. Every specification is a manufacturer claim, a figure in a paper, or a figure in a press article. Table IV and Table V split the 33 hand rows by whether the hand can be bought or built, and Table VI carries the 15 engines. The controlled vocabularies in all three compress a sentence the corpus stores in full: `corpus/rows/` holds the actuation, tactile, force-kind, contact-model and solver text at its own length, with the source’s own punctuation and wording, and `papers/notes/` holds the quotation each was read from.

Each caption states how many of its own cells no source filled. Two conventions behind those counts are worth stating once. A blank price is a hand with no public list price rather than a free hand; and a hand with no entry in the manifest was recorded from a page that could not be stored, so its row should be read as a claim with no retrievable source.

APPENDIX C

PER-PAPER REWARD TERM EXTRACTION

Section V marks nine recurring term families across the in-hand reorientation methods, from a matrix tabulated in the repository. This appendix is the extraction underneath that matrix, over every method row that records a reward at all. `terms stated` counts the terms the paper itself names, where the reward was read gives the section of the paper or the file in the released code that the note quotes. A row with a `paper/code class` is one where the two disagree, and the second part of this appendix prints the disagreement

in full. Cells and entries reproduce the text stored in the row, with the source’s own punctuation and wording.

A. The extraction

The extraction itself is a table of 77 rows and ten columns, and it is in the repository rather than here, with the rest of the tabulation Appendix A describes: as structured rows in `corpus/rows/` and as a markdown table in `paper/tables/`. Three things are worth knowing before reading it. 253 of its 770 cells are values no source stated. A blank `terms stated` with a filled `term names` column is a paper that names its terms without numbering them. And its `in Table 5` column marks the rows that are also in the reward-term matrix, which covers in-hand reorientation only.

B. Paper against released code, in full

Each entry below is the disagreement text stored in the row, unedited. The class is what Sec. 5.8 and Sec. 8.1 count. `contradiction` means the paper states one value and the shipped code demonstrably states another. `parse-limitation` means this survey’s own parse could not settle it and the accusation is withdrawn. `code-absent` means the described component is not in the released repository. `version-skew` means the repository is a later generation than the paper. `internal-inconsistency` means the paper disagrees with itself and no code is implicated.

contradiction, 9 rows.

- PDDM [116] (high). Table 2 states `obs-dim 46` for In-hand Reorientation while the released `cube_env.py` code sums to 39; the Baoding reward code includes an extra `-10*wrist_too_high` term absent from Table 2’s printed formula.
- DexPoint [48] (high). The released code’s reward adds several terms absent from the paper’s four-term Eq. 5 (a lift-threshold bonus, a target-distance term, a rotation bonus, and an IK controller-tracking penalty) and reshapes the reach/lift terms into inverse-distance and clipped forms rather than the paper’s plain distance/height-difference formulas.
- DeXtreme [26] (high). Action Delta Penalty weight is `-0.25` in paper Table 2 but `-0.2` in the ADR yaml and `-0.01` in the ManualDR yaml; the Joint Velocity Penalty in code normalises velocity by `(max_velocity-vel_tolerance)` unlike the paper’s stated formula; code has a `timeout_rew` term absent from the paper’s reward table; Appendix Table 12 states critic learning rate `5e-4` and KL threshold `0.16`, vs body text/code values of `5e-5` and `0.016`.
- Visual Dexterity [28] (high). Table S1 states 32000 teacher training environments, but the released config sets `alg.num_envs` to 8000 (parent config 16384); `fallDistance` differs across two shipped configs (`0.24` vs `0.15`, only the latter matching Table S1’s threshold); the paper’s Eq 8 penultimate-joint penalty (`c7=-2`) does not appear anywhere in the released reward code; the config carries a dead `distRewardScale=-10.0` key never used in `compute_reward`; and the paper’s table-friction lower bound (`0.05`)

TABLE IV: Hands that can be obtained: sold, open-source, or with no release status stated. 20 rows, sorted by joint count. 68 of the 160 cells in the 8 specification columns (DoF through price) (42%) are -: no source stated the value.

hand	maker	DoF	act.	actuation	mass	force	force kind	tactile	price	open	status	uses
			DoF		g	N			USD	HW		
Shadow Dexterous Hand [62]	Shadow Robot Co.	24	20	tendon	4 300	–	–	optional	–	no	sold	21
Proception ProHand 1.0 [186]	Proception Inc	22	–	tendon	–	–	–	unstated	–	no	sold	0
SharpaWave 22-DoF [78]	Sharpa Robotics	22	22	–	1 300	20 / 12	grip	unstated	–	no	sold	6
BiDexHand [70]	Northwestern Univ.	21	16	tendon	–	2.14	fingertip normal	none	–	yes	open-source	0
Ruka-v2 [69]	NYU and NYU Shanghai	20	16	tendon	–	–	–	optional	1 500	yes	open-source	0
TESOLLO DG-5F [94]	Tesollo Inc.	20	20	direct drive	1 763	–	–	none	–	no	sold	0
Unitree Dex5-1 [93]	Unitree Robotics	20	16	tendon	1 100	10	fingertip normal	array	–	no	sold	0
Wuji Hand [79]	Wuji Tech.	20	20	direct drive	–	–	–	fingertip	–	no	sold	2
ORCA [71]	ETH Zurich	17	17	tendon	1 200	–	–	fingertip	–	yes	open-source	0
Allegro Hand V4 [72]	Wonik Robotics Co. Ltd.; earlier versions...	16	16	–	–	–	–	none	–	no	sold	35
LEAP Hand [61]	CMU	16	16	direct drive	595	19.5	pull-out	none	2 000	yes	open-source	11
RUKA [73]	NYU	15	11	tendon	–	2.74	pinch	none	1 300	yes	open-source	0
Inspire Robots RH56DFX [63]	Beijing Inspire Robots Tech. Co., Ltd.	12	6	–	540	10	vendor, no protocol	none	–	no	sold	19
ROBOTERA XHAND1 [187]	ROBOTERA	12	12	geared	1 100	–	–	unstated	14 000	no	sold	8
DEX-EE [188]	Shadow Robot Co., in collaboration with...	12	–	–	4 100	–	–	unstated	–	no	sold	1
BrainCo Revo 2 [189]	BrainCo Inc.	11	6	–	383	15	pinch	unstated	–	no	sold	1
AgiBot OmniHand 2025 [190]	–	–	–	–	–	–	–	–	–	–	unstated	1
DexHand [64]	IoT Design Shop	–	–	tendon	–	–	–	none	300	yes	open-source	0
LinkerBot Linker Hand [191]	LinkerBot	–	–	–	–	–	–	unstated	–	no	sold	1
PSYONIC Ability Hand [192]	PSYONIC	–	6	linkage	–	–	–	unstated	–	no	sold	7

A row whose joint count no source states sorts last rather than being ranked by a number nobody stated. The hand, maker, open-hardware, status and usage columns are left out of the emptiness count because they are never blank, and counting them would dilute the finding. *actuation*, *force kind* and *tactile* are controlled vocabularies; the source sentence each was read from is in *corpus/rows/*. *force N* is not a ranking: read *force kind* first, because pull-out resistance, pinch force, a fingertip normal force under an indenter and an unlabelled vendor number are different measurements. *uses* counts the method rows whose own experiments run on that hand. Every figure is the maker's or the authors' own claim.

differs by a factor of 10 from the code's randomized lower bound (0.005).

- DexPBT [117] (high). Paper presents the reward as 4 mutually exclusive stage terms (r_{reach} , r_{pick} , r_{targ} , $-r_{\text{vel}}$), but code's `compute_kuka_reward` sums 8 named components, one of which (`hand_delta_penalty`) is multiplied by 0 and disabled; there is no single r_{vel} term in code, instead separate `kuka/allegro` action penalties whose exact formula is not shown. Also, the paper reports zero experiments with domain randomization, yet the shipped `AllegroKuka.yaml` already carries a fully specified DR schedule (disabled via `randomize: False`).
- Review: R3 adversarial review: the disabled-randomisation half is withdrawn, since the note finds it consistent with the paper; the zeroed reward term stands
- PhysHOI [1] (high). The released code hardcodes the body

position-velocity error and the object rotation/rotation-velocity errors to zero in `compute_humanoid_reward`, so despite Table 4 listing nonzero $\lambda^{or}=0.1/\lambda^{orv}=0.01$ weights for GRAB, the trained reward never actually tracks object orientation (position-only in practice).

- UniDexGrasp [156] (high). The paper describes a four-term weighted reward ($r_{\text{goal}} + r_{\text{reach}} + r_{\text{lift}} + r_{\text{move}}$ via Table 7's omega weights) but the released `compute_hand_reward` implements a different threshold-gated torch.where cascade with distinct hardcoded coefficients that do not map one-to-one onto the paper's weights.
- PenSpin [88] (medium). The appendix states a randomised disturbance force and the released task config sets its scale to zero. A second half of the original charge, that the released code disables the paper's tactile observation channel, is withdrawn: the config read has 96 observation

TABLE V: Hands announced, prototyped or held internally, which cannot be bought or built. 13 rows, the columns of Table IV plus the date of the claim and the class of artefact it was read off. 60 of the 104 cells in the 8 specification columns (DoF through price) (58%) are –: no source stated the value.

hand	maker	DoF	act. DoF	actuation	mass g	force N	force kind	tactile	price USD	open HW	status	uses	claim date	evidence
Clone Robotics [76]	Clone Robotics	27	–	fluidic muscle	<910	–	–	array	–	no	internal-only	0	2025-11-24	video
1X NEO [74]	1X Tech.	25	25	tendon	–	45	vendor, no protocol	unstated	–	no	announced	0	2026-07-09	datasheet
LEAP Hand V2 [193]	CMU	21	17	tendon	–	–	–	none	3 000	yes	announced	2	undated	datasheet
Sanctuary AI Phoenix [194]	Sanctuary AI	21	–	fluidic muscle	–	–	–	unstated	–	no	internal-only	0	2024-12-17	press
ILDA hand [66]	Korea Univ.	20	15	linkage	1 100	34	fingertip normal	array	–	no	prototype	0	2021-11-05	datasheet
Pisa/IIT SoftHand [67]	Centro E. Piaggio, Univ.\ Pisa and IIT	19	1	tendon	–	–	–	none	–	no	prototype	0	2014	datasheet
Faive Hand [68]	ETH Zurich	16	11	tendon	1 100	–	–	none	–	–	prototype	1	2023	datasheet
Boston Dynamics [195]	Boston Dynamics	–	–	geared	–	–	–	unstated	–	no	prototype	0	2026-01-05	press
Daxo Robotics Muscle [65]	Daxo Robotics	–	–	tendon	750	–	–	–	–	–	prototype	0	2026-09-16	press
Figure 03 [77]	Figure AI	–	–	–	–	–	–	unstated	–	no	prototype	0	undated	press
PaXini DexH13 [196]	PaXini Tech	–	–	–	–	–	–	array	–	–	announced	0	2026-01-07	press
Tesla Optimus [75]	Tesla	–	–	tendon	–	–	–	unstated	–	no	prototype	0	2023-12-13	video
Xiaomi [197]	Xiaomi	–	–	tendon	–	–	–	unstated	–	no	prototype	0	2026-03-30	press

That share is far higher than in Table IV and it is the finding of this table rather than a defect of it: an unreleased hand is announced without a specification. *evidence* is *datasheet* for a paper, *datasheet* or vendor specification page, *video* for a demonstration reel, *press* for a press release or a third-party tracker. *claim date* prints *undated* where the source page carries no date of its own; a fetch date is not a claim date.

TABLE VI: 15 simulators and physics engines, with the contact formulation each names, whether it exposes interpenetration depth, and how many first-party hand models it ships. 40 of the 105 cells in the 7 property columns (38%) are –: no source stated the value.

engine	contact model	solver	diff.	GPU	dt (s)	penetration exposed	hands
Brax [198]	convex relaxation	explicit/spring	yes	yes	–	no	1
ComFree-Sim [102]	complementarity-free	closed form	–	yes	0.002	yes	–
Dojo [101]	hard NCP	interior point	yes	no	–	yes	–
Genesis [107]	soft	Newton	yes	yes	–	–	1
Isaac Gym [97]	convex relaxation	PGS	no	yes	0.008333	no	3
Isaac Lab [105]	rigid, model unstated	Gauss-Seidel	no	yes	–	no	4
ManiSkill3 [110]	convex relaxation	PGS	–	yes	–	–	5
MuJoCo [100]	complementarity	Newton	no	no	–	–	–
MuJoCo convex [99]	complementarity-free	closed form	no	no	0.01	–	–
MuJoCo Warp [106]	soft	PGS	no	yes	–	–	–
Newton [108]	soft	primal-dual	–	yes	–	–	1
Orbit [109]	convex relaxation	PGS	no	yes	–	no	1
PyBullet [199]	–	–	–	yes	–	–	–
RaiSim [200]	–	–	–	–	–	–	–
SAPIEN [201]	convex relaxation	PGS	–	no	–	–	–

contact model and *solver* are controlled vocabularies: *soft* for a compliant or regularised contact, *convex relaxation* for a convexified formulation, *complementarity* for an LCP, *hard NCP* for a nonlinear complementarity problem, *impulse* for a velocity-level update, and *rigid, model unstated* where the source describes collision geometry but names no contact law. The sentence each was read from runs to several hundred characters; *corpus/rows/* carries each one verbatim, because Section IV’s argument turns on its wording. *penetration exposed* is whether the engine makes interpenetration depth readable by a user’s code without patching it. *hands* counts first-party hand models in the parsed snapshot, and they are: *Brax* synthetic 4-fingered claw hand; *Genesis* Shadow Hand; *Isaac Gym* Shadow Dexterous Hand, Allegro Hand, TriFinger; *Isaac Lab* KUKA Allegro hand, ShadowHand, AbilityHand, Fourier GR1-T2 dex hands, Unitree Inspire hand, Unitree trihand; *ManiSkill3* Allegro Hand, Ability Hand, Inspire Hand, Delta 3-finger hand, TriFinger; *Newton* Allegro Hand; *Orbit* Allegro hand.

dimensions and `enable_tactile`: `False`, consistent with the proprioception-only student policy rather than the tactile-and-point-cloud oracle, and the paper never claims the student has tactile input. The code’s reward scale-key names (e.g. `rotate_reward_scale`, `pencil_z_dist_penalty_scale`) also do not 1:1 name-match the paper’s Table 4 weight list, though matched values agree.

Review: Narrowed before author contact. The tactile-

channel half is withdrawn as a plausible reading of which pipeline stage the config belongs to; the disturbance-force half stands. Held at medium confidence pending a direct code read.

- PianoMime [43] (high). Paper’s Table 3 states 2 weighted reward terms (Key Press 2/3, Mimic 1/3), but the released code sums roughly 5 unweighted terms (key press doubled, sustain, energy and fingering hardcoded to return 0,

forearm-collision) plus a separately-added mimic wrapper term.

internal-inconsistency, 4 rows.

- ALOHA [133] (high). Algorithm 1 pseudocode states $L_{\text{reconst}} = \text{MSE}$, but Sec.IV.C’s prose explicitly states L_1 loss is used instead; the algorithm box and the implementation text disagree; code/md captures only function signatures, not bodies, for policy.py’s loss implementation
- 3D Diffusion Policy [135] (medium). the paper’s prose states the network predicts the noise added to the data, but the shipped default config trains with prediction_type: sample (predicting the denoised action a^0 directly), not epsilon; the paper only qualifies this later in the same section (Fig. 7 ablates both).
- OmniH2O [39] (medium). stumble weight -0.00125 (paper) vs -1250 (code); max-feet-height sign/magnitude differ (+1000 paper vs -2500 code, a penalty not a bonus); paper’s $\exp(-c*///)$ form vs code’s $\exp(-\text{err}^2/\text{sigma})$; curriculum level-down threshold 40 (paper) vs 50 (code). *Review:* Withdrawn as a contradiction before author contact. Four of the five weight comparisons match the paper’s table to the digit once a systematic $\times 1.25$ curriculum factor is accounted for, and only the stumble weight differs, by a factor of about a million, which is a typo signature in the paper’s own table rather than evidence of a different trained objective. The hands in this work are also driven open-loop from VR pose, outside the policy and outside the reward, making it a poor fit for a dexterous-manipulation reward census regardless.
- DexMachina [30] (low). paper describes a plain weighted sum $\lambda_{\text{task}}*r_{\text{task}} + \lambda_{\text{imi}}*r_{\text{imi}} + \lambda_{\text{bc}}*r_{\text{bc}} + \lambda_{\text{con}}*r_{\text{con}}$ with unspecified weights; code implements a multiplicative task term with per-component beta decay, an unmentioned 0.1 force-penalty term, and curriculum-driven decay of auxiliary weights not described as such in the paper *Review:* R3 adversarial review: the multiplicative form credited to the code is printed in the paper itself

version-skew, 4 rows.

- Hora [15] (high). The parsed code commit’s joint-noise range, disabled default disturbance force, and default cube object type differ from the paper’s stated values; the code README itself says to use tag v0.0.1, not the parsed commit, to reproduce paper numbers.
- π_0 [33] (high). The released repo (openpi, commit 215abfb2) has evolved past this paper: it also documents pi0-FAST (autoregressive) and pi0.5 checkpoints/configs not described in this paper, which describes only the flow-matching pi0 and the non-VLM pi0-small ablation.
- GR00T N1.6 [40] (high). code/md is the current Isaac-GR00T main branch (N1.7 generation, commit 51d4c89f), not the N1.6 checkpoint; the repo names a separate n1d6 branch for N1.6 that was not fetched. The page states N1.6’s backbone is “an internal NVIDIA Cosmos-2B VLM variant,” but the repo’s own changelog states the Eagle backbone (nvidia/Eagle-Block2A-2B-v2) was used through N1.6 and only replaced by Cosmos-Reason2-2B

in N1.7 – whether the page’s Cosmos-2B variant is a third distinct model or the page is describing N1.7 under an N1.6 headline is not resolvable from what was fetched.

Review: The page attributes a Cosmos backbone to N1.6 while the repository changelog says N1.6 used Eagle and N1.7 made the switch, so the repository is a later generation than the page describes.

- GR00T N1 [151] (high). The parsed code repo (Isaac-GR00T, commit 51d4c89f) is a later N1.7 generation, not the paper’s GR00T-N1-2B: VLM backbone changed from the paper’s Eagle-2 to Cosmos-Reason2-2B (via Qwen3-VL); action horizon expanded from the paper’s stated $H=16$ to 40; state/action dimensions expanded from 29 to 132; DiT layers changed from 32 to 16; embodiment scope widened to include Unitree G1, AgiBot G1, and a ‘YAM’ arm not described in the paper.

code-absent, 8 rows.

- AsymDex [51] (medium). Code contains a GraspAndPlace-task reward variant (AllegroHandDualArmGraspAndPlaceAsymDex.py) with exponential hand-distance and block-alignment terms not one of the four BiDexHands tasks described in the paper body.
- DexMimicGen [55] (high). The environment code includes an unused ‘reward_shaping’ branch (if self.reward_shaping: pass) that is never described or used in the paper text; only the sparse binary completion reward is used, as the paper states.
- DrEureka [123] (high). The released DrEureka repo (code/md) contains only forward_locomotion/ and globe_walking/ trees; the LEAP-hand cube-rotation training/deployment code and the quoted Prompt-13 reward are the LEAP-hand authors’ own code, not included in the released repo, so the cube-rotation reward cannot be cross-checked against a code file.
- Open-TeleVision [126] (low). The paper’s prose states 25k training iterations, lr 5e-5, batch size 45, but the code repo’s example training command instead shows 50000 epochs and an explicit kl_weight=10 not mentioned anywhere in the paper text. *Review:* R3 adversarial review: a behaviour-cloning paper with no reward; the charge compared paper iterations to a README example’s epochs
- Being-H0 [202] (high). The actual downstream action-head module referenced in §4.3 (learnable query tokens, proprioceptive projector f_p , regression head f_r) is not present in the parsed code; the repo’s own TODO list marks ‘Training code and scripts’ as unreleased, so the action-chunk-length=16 value and exact proprioception-vector fields come only from inference/eval CLI examples, not a verifiable implementation.
- $\pi_{0.5}$ [34] (high). The paper describes a hybrid discrete-FAST-token-pretrain-then-flow-matching-post-train recipe (Eq. 1, alpha: 0 to 10), but the released repo states plainly it ‘currently only support[s] the flow matching head for both pi0.5 training and inference.’
- $\pi_{0.6}^*$ [35] (high). The general openpi framework (pi0/pi0.5/FAST, flow-matching action head,

ALOHA/DROID/LIBERO adapters) is released at commit 215abfb2, but the RECAP-specific pieces described in the paper – the advantage-conditioning input, the distributional value function, the RECAP training loop (Algorithm 1), and the espresso/laundry/box-assembly checkpoints and datasets – are not present in the parsed repo.

- Being-H0.5 [203] (high). Released code (commit e12ac44f) exposes only benchmark/inference/config scaffolding (policy.py, beingh_policy.py, dataset-transform configs). no MoT/MoF model definition, no rectified-flow action-expert source, and no config for any dexterous-hand embodiment; the only post-train configs present (libero_robotcasa.yaml, libero_all.yaml, robotcasa_human.yaml, so101_example.yaml) are all gripper-only, so the dexterous-hand claims cannot be cross-checked against released code.

parse-limitation, 13 rows.

- ArtiGrasp [157] (low). paper’s two regulariser weights ($w_{rh}=0.5$, $w_{ro}=0.2$) correspond to five separate velocity penalties in code $(-0.5, -0.2, -0.5, -0.5, -0.3)$; the fingertip weight 12.0 and lambda 5.0 in Table 6 do not appear in the yaml; reward weights differ between curriculum phase-1 and phase-2 configs though Table 6 reports only one set

Review: R3 adversarial review: the reward body is C++ and absent from the parse; the weight split is the paper’s documented curriculum

- Diffusion Policy [134] (high). the paper’s core loss equations (DDPM training loss, EBM/InfoNCE, score-matching) are unrecovered images in the parsed text, and code/md is a signature-only API map, so compute_loss’s body could not be verified from code; only the exact shipped noise-scheduler config (DDPMScheduler, 100 train steps) is confirmed.
- Eureka [27] (low). The released repo’s default eureka/cfg/config.yaml ships iteration:1, sample:3 (a fast-test default), not the paper’s reported experimental configuration of $N=5$ iterations $\times$ $K=16$ samples $\times$ 5 independent runs, and LLM-generated reward code itself is never checked into the repo (written at runtime to a gitignored outputs/ directory).

Review: R3 adversarial review: the repo README documents the paper’s defaults; the cited config block is a fast-test preset

- UniDexGrasp++ [23] (low). The paper’s four named reward weights (0.5, 0.1, 2, 10) numerically match literals in the released compute_hand_reward, but the code never labels them with the paper’s variable names, and the paper’s own display equations were dropped from the PDF, so the correspondence is inferred rather than confirmed by either source.
- BiDexHD [164] (high). Table 5 weights (w_r , w_t , w_l , w_4 , $\lambda_{w}=0.12$, $\lambda_{ft}=0.48$) cannot be matched to code reward lines because the code md truncates the reward function before stage-2/final sum; λ_w and λ_{ft} numerically match the code’s grasp-condition

thresholds (0.12, 0.12x4 fingers) rather than confirmed reward weights

- GraspXL [57] (low). The code splits the paper’s single $r_{reg} = -w_h/u_h/l^2 - w_o/u_o/l^2$ into four separately-weighted terms (wrist vel, wrist qvel, object vel, object qvel), and the floating-phase object-velocity coefficient (-1.5) does not match the paper’s stated $w_o=0.1$, so the regularization weighting cannot be reconciled 1:1 with Table 9.

Review: R3 adversarial review: the environment source is not in the parse and the note says the reward cannot be verified

- HumanPlus [38] (medium). code implements a superset of legged_gym reward terms (lin_vel_z, ang_vel_xy, orientation, base_height, torques, dof_vel, dof_acc, action_rate, collision, termination, dof/torque limits, tracking_lin/ang_vel, feet_air_time, stumble, stand_still, feet_contact_forces, target_jt) beyond the 8 terms documented in the paper’s Table 1
- Omnigrasp [22] (medium). The code’s compute_pregrasp_reward_time hard-codes w_{pos} , $w_{rot} = 0.9, 0.1$, silently overriding the rwd_specs weight arguments passed into the function; separately, the paper’s pre-grasp reward weights (w_{hp} , w_{hr}) are not resolvable from either the paper text or Appendix Table 7.
- ResDex [155] (medium). The base-policy and residual-policy task files implement different variants of compute_hand_reward (whether finger/hand-distance terms are included alongside goal_hand_rew/hand_up/bonus differs), a difference the paper’s Sec 4.1/4.4 text does not describe, and the released configs’ env count (4096 default) and iteration counts (2500/5000) do not match the paper’s stated 11,000 envs and 5,000/20,000 iterations.
- DexGraspVLA [21] (high). Hardware-interfacing code is explicitly withheld (“Due to intellectual property constraints, we are unable to open-source the hardware-related code”); only function signatures for the controller loss (compute_loss, noise_assignment) are present in code/md, not their bodies, so the MSE/Immiscible-Diffusion loss cannot be verified against code.
- DexTrack [104] (low). The paper gives one canonical reward-weight table (Table 3), but the released repo’s task-config YAMLS contain several divergent, unreconciled reward-coefficient sets across AllegroHandTracking/...TrackingGeneralist/other variants, and it is not identifiable from the parsed source which config produced the paper’s headline Table 1 numbers.
- DexVLA [58] (high). code/md only carries class/function signatures, not bodies, for the diffusion head (modeling_scaledp.py) and no config file with the α weight was captured; the paper’s diffusion loss and denoising process cannot be verified against code bodies. only the multi-head ScaledDP signature and a DDIM fp32 patch are available, so none of the paper’s tables can be reproduced from what was parsed.
- ManipTrans [29] (low). paper’s PPO learning rate (5e-4) and env count (4096) differ from the shipped README/yaml defaults (2e-4, 8192)

Review: R3 adversarial review: the cited values are a README override and an unused fallback; the repo’s own config matches the paper

APPENDIX D

THE EXISTING SURVEYS, AND WHAT EACH COVERS

Fourteen corpus entries are themselves surveys or engine-comparison studies. The repository sets them on one set of columns, recording what each work covers, not how well. This appendix reads those columns out in prose. Section 1 states what this survey adds over them and does not repeat the comparison here.

The survey of An et al. [3] is the closest in subject, covering imitation learning for multi-fingered hands by learning family, end-effector class and demonstration source. It gives reinforcement learning no taxonomy, treats bimanual work as a single “multi-agent” subsection, compares no simulator, and defines no evaluation metric. The survey of Welte et al. [204] finds only seven dexterous works that use interactive imitation learning and carries a fifteen-hand commercial table, with no bimanual section, no benchmark table and no contact modelling. The survey of Zhao et al. [205] supplies the standard sim-to-real split and predates GPU-parallel simulation. The survey of Firoozi et al. [206] has zero occurrences of bimanual, tactile or in-hand.

The survey of Bai et al. [47] spans all of manipulation across 212 pages. Its Sec. 4.3 on dexterous manipulation runs about 720 words, the third longest of its ten task subsections behind grasping and quadrupedal manipulation, which is a real treatment and not a passing mention. The difference is elsewhere. Its Sec. 1.2 lists dexterous manipulation among the topics that “existing surveys” cover from “narrower perspectives” and defers it to two of them.

The survey of Zhao et al. [19] is the most recent hand-centred one, with a 29-hand anatomy table and a task-by-paradigm taxonomy. Its Sec. IV-C names the reference-versus-rollout split that Section 1 credits it with, and its Sec. III-F is the one bimanual subsection, 16 cited works with no coordination analysis.

The review of Kaup et al. [16] is the predecessor closest to the engine comparison, and it reviews Brax, Chrono, Gazebo, MuJoCo, ODE, PhysX, PyBullet, Unity and Webots for reinforcement learning research. It scores each on documentation, model and environment creation, URDF and MJCF support, and readiness for multi-agent work. It runs no benchmark of its own and says so in its Sec. V, that implementing the same scenarios across nine engines “goes beyond the scope of this paper”. Its running bodies are ant-and-humanoid RL benchmarks rather than hands, and it discusses no timestep, no friction model, no contact formulation and no penetration.

The oldest rows carry the cost of the corpus. Okamura et al. and Piazza et al. [207], [208], and Roa and Suárez [179], are behind publisher paywalls with no author-hosted copy found on 2026-09-18, and Bicchi [209] is in the same position with no row at all. Ma and Dollar [210] are a different case. The fetch that failed when its note was written succeeded afterwards, so a seven-page PDF is on disk with a recorded hash, and no

note has been read from it. All five are cited by metadata only and nothing in this survey describes their contents. The open chapter [2] overlaps the paywalled Bicchi paper without being identical to it, so it is quoted in its own right.

REFERENCES

- [1] Wang et al., “PhysHOI: Physics-Based Imitation of Dynamic Human-Object Interaction,” arXiv:2312.04393, 2023.
- [2] Bicchi, “Robotic Grasping and Manipulation (book chapter),” in *RAMSETE: Articulated and Mobile Robotics, Springer LNCIS*. Springer, 2001.
- [3] An et al., “Dexterous Manipulation through Imitation Learning: A Survey,” arXiv:2504.03515, 2025.
- [4] Yang et al., “OakInk: A Large-scale Knowledge Repository for Understanding Hand-Object Interaction,” in *CVPR*, 2022.
- [5] Lin et al., “BiDexGrasp: Coordinated Bimanual Dexterous Grasps across Object Geometries and Sizes,” arXiv:2604.06589, 2026.
- [6] Shao et al., “Bimanual Grasp Synthesis for Dexterous Robot Hands,” *IEEE RA-L*, 2024.
- [7] Wu et al., “TopoRetarget: Interaction-Preserving Retargeting for Dexterous Manipulation,” arXiv:2606.16272, 2026.
- [8] Collberg and Proebsting, “Repeatability in Computer Systems Research,” *Communications of the ACM*, vol. 59, no. 3, 2016, related work outside this survey’s corpus; no source parsed, no structured row, and no count rests on it.
- [9] Xu et al., “Do Papers Tell the Whole Story? A Benchmark and Framework for Uncovering Hidden Implementation Gaps in Bioinformatics,” arXiv:2603.22018, 2026, related work outside this survey’s corpus; no source parsed, no structured row, and no count rests on it.
- [10] Baumgärtner and Gurevych, “SciCoQA: Quality Assurance for Scientific Paper-Code Alignment,” arXiv:2601.12910, 2026, related work outside this survey’s corpus; no source parsed, no structured row, and no count rests on it.
- [11] Engstrom, Ilyas, Santurkar, Tsipras, Janoos, Rudolph, and Madry, “Implementation Matters in Deep Policy Gradients: A Case Study on PPO and TRPO,” in *ICLR*, 2020, related work outside this survey’s corpus; no source parsed, no structured row, and no count rests on it.
- [12] McLean et al., “Meta-World+: An Improved, Standardized, RL Benchmark,” in *NeurIPS Datasets and Benchmarks*, 2025, related work outside this survey’s corpus; no source parsed, no structured row, and no count rests on it.
- [13] Knox, Allievi, Banzhaf, Schmitt, and Stone, “Reward (Mis)design for Autonomous Driving,” *Artificial Intelligence*, vol. 316, 2023, related work outside this survey’s corpus; no source parsed, no structured row, and no count rests on it.
- [14] Raff, “A Step Toward Quantifying Independently Reproducible Machine Learning Research,” in *NeurIPS*, 2019, related work outside this survey’s corpus; no source parsed, no structured row, and no count rests on it.
- [15] Qi et al., “In-Hand Object Rotation via Rapid Motor Adaptation,” in *CoRL*, 2022.
- [16] Kaup et al., “A Review of Nine Physics Engines for Reinforcement Learning Research,” arXiv:2407.08590, 2024.
- [17] Erez et al., “Simulation tools for model-based robotics: Comparison of Bullet, Havok, MuJoCo, ODE and PhysX,” in *ICRA*, 2015.
- [18] Le Lidec et al., “Contact Models in Robotics: a Comparative Analysis,” *IEEE T-RO*, 2024, 2023.
- [19] Zhao et al., “Towards Robotic Dexterous Hand Intelligence: A Survey,” arXiv:2605.13925, 2026.
- [20] Taheri et al., “GRAB: A Dataset of Whole-Body Human Grasping of Objects,” in *ECCV*, 2020.
- [21] Zhong et al., “DexGraspVLA: A Vision-Language-Action Framework Towards General Dexterous Grasping,” arXiv:2502.20900, 2025.
- [22] Luo et al., “Omnigrasp: Grasping Diverse Objects with Simulated Humanoids,” in *NeurIPS*, 2024.
- [23] Wan et al., “UniDexGrasp++: Improving Dexterous Grasping Policy Learning via Geometry-aware Curriculum and Iterative Generalist-Specialist Learning,” in *ICCV*, 2023.
- [24] Agarwal et al., “Dexterous Functional Grasping,” in *CoRL*, 2023.
- [25] OpenAI et al., “Learning Dexterous In-Hand Manipulation,” *IJRR*, 2020, 2018.
- [26] Handa et al., “DeXtreme: Transfer of Agile In-hand Manipulation from Simulation to Reality,” in *ICRA*, 2023, 2022.
- [27] Ma et al., “Eureka: Human-Level Reward Design via Coding Large Language Models,” in *ICLR*, 2024, 2023.

- [28] Chen *et al.*, “Visual Dexterity: In-Hand Reorientation of Novel and Complex Object Shapes,” *Science Robotics* 2023, 2022.
- [29] Li *et al.*, “ManipTrans: Efficient Dexterous Bimanual Manipulation Transfer via Residual Learning,” in *CVPR*, 2025.
- [30] Mandi *et al.*, “DexMachina: Functional Retargeting for Bimanual Dexterous Manipulation,” arXiv:2505.24853, 2025.
- [31] Huang *et al.*, “Dynamic Handover: Throw and Catch with Bimanual Hands,” in *CoRL*, 2023.
- [32] Frau-Alfaro *et al.*, “Learning Dexterous Object Handover,” in *RO-MAN*, 2025.
- [33] Black *et al.*, “pi0: A Vision-Language-Action Flow Model for General Robot Control,” arXiv:2410.24164, 2024.
- [34] Physical Intelligence, “pi0.5: a Vision-Language-Action Model with Open-World Generalization,” arXiv:2504.16054, 2025.
- [35] —, “pi*0.6: a VLA That Learns From Experience,” arXiv:2511.14759, 2025.
- [36] Kim *et al.*, “OpenVLA: An Open-Source Vision-Language-Action Model,” in *CoRL*, 2024.
- [37] Gemini Robotics Team, “Gemini Robotics 1.5: Pushing the Frontier of Generalist Robots with Advanced Embodied Reasoning, Thinking, and Motion Transfer,” arXiv:2510.03342, 2025.
- [38] Fu *et al.*, “HumanPlus: Humanoid Shadowing and Imitation from Humans,” in *CoRL*, 2024.
- [39] He *et al.*, “OmniH2O: Universal and Dexterous Human-to-Humanoid Whole-Body Teleoperation and Learning,” in *CoRL*, 2024.
- [40] NVIDIA, “GR00T N1.6: An Improved Open Foundation Model for Generalist Humanoid Robots,” https://research.nvidia.com/labs/gear/gr00t-n1_6/, 2025, company press release, December 2025, research.nvidia.com, fetched 2026-09-17.
- [41] Zakka *et al.*, “RoboPianist: Dexterous Piano Playing with Deep Reinforcement Learning,” in *CoRL*, 2023.
- [42] Zhao *et al.*, “RPIM: A Large-Scale Motion Dataset for Piano Playing with Bi-Manual Dexterous Robot Hands,” in *CoRL*, 2024.
- [43] Qian *et al.*, “PianoMime: Learning a Generalist, Dexterous Piano Player from Internet Demonstrations,” in *CoRL*, 2024.
- [44] Li *et al.*, “DexDeform: Dexterous Deformable Object Manipulation with Human Demonstrations and Differentiable Physics,” in *ICLR*, 2023.
- [45] Ferrari and Canny, “Planning optimal grasps,” in *ICRA*, 1992.
- [46] Pang *et al.*, “Global Planning for Contact-Rich Manipulation via Local Smoothing of Quasi-dynamic Contact Models,” *IEEE T-RO* 2023, 2022.
- [47] Bai *et al.*, “Towards a Unified Understanding of Robot Manipulation: A Comprehensive Survey,” arXiv:2510.10903, 2025.
- [48] Qin *et al.*, “DexPoint: Generalizable Point Cloud Reinforcement Learning for Sim-to-Real Dexterous Manipulation,” in *CoRL*, 2022.
- [49] Yin *et al.*, “Rotating without Seeing: Towards In-hand Dexterity through Touch,” in *RSS*, 2023.
- [50] Yang *et al.*, “AnyRotate: Gravity-Invariant In-Hand Object Rotation with Sim-to-Real Touch,” in *CoRL*, 2024.
- [51] —, “AsymDex: Asymmetry and Relative Coordinates for RL-based Bimanual Dexterity,” in *RSS 2025 WCBM workshop (spotlight)*, 2024.
- [52] Zhou *et al.*, “DyDexHandover: Human-like Bimanual Dynamic Dexterous Handover using RGB-only Perception,” arXiv:2509.17350, 2025.
- [53] Ding *et al.*, “Bunny-VisionPro: Real-Time Bimanual Dexterous Teleoperation for Imitation Learning,” arXiv:2407.03162, 2024.
- [54] Shaw *et al.*, “Bimanual Dexterity for Complex Tasks,” in *CoRL*, 2024.
- [55] Jiang *et al.*, “DexMimicGen: Automated Data Generation for Bimanual Dexterous Manipulation via Imitation Learning,” in *ICRA* 2025, 2024.
- [56] Chen *et al.*, “Towards Human-Level Bimanual Dexterous Manipulation with Reinforcement Learning,” in *NeurIPS Datasets and Benchmarks*, 2022.
- [57] Zhang *et al.*, “GraspXL: Generating Grasping Motions for Diverse Objects at Scale,” in *ECCV*, 2024.
- [58] Wen *et al.*, “DexVLA: Vision-Language Model with Plug-In Diffusion Expert for General Robot Control,” arXiv:2502.05855, 2025.
- [59] Atar *et al.*, “In-Hand Manipulation of Articulated Tools with Dexterous Robot Hands with Sim-to-Real Transfer,” arXiv:2509.23075, 2025.
- [60] Lin *et al.*, “Learning Visuotactile Skills with Two Multifingered Hands,” in *ICRA* 2025, 2024.
- [61] Shaw *et al.*, “LEAP Hand: Low-Cost, Efficient, and Anthropomorphic Hand for Robot Learning,” in *RSS*, 2023.
- [62] Shadow Robot Company, “Shadow Dexterous Hand (technical specification, Dec 2024),” https://shadowrobot.com/wp-content/uploads/2024/12/shadow_dexterous_hand_e_technical_specification_20241204.pdf, 2005, vendor product page, shadowrobot.com, fetched 2026-09-17.
- [63] Inspire Robots, “Inspire Robots RH56DFX dexterous hand (RH56 series, FTP tactile variants),” <https://en.inspire-robots.com/product/rh56dfx/>, 2023, vendor product page, en.inspire-robots.com, fetched 2026-09-17.
- [64] Knight (The Robot Studio) and Trevor Blank, “DexHand: An Open Source Dexterous Humanoid Robot Hand,” <https://www.dexhand.org/>, 2023, open-hardware project page, dexhand.org, fetched 2026-09-17.
- [65] Daxo Robotics, “Daxo Robotics Muscle V0 (120-actuator compliant tendon hand),” <https://www.daxo-robotics.com/>, 2025, company announcement, hand not released, daxo-robotics.com, fetched 2026-09-17.
- [66] Kim *et al.*, “Integrated linkage-driven dexterous anthropomorphic robotic hand,” *Nature Communications*, vol. 12, 2021.
- [67] Catalano *et al.*, “Adaptive synergies for the design and control of the Pisa/IIT SoftHand,” *IJRR*, vol. 33, no. 5, 2014.
- [68] Toshimitsu *et al.*, “Getting the Ball Rolling: Learning a Dexterous Policy for a Biomimetic Tendon-Driven Hand with Rolling Contact Joints,” in *IEEE Humanoids*, 2023.
- [69] Liu *et al.*, “Ruka-v2: Tendon Driven Open-Source Dexterous Hand with Wrist and Abduction for Robot Learning,” arXiv:2603.26660, 2026.
- [70] Weng, “BiDexHand: Design and Evaluation of an Open-Source 16-DoF Biomimetic Dexterous Hand,” in *ICRA Dexterity Workshop*, 2025.
- [71] Christoph *et al.*, “ORCA: An Open-Source, Reliable, Cost-Effective, Anthropomorphic Robotic Hand for Uninterrupted Dexterous Task Learning,” in *IROS*, 2025.
- [72] Wonik Robotics, “Allegro Hand V4 (Wonik Robotics),” <https://www.allegrohand.com/v4>, 2016, vendor product page, allegrohand.com, fetched 2026-09-17.
- [73] Zorin *et al.*, “RUKA: Rethinking the Design of Humanoid Hands with Learning,” in *RSS*, 2025.
- [74] 1X Technologies, “NEO’s Hands: An API to the Physical World (1X tendon-driven hand),” <https://www.1x.tech/discover/neos-hands>, 2026, company announcement, 1x.tech, fetched 2026-09-17.
- [75] Tesla, “Tesla Optimus hands: Gen 2 (11 DoF, in-hand actuators, 2023) and Gen 3 / V3 (22 DoF, forearm-actuated tendons, patents 2025-2026),” <https://image-ppubs.uspto.gov/dirsearch-public/print/downloadPdf/9505134>, 2025, company announcement and USPTO patent filings, hand not released, image-ppubs.uspto.gov, fetched 2026-09-17.
- [76] Clone Robotics, “Clone Robotics hydraulic Myofiber hand (27 DoF),” <https://www.theresarobotforthat.com/clones-robot-hand-has-artificial-muscles/>, 2024, company announcement, hand not released, thesarobotforthat.com, fetched 2026-09-17.
- [77] Figure AI, “Figure 03 hands (palm camera, 3 g fingertip sensitivity),” <https://www.figure.ai/news/introducing-figure-03>, 2025, company announcement, hand not released, figure.ai, fetched 2026-09-17.
- [78] Sharpa Robotics, “SharpaWave 22-DoF dexterous hand with Dynamic Tactile Array,” <https://www.sharpa.com/pages/wave>, 2026, vendor product page, sharpa.com, fetched 2026-09-17.
- [79] Wuji Technology, “Wuji Hand / Wuji Hand 2: 20-DoF direct-drive dexterous hand,” <https://docs.wuji.tech/docs/en/wuji-hand/latest/overview/>, 2025, vendor product page, docs.wuji.tech, fetched 2026-09-17.
- [80] Lambeta *et al.*, “DIGIT: A Novel Design for a Low-Cost Compact High-Resolution Tactile Sensor with Application to In-Hand Manipulation,” *IEEE RA-L*, 2020.
- [81] —, “Digitizing Touch with an Artificial Multimodal Fingertip,” arXiv:2411.02479, 2024.
- [82] XELA Robotics, “XELA uSkin magnetic 3-axis tactile skin (Allegro Hand fingertips),” [https://www.hannovermesse.de/apollo/hannover_messe_2022/obs/Binary/A1167836/XELA%20Robotics%20-%20uSkin%20Curved%20\(A4\).pdf](https://www.hannovermesse.de/apollo/hannover_messe_2022/obs/Binary/A1167836/XELA%20Robotics%20-%20uSkin%20Curved%20(A4).pdf), 2020, vendor product page, hannovermesse.de, fetched 2026-09-17.
- [83] Higuera *et al.*, “Sparsh: Self-supervised touch representations for vision-based tactile sensing,” in *CoRL*, 2024.
- [84] Liu *et al.*, “DexTeleop-0: Force-Aware Bimanual Dexterous Teleoperation with Ego-Centric Perception towards Shared Autonomy,” arXiv:2606.23431, 2026.
- [85] Xu *et al.*, “DexUMI: Using Human Hand as the Universal Manipulation Interface for Dexterous Manipulation,” in *CoRL*, 2025.
- [86] Yuan *et al.*, “Robot Synesthesia: In-Hand Manipulation with Visuotactile Sensing,” in *ICRA* 2024, 2023.
- [87] Qi *et al.*, “General In-Hand Object Rotation with Vision and Touch,” in *CoRL*, 2023.
- [88] Wang *et al.*, “Lessons from Learning to Spin ‘Pens,’” in *CoRL*, 2024.
- [89] Liu *et al.*, “DexNDM: Closing the Reality Gap for Dexterous In-Hand Rotation via Joint-Wise Neural Dynamics Model,” arXiv:2510.08556, 2025.
- [90] Xu *et al.*, “Dexplore: Scalable Neural Control for Dexterous Manipulation from Reference-Scoped Exploration,” in *CoRL*, 2025.

- [91] Figure AI, “Helix: A Vision-Language-Action Model for Generalist Humanoid Control,” <https://www.figure.ai/news/helix>, 2025, company blog post, February 2025, figure.ai, fetched 2026-09-17.
- [92] Wen *et al.*, “GR-Dexter Technical Report,” arXiv:2512.24210, 2025.
- [93] Unitree Robotics, “Unitree Dex5-1 / Dex5-1P dexterous hand,” <https://www.unitree.com/Dex5-1>, 2025, vendor product page, unitree.com, fetched 2026-09-17.
- [94] TESOLLO, “TESOLLO DG-5F / DG-5F-M / DG-5F-S humanoid robot hand,” <https://en.tesollo.com/wp-content/uploads/2025/02/TESOLLO-DG-5F-catalogue-En-2.pdf>, 2024, vendor product page, en.tesollo.com, fetched 2026-09-17.
- [95] Zakka *et al.*, “MuJoCo Playground,” arXiv:2502.08844, 2025.
- [96] Wan *et al.*, “DexReMoE: In-hand Reorientation of General Object via Mixtures of Experts,” arXiv:2508.01695, 2025.
- [97] Makoviychuk *et al.*, “Isaac Gym: High Performance GPU-Based Physics Simulation For Robot Learning,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [98] Castro *et al.*, “An Unconstrained Convex Formulation of Compliant Contact,” *IEEE T-RO* 2022, 2021.
- [99] Todorov, “Convex and analytically-invertible dynamics with contacts and constraints: Theory and implementation in MuJoCo,” in *ICRA*, 2014.
- [100] Todorov *et al.*, “MuJoCo: A physics engine for model-based control,” in *IROS*, 2012.
- [101] Howell *et al.*, “Dojo: A Differentiable Physics Engine for Robotics,” arXiv:2203.00806, 2022.
- [102] Borse *et al.*, “ComFree-Sim: A GPU-Parallelized Analytical Contact Physics Engine for Scalable Contact-Rich Robotics Simulation and Control,” arXiv:2603.12185, 2026.
- [103] Chung *et al.*, “Tactile Genesis: Exploring Tactile Sensors at Scale for Learning Dexterous Tasks,” arXiv:2606.22332, 2026.
- [104] Liu *et al.*, “DexTrack: Towards Generalizable Neural Tracking Control for Dexterous Manipulation from Human References,” in *ICLR*, 2025.
- [105] Mittal *et al.*, “Isaac Lab: A GPU-Accelerated Simulation Framework for Multi-Modal Robot Learning,” arXiv:2511.04831, 2025.
- [106] Google DeepMind and NVIDIA, “MuJoCo Warp: GPU-optimized version of the MuJoCo physics simulator,” https://github.com/google-deepmind/mujoco_warp, 2025, software release announcement, NVIDIA GTC 2025, github.com, no fetch date recorded.
- [107] Genesis Authors (Zhou *et al.*), “Genesis: A Generative and Universal Physics Engine for Robotics and Beyond,” <https://github.com/Genesis-Embodied-AI/Genesis>, 2024, software release page, github.com, no fetch date recorded.
- [108] NVIDIA, Google DeepMind, and Disney Research, “Newton: An open-source, GPU-accelerated physics simulation engine built upon NVIDIA Warp,” <https://github.com/newton-physics/newton>, 2025, software release announcement, Linux Foundation, github.com, no fetch date recorded.
- [109] Mittal *et al.*, “Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments,” *IEEE RA-L*, 2023.
- [110] Tao *et al.*, “ManiSkill3: GPU Parallelized Robotics Simulation and Rendering for Generalizable Embodied AI,” in *RSS* 2025, 2024.
- [111] Gu *et al.*, “ManiSkill2: A Unified Benchmark for Generalizable Manipulation Skills,” in *ICLR*, 2023.
- [112] Wang *et al.*, “TACTO: A Fast, Flexible, and Open-source Simulator for High-Resolution Vision-based Tactile Sensors,” *IEEE RA-L* 2022, 2020.
- [113] Si and Yuan, “Taxim: An Example-based Simulation Model for GelSight Tactile Sensors,” *IEEE RA-L* 2022, 2021.
- [114] OpenAI *et al.*, “Solving Rubik’s Cube with a Robot Hand,” arXiv:1910.07113, 2019.
- [115] Lin *et al.*, “Sim-to-Real Reinforcement Learning for Vision-Based Dexterous Manipulation on Humanoids,” in *CoRL*, 2025.
- [116] Nagabandi *et al.*, “Deep Dynamics Models for Learning Dexterous Manipulation,” in *CoRL*, 2019.
- [117] Petrenko *et al.*, “DexPBT: Scaling up Dexterous Manipulation for Hand-Arm Systems with Population Based Training,” in *RSS*, 2023.
- [118] Lin *et al.*, “Learning In-Hand Object Reaching to General 6D Poses,” arXiv:2609.13761, 2026.
- [119] Zhao *et al.*, “Closing the Reality Gap: Zero-Shot Sim-to-Real Deployment for Dexterous Force-Based Grasping and Manipulation,” arXiv:2601.02778, 2026.
- [120] Li *et al.*, “Towards Human-level Dexterous Teleoperation,” arXiv:2607.11481, 2026.
- [121] Bauza *et al.*, “DemoStart: Demonstration-led auto-curriculum applied to sim-to-real with multi-fingered robots,” arXiv:2409.06613, 2024.
- [122] Bhargwaj *et al.*, “ViserDex: Visual Sim-to-Real for Robust Dexterous In-hand Reorientation,” arXiv:2604.11138, 2026.
- [123] Ma *et al.*, “DrEureka: Language Model Guided Sim-To-Real Transfer,” in *RSS*, 2024.
- [124] Handa *et al.*, “DexPilot: Vision Based Teleoperation of Dexterous Robotic Hand-Arm System,” in *ICRA*, 2020.
- [125] Qin *et al.*, “AnyTeleop: A General Vision-Based Dexterous Robot Arm-Hand Teleoperation System,” in *RSS*, 2023.
- [126] Cheng *et al.*, “Open-TeleVision: Teleoperation with Immersive Active Visual Feedback,” in *CoRL*, 2024.
- [127] Yang *et al.*, “ACE: A Cross-Platform Visual-Exoskeletons System for Low-Cost Dexterous Teleoperation,” in *CoRL*, 2024.
- [128] Arunachalam *et al.*, “Holo-Dex: Teaching Dexterity with Immersive Mixed Reality,” in *ICRA* 2023, 2022.
- [129] Yin *et al.*, “Geometric Retargeting: A Principled, Ultrafast Neural Hand Retargeting Algorithm,” arXiv:2503.07541, 2025.
- [130] Zhang *et al.*, “DOGlove: Dexterous Manipulation with a Low-Cost Open-Source Haptic Force Feedback Glove,” in *RSS*, 2025.
- [131] Wang *et al.*, “DexCap: Scalable and Portable Mocap Data Collection System for Dexterous Manipulation,” in *RSS*, 2024.
- [132] Qiu *et al.*, “Humanoid Policy ~ Human Policy,” in *CoRL*, 2025.
- [133] Zhao *et al.*, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” in *RSS*, 2023.
- [134] Chi *et al.*, “Diffusion Policy: Visuomotor Policy Learning via Action Diffusion,” in *RSS*, 2023.
- [135] Ze *et al.*, “3D Diffusion Policy: Generalizable Visuomotor Policy Learning via Simple 3D Representations,” in *RSS*, 2024.
- [136] Fu *et al.*, “METIS: Multi-Source Egocentric Training for Integrated Dexterous Vision-Language-Action Model,” arXiv:2511.17366, 2025.
- [137] Ye *et al.*, “Dex1B: Learning with 1B Demonstrations for Dexterous Manipulation,” in *RSS*, 2025.
- [138] Qin *et al.*, “DexMV: Imitation Learning for Dexterous Manipulation from Human Videos,” in *ECCV* 2022, 2021.
- [139] Shaw *et al.*, “VideoDex: Learning Dexterity from Internet Videos,” in *CoRL*, 2022.
- [140] Mandikal and Grauman, “DexVIP: Learning Dexterous Grasping with Human Hand Pose Priors from Video,” in *CoRL* 2021, 2022.
- [141] Li *et al.*, “OKAMI: Teaching Humanoid Robots Manipulation Skills through Single Video Imitation,” in *CoRL*, 2024.
- [142] Lum *et al.*, “Crossing the Human-Robot Embodiment Gap with Sim-to-Real RL using One Human Demonstration,” arXiv:2504.12609, 2025.
- [143] Guzey *et al.*, “Bridging the Human to Robot Dexterity Gap through Object-Oriented Rewards,” in *ICRA* 2025, 2024.
- [144] Goswami *et al.*, “World Models for Learning Dexterous Hand-Object Interactions from Human Videos,” arXiv:2512.13644, 2025.
- [145] Liu *et al.*, “EgoZero: Robot Learning from Smart Glasses,” in *CoRL*, 2025.
- [146] Chen *et al.*, “Object-Centric Dexterous Manipulation from Human Motion Data,” in *CoRL*, 2024.
- [147] Hsieh *et al.*, “DexMan: Learning Bimanual Dexterous Manipulation from Human and Generated Videos,” arXiv:2510.08475, 2025.
- [148] Zhang *et al.*, “Dexora: Open-source VLA for High-DoF Bimanual Dexterity,” arXiv:2605.18722, 2026.
- [149] Zheng *et al.*, “EgoScale: Scaling Dexterous Manipulation with Diverse Egocentric Human Data,” arXiv:2602.16710, 2026.
- [150] Gemini Robotics Team, “Gemini Robotics: Bringing AI into the Physical World,” arXiv:2503.20020, 2025.
- [151] NVIDIA, “GR00T N1: An Open Foundation Model for Generalist Humanoid Robots,” arXiv:2503.14734, 2025.
- [152] Howell *et al.*, “Predictive Sampling: Real-time Behaviour Synthesis with MuJoCo,” arXiv:2212.00541, 2022.
- [153] Yin *et al.*, “DexterityGen: Foundation Controller for Unprecedented Dexterity,” in *RSS*, 2025.
- [154] Mu *et al.*, “DexImit: Learning Bimanual Dexterous Manipulation from Monocular Human Videos,” arXiv:2602.10105, 2026.
- [155] Huang *et al.*, “Efficient Residual Learning with Mixture-of-Experts for Universal Dexterous Grasping,” in *ICLR* 2025, 2024.
- [156] Xu *et al.*, “UniDexGrasp: Universal Robotic Dexterous Grasping via Learning Diverse Proposal Generation and Goal-Conditioned Policy,” in *CVPR*, 2023.
- [157] Zhang *et al.*, “ArtiGrasp: Physically Plausible Synthesis of Bi-Manual Dexterous Grasping and Articulation,” in *3DV* 2024, 2023.
- [158] Liu *et al.*, “RDT-1B: a Diffusion Foundation Model for Bimanual Manipulation,” in *ICLR* 2025, 2024.
- [159] Kareer *et al.*, “EgoMimic: Scaling Imitation Learning via Egocentric Video,” in *ICRA* 2025, 2024.
- [160] Bi *et al.*, “H-RDT: Human Manipulation Enhanced Bimanual Robotic Manipulation,” arXiv:2507.23523, 2025.

- [161] Chi *et al.*, “Universal Manipulation Interface: In-The-Wild Robot Teaching Without In-The-Wild Robots,” in *RSS*, 2024.
- [162] Yang *et al.*, “Bench2Dex: Benchmarking Visuo-Tactile Bimanual Dexterous Manipulation Across Dexterous Hands,” arXiv:2609.15726, 2026.
- [163] Jing *et al.*, “HumanoidGen: Data Generation for Bimanual Dexterous Manipulation via LLM Reasoning,” arXiv:2507.00833, 2025.
- [164] Zhou *et al.*, “Learning Diverse Bimanual Dexterous Manipulation Skills from Human Demonstrations,” in *AAAI 2026 (oral)*, 2024.
- [165] Lin *et al.*, “Twisting Lids Off with Two Hands,” in *CoRL*, 2024.
- [166] Liu *et al.*, “TACO: Benchmarking Generalizable Bimanual Tool-Action-Object Understanding,” in *CVPR*, 2024.
- [167] Tao *et al.*, “DexWild: Dexterous Human Interactions for In-the-Wild Robot Policies,” in *RSS*, 2025.
- [168] Fan *et al.*, “ARCTIC: A Dataset for Dexterous Bimanual Hand-Object Manipulation,” in *CVPR 2023*, 2022.
- [169] Zhan *et al.*, “OAKINK2: A Dataset of Bimanual Hands-Object Manipulation in Complex Task Completion,” in *CVPR*, 2024.
- [170] Banerjee *et al.*, “HOT3D: Hand and Object Tracking in 3D from Egocentric Multi-View Videos,” in *CVPR 2025*, 2024.
- [171] Fu *et al.*, “GigaHands: A Massive Annotated Dataset of Bimanual Hand Activities,” in *CVPR 2025 (highlight)*, 2024.
- [172] Yao *et al.*, “DexVerse: A Modular Benchmark for Multi-Task, Multi-Embodiment Dexterous Manipulation,” arXiv:2607.08751, 2026.
- [173] Pumacay *et al.*, “THE COLOSSEUM: A Benchmark for Evaluating Generalization for Robotic Manipulation,” in *RSS*, 2024.
- [174] Snyder *et al.*, “Beyond Binary Success: Sample-Efficient and Statistically Rigorous Robot Policy Comparison,” arXiv:2603.13616, 2026.
- [175] Kress-Gazit *et al.*, “Robot Learning as an Empirical Science: Best Practices for Policy Evaluation,” arXiv:2409.09491, 2024.
- [176] Zhou *et al.*, “AutoEval: Autonomous Evaluation of Generalist Robot Manipulation Policies in the Real World,” arXiv:2503.24278, 2025.
- [177] Badithela *et al.*, “Reliable and Scalable Robot Policy Evaluation with Imperfect Simulators,” arXiv:2510.04354, 2025.
- [178] Chen *et al.*, “ClutterDexGrasp: A Sim-to-Real System for General Dexterous Grasping in Cluttered Scenes,” in *CoRL*, 2025.
- [179] Roa and Suárez, “Grasp quality measures: review and performance,” *Autonomous Robots*, 2015, full text not obtained; cited by metadata only.
- [180] TRI LBM Team, “A Careful Examination of Large Behavior Models for Multitask Dexterous Manipulation,” arXiv:2507.05331, 2025.
- [181] Atreya *et al.*, “RoboArena: Distributed Real-World Evaluation of Generalist Robot Policies,” arXiv:2506.18123, 2025.
- [182] Zhang *et al.*, “UniDex: A Robot Foundation Suite for Universal Dexterous Hand Control from Egocentric Human Videos,” arXiv:2603.22264, 2026.
- [183] Arunachalam *et al.*, “Dexterous Imitation Made Easy: A Learning-Based Framework for Efficient Dexterous Manipulation,” in *ICRA 2023*, 2022.
- [184] Rajeswaran *et al.*, “Learning Complex Dexterous Manipulation with Deep Reinforcement Learning and Demonstrations,” in *RSS 2018*, 2017.
- [185] Li *et al.*, “Evaluating Real-World Robot Manipulation Policies in Simulation,” in *CoRL*, 2024.
- [186] Proception (YC), “Proception ProHand 1.0 (22-DoF tendon-driven hand),” <https://www.ycombinator.com/launches/R78-proception-the-dexterous-robot-hand-we-wish-existed>, 2026, company announcement, ycombinator.com, fetched 2026-09-17.
- [187] ROBOTERA, “ROBOTERA XHAND1 dexterous hand,” <https://humanoid.guide/product/xhand1/>, 2024, vendor product page, humanoid.guide, fetched 2026-09-17.
- [188] Shadow Robot Company and Google DeepMind, “DEX-EE: Shadow Robot hand developed with Google DeepMind,” https://shadowrobot.com/dex-ee_series/, 2024, vendor product page, shadowrobot.com, fetched 2026-09-17.
- [189] BrainCo, “BrainCo Revo 2 dexterous hand,” <https://www.brainco-hz.com/docs/revolimb-hand/en/revo2/parameters.html>, 2025, vendor product page, brainco-hz.com, fetched 2026-09-17.
- [190] AgiBot (Zhiyuan), “AgiBot OmniHand 2025 / OmniHand Pro 2025,” <https://store.agibot.com/products/omnihand-2025>, 2025, vendor product page, store.agibot.com, fetched 2026-09-17.
- [191] LinkerBot, “LinkerBot Linker Hand L20 (and O6),” <https://openlab.io/products/linker-hand-linkerbot-l20-21>, 2025, vendor product page, openlab.io, fetched 2026-09-17.
- [192] PSYONIC, “PSYONIC Ability Hand,” <https://www.psyonic.io/robots>, 2021, vendor product page, psyonic.io, fetched 2026-09-17.
- [193] Shaw and Pathak, “LEAP Hand V2 Advanced: Dexterous, Low-Cost Hybrid Rigid-Soft Hand for Robot Learning,” in *IEEE Humanoids*, 2025.
- [194] Sanctuary AI, “Sanctuary AI Phoenix hydraulic hand (21 DoF),” <https://sanctuary.ai/news/sanctuary-ai-demonstrates-in-hand-manipulation-capabilities-for-improved-general-pur>, 2024, company announcement, hand not released, sanctuary.ai, fetched 2026-09-17.
- [195] Boston Dynamics, “Boston Dynamics electric Atlas hands (7-DoF three-finger, production four-digit gripper),” <https://bostondynamics.com/blog/boston-dynamics-unveils-new-atlas-robot-to-revolutionize-industry/>, 2026, company announcement, hand not released, bostondynamics.com, fetched 2026-09-17.
- [196] PaXini Tech, “PaXini DexH13 tactile dexterous hand and multi-dimensional tactile sensors,” <https://humanoid.guide/product/dexh13/>, 2024, vendor product page, humanoid.guide, fetched 2026-09-17.
- [197] Xiaomi, “Xiaomi next-gen CyberOne bionic hand (22-27 DoF, liquid-cooled forearm),” <https://eu.36kr.com/en/p/3947315943472512>, 2026, company announcement, hand not released, eu.36kr.com, fetched 2026-09-17.
- [198] Freeman *et al.*, “Brax - A Differentiable Physics Engine for Large Scale Rigid Body Simulation,” in *NeurIPS Datasets and Benchmarks*, 2021.
- [199] Coumans and Bai, “PyBullet, a Python module for physics simulation for games, robotics and machine learning,” <https://github.com/bulletphysics/bullet3>, 2016, software documentation page, github.com, no fetch date recorded.
- [200] Hwangbo *et al.*, “Per-Contact Iteration Method for Solving Contact Dynamics,” *IEEE RA-L*, 2018, full text not obtained; cited by metadata only.
- [201] Xiang *et al.*, “SAPIEN: A Simulated Part-based Interactive ENvironment,” in *CVPR*, 2020.
- [202] Luo *et al.*, “Being-H0: Vision-Language-Action Pretraining from Large-Scale Human Videos,” arXiv:2507.15597, 2025.
- [203] —, “Being-H0.5: Scaling Human-Centric Robot Learning for Cross-Embodiment Generalization,” arXiv:2601.12993, 2026.
- [204] Welte *et al.*, “Interactive Imitation Learning for Dexterous Robotic Manipulation: Challenges and Perspectives – A Survey,” *Frontiers in Robotics and AI*, 2025.
- [205] Zhao *et al.*, “Sim-to-Real Transfer in Deep Reinforcement Learning for Robotics: a Survey,” in *IEEE SSCI*, 2020.
- [206] Firoozi *et al.*, “Foundation Models in Robotics: Applications, Challenges, and the Future,” *IJRR*, 2023.
- [207] Okamura, Smith, and Cutkosky, “An overview of dexterous manipulation,” in *ICRA*, 2000, full text not obtained; cited by metadata only.
- [208] Piazza *et al.*, “A Century of Robotic Hands,” *Annual Review of Control, Robotics, and Autonomous Systems*, 2019, full text not obtained; cited by metadata only.
- [209] Bicchi, “Hands for dexterous manipulation and robust grasping: a difficult road toward simplicity,” *IEEE Transactions on Robotics and Automation*, 2000, full text not obtained; cited by metadata only.
- [210] Ma and Dollar, “On dexterity and dexterous manipulation,” in *ICAR*, 2011.